claude-windows / 02b9e9c9-3806-45ca-9b24-2ecc35a81efd

Metadata

- Source: `claude-windows`
- Kind: `claude`
- Source file: `C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-muneem\02b9e9c9-3806-45ca-9b24-2ecc35a81efd.jsonl`
- SHA-256: `db841708bf65c41127c01da7de638b04a60512a426cd656cec5f7decc44ac9d1`
- Source modified: `2026-06-10T16:58:46+00:00`
- Imported at: `2026-07-05T16:48:20+00:00`
- project: `C--Users-avidu-Projects-muneem`
- session_id: `02b9e9c9-3806-45ca-9b24-2ecc35a81efd`

Transcript

1. user (2026-06-10T16:07:23.996Z)

Hello. Can you take deep read of this repository and its sister repository gmail-attachments-gdrive and identify any issues in code, architecture and roadmaps? Also want you to do a product market fit to analyse whether the direction is reasonable. Also identify key areas for improvements in both short term (code and components which you can fix right away) and longer horizon (product definitions, architectural evolution etc.) ? I am looking to decide whether it is better to focus on the grocery billing consolidation or the finance angle as a first step or maybe there are others which can be an easier/promising start.

2. assistant (2026-06-10T16:07:44.894Z)

I'll do this in phases: first scout both repos to scope the work, then run a multi-agent workflow for the deep review (code, architecture, security, roadmap), plus a market-landscape pass for the PMF question, and synthesize a recommendation at the end.

Let me start by checking the session handoff and the shape of both repos.

3. user (2026-06-10T16:07:45.893Z)

File does not exist. Note: your current working directory is C:\Users\avidu\Projects\muneem.

4. user (2026-06-10T16:07:46.742Z)

C:\Users\avidu\Projects\gmail-attachments-to-gdrive\.git\description
C:\Users\avidu\Projects\gmail-attachments-to-gdrive\.git\hooks\applypatch-msg.sample
C:\Users\avidu\Projects\gmail-attachments-to-gdrive\.git\hooks\commit-msg.sample
C:\Users\avidu\Projects\gmail-attachments-to-gdrive\.git\hooks\fsmonitor-watchman.sample
C:\Users\avidu\Projects\gmail-attachments-to-gdrive\.git\hooks\post-update.sample
C:\Users\avidu\Projects\gmail-attachments-to-gdrive\.git\hooks\pre-applypatch.sample
C:\Users\avidu\Projects\gmail-attachments-to-gdrive\.git\hooks\pre-commit.sample
C:\Users\avidu\Projects\gmail-attachments-to-gdrive\.git\hooks\pre-merge-commit.sample
C:\Users\avidu\Projects\gmail-attachments-to-gdrive\.git\hooks\pre-push.sample
C:\Users\avidu\Projects\gmail-attachments-to-gdrive\.git\hooks\pre-rebase.sample
C:\Users\avidu\Projects\gmail-attachments-to-gdrive\.git\hooks\pre-receive.sample
C:\Users\avidu\Projects\gmail-attachments-to-gdrive\.git\hooks\prepare-commit-msg.sample
C:\Users\avidu\Projects\gmail-attachments-to-gdrive\.git\hooks\push-to-checkout.sample
C:\Users\avidu\Projects\gmail-attachments-to-gdrive\.git\hooks\sendemail-validate.sample
C:\Users\avidu\Projects\gmail-attachments-to-gdrive\.git\hooks\update.sample
C:\Users\avidu\Projects\gmail-attachments-to-gdrive\.git\info\exclude
C:\Users\avidu\Projects\gmail-attachments-to-gdrive\.git\objects\pack\pack-112d4eba9cdc2a5df0efeb14af0da29c956eb96f.idx
C:\Users\avidu\Projects\gmail-attachments-to-gdrive\.git\objects\pack\pack-112d4eba9cdc2a5df0efeb14af0da29c956eb96f.pack
C:\Users\avidu\Projects\gmail-attachments-to-gdrive\.git\objects\pack\pack-112d4eba9cdc2a5df0efeb14af0da29c956eb96f.rev
C:\Users\avidu\Projects\gmail-attachments-to-gdrive\.git\logs\refs\remotes\origin\HEAD

```
C:\Users\avidu\Projects\gmail-attachments-to-gdrive\.git\packed-refs
C:\Users\avidu\Projects\gmail-attachments-to-gdrive\.git\refs\remotes\origin\HEAD
C:\Users\avidu\Projects\gmail-attachments-to-gdrive\.clasp.json
C:\Users\avidu\Projects\gmail-attachments-to-gdrive\.claspignore
C:\Users\avidu\Projects\gmail-attachments-to-gdrive\.git\config
C:\Users\avidu\Projects\gmail-attachments-to-gdrive\.git\HEAD
C:\Users\avidu\Projects\gmail-attachments-to-gdrive\.git\logs\HEAD
C:\Users\avidu\Projects\gmail-attachments-to-gdrive\.git\logs\refs\heads\master
C:\Users\avidu\Projects\gmail-attachments-to-gdrive\.git\refs\heads\master
C:\Users\avidu\Projects\gmail-attachments-to-gdrive\.github\workflows\ci.yml
C:\Users\avidu\Projects\gmail-attachments-to-gdrive\.gitignore
C:\Users\avidu\Projects\gmail-attachments-to-gdrive\appsscript.json
C:\Users\avidu\Projects\gmail-attachments-to-gdrive\docs\ARCHITECTURE.md
C:\Users\avidu\Projects\gmail-attachments-to-gdrive\docs\VISION.md
C:\Users\avidu\Projects\gmail-attachments-to-gdrive\esbuild.config.mjs
C:\Users\avidu\Projects\gmail-attachments-to-gdrive\eslint.config.mjs
C:\Users\avidu\Projects\gmail-attachments-to-gdrive\LICENSE
C:\Users\avidu\Projects\gmail-attachments-to-gdrive\PRIVACY.md
C:\Users\avidu\Projects\gmail-attachments-to-gdrive\README.md
C:\Users\avidu\Projects\gmail-attachments-to-gdrive\ROADMAP.md
C:\Users\avidu\Projects\gmail-attachments-to-gdrive\html\index.html
C:\Users\avidu\Projects\gmail-attachments-to-gdrive\jest.config.js
C:\Users\avidu\Projects\gmail-attachments-to-gdrive\package-lock.json
C:\Users\avidu\Projects\gmail-attachments-to-gdrive\package.json
C:\Users\avidu\Projects\gmail-attachments-to-gdrive\src\config.ts
C:\Users\avidu\Projects\gmail-attachments-to-gdrive\src\contentTypes.ts
C:\Users\avidu\Projects\gmail-attachments-to-gdrive\src\drive.ts
C:\Users\avidu\Projects\gmail-attachments-to-gdrive\src\entry.ts
C:\Users\avidu\Projects\gmail-attachments-to-gdrive\src\gmailHistory.ts
C:\Users\avidu\Projects\gmail-attachments-to-gdrive\src\historySync.ts
C:\Users\avidu\Projects\gmail-attachments-to-gdrive\src\logger.ts
C:\Users\avidu\Projects\gmail-attachments-to-gdrive\src\naming.ts
C:\Users\avidu\Projects\gmail-attachments-to-gdrive\src\notify.ts
C:\Users\avidu\Projects\gmail-attachments-to-gdrive\src\processed.ts
C:\Users\avidu\Projects\gmail-attachments-to-gdrive\src\query.ts
C:\Users\avidu\Projects\gmail-attachments-to-gdrive\src\quota.ts
C:\Users\avidu\Projects\gmail-attachments-to-gdrive\src\state.ts
C:\Users\avidu\Projects\gmail-attachments-to-gdrive\src\stats.ts
C:\Users\avidu\Projects\gmail-attachments-to-gdrive\src\sync.ts
C:\Users\avidu\Projects\gmail-attachments-to-gdrive\src\types\gmailAdvanced.d.ts
C:\Users\avidu\Projects\gmail-attachments-to-gdrive\src\ui.ts
C:\Users\avidu\Projects\gmail-attachments-to-gdrive\tests\auditability.test.ts
C:\Users\avidu\Projects\gmail-attachments-to-gdrive\tests\contentTypes.test.ts
C:\Users\avidu\Projects\gmail-attachments-to-gdrive\tests\drive.test.ts
C:\Users\avidu\Projects\gmail-attachments-to-gdrive\tests\gmailHistory.test.ts
C:\Users\avidu\Projects\gmail-attachments-to-gdrive\tests\historySync.test.ts
C:\Users\avidu\Projects\gmail-attachments-to-gdrive\tests\naming.test.ts
C:\Users\avidu\Projects\gmail-attachments-to-gdrive\tests\notify.test.ts
C:\Users\avidu\Projects\gmail-attachments-to-gdrive\tests\processed.test.ts
C:\Users\avidu\Projects\gmail-attachments-to-gdrive\tests\query.test.ts
C:\Users\avidu\Projects\gmail-attachments-to-gdrive\tests\quota.test.ts
C:\Users\avidu\Projects\gmail-attachments-to-gdrive\.git\index
C:\Users\avidu\Projects\gmail-attachments-to-gdrive\tests\setup.ts
C:\Users\avidu\Projects\gmail-attachments-to-gdrive\tests\stats.test.ts
C:\Users\avidu\Projects\gmail-attachments-to-gdrive\tests\sync.test.ts
C:\Users\avidu\Projects\gmail-attachments-to-gdrive\tests\syncIncremental.test.ts
C:\Users\avidu\Projects\gmail-attachments-to-gdrive\tests\ui.test.ts
C:\Users\avidu\Projects\gmail-attachments-to-gdrive\tsconfig.json
```

5. user (2026-06-10T16:07:47.230Z)

```
.venv\Scripts\python.exe
.venv\Scripts\pythonw.exe
.git\description
.git\hooks\applypatch-msg.sample
```

```
.git\hooks\commit-msg.sample
.git\hooks\fsmonitor-watchman.sample
.git\hooks\post-update.sample
.git\hooks\pre-applypatch.sample
.git\hooks\pre-commit.sample
.git\hooks\pre-merge-commit.sample
.git\hooks\pre-push.sample
.git\hooks\pre-rebase.sample
.git\hooks\pre-receive.sample
.git\hooks\prepare-commit-msg.sample
.git\hooks\push-to-checkout.sample
.git\hooks\sendemail-validate.sample
.git\hooks\update.sample
.git\info\exclude
.git\refs\remotes\origin\HEAD
testdata\promo-emails\dataquest.pdf
testdata\promo-emails\makemytrip-2.pdf
testdata\promo-emails\airasia-1.pdf
testdata\promo-emails\airasia-2.pdf
testdata\promo-emails\airasia-3.pdf
testdata\promo-emails\flipkart.pdf
testdata\promo-emails\makemytrip-1.pdf
testdata\promo-emails\swiggy.pdf
testdata\promo-emails\unacademy.pdf
testdata\promo-emails\tataneu.pdf
CLAUDE.md
testdata\promo-emails\README.md
docs\TESTING.md
.venv\.gitignore
.venv\pyvenv.cfg
.venv\Scripts\activate
.venv\Scripts\activate.fish
.venv\Scripts\Activate.ps1
.venv\Scripts\activate.bat
.venv\Scripts\deactivate.bat
.venv\Lib\site-packages\pip\__init__.py
.venv\Lib\site-packages\pip\__main__.py
.venv\Lib\site-packages\pip\py.typed
.venv\Lib\site-packages\pip\__pip-runner__.py
.venv\Lib\site-packages\pip\_internal\__init__.py
.venv\Lib\site-packages\pip\_internal\build_env.py
.venv\Lib\site-packages\pip\_internal\cache.py
.venv\Lib\site-packages\pip\_internal\configuration.py
.venv\Lib\site-packages\pip\_internal\exceptions.py
.venv\Lib\site-packages\pip\_internal\main.py
.venv\Lib\site-packages\pip\_internal\pyproject.py
.venv\Lib\site-packages\pip\_internal\self_outdated_check.py
.venv\Lib\site-packages\pip\_internal\cli\__init__.py
.venv\Lib\site-packages\pip\_internal\wheel_builder.py
.venv\Lib\site-packages\pip\_internal\cli\autocompletion.py
.venv\Lib\site-packages\pip\_internal\cli\base_command.py
.venv\Lib\site-packages\pip\_internal\cli\cmdoptions.py
.venv\Lib\site-packages\pip\_internal\cli\command_context.py
.venv\Lib\site-packages\pip\_internal\cli\index_command.py
.venv\Lib\site-packages\pip\_internal\cli\main.py
.venv\Lib\site-packages\pip\_internal\cli\main_parser.py
.venv\Lib\site-packages\pip\_internal\cli\parser.py
.venv\Lib\site-packages\pip\_internal\cli\progress_bars.py
.venv\Lib\site-packages\pip\_internal\cli\req_command.py
.venv\Lib\site-packages\pip\_internal\cli\spinners.py
.venv\Lib\site-packages\pip\_internal\cli\status_codes.py
.venv\Lib\site-packages\pip\_internal\commands\cache.py
.venv\Lib\site-packages\pip\_internal\commands\__init__.py
.venv\Lib\site-packages\pip\_internal\commands\check.py
.venv\Lib\site-packages\pip\_internal\commands\completion.py
```

```
.venv\Lib\site-packages\pip\_internal\commands\configuration.py
.venv\Lib\site-packages\pip\_internal\commands\debug.py
.venv\Lib\site-packages\pip\_internal\commands\download.py
.venv\Lib\site-packages\pip\_internal\commands\freeze.py
.venv\Lib\site-packages\pip\_internal\commands\hash.py
.venv\Lib\site-packages\pip\_internal\commands\help.py
.venv\Lib\site-packages\pip\_internal\commands\index.py
.venv\Lib\site-packages\pip\_internal\commands\inspect.py
.venv\Lib\site-packages\pip\_internal\commands\install.py
.venv\Lib\site-packages\pip\_internal\commands\list.py
.venv\Lib\site-packages\pip\_internal\commands\lock.py
.venv\Lib\site-packages\pip\_internal\commands\search.py
.venv\Lib\site-packages\pip\_internal\commands\show.py
.venv\Lib\site-packages\pip\_internal\commands\uninstall.py
.venv\Lib\site-packages\pip\_internal\commands\wheel.py
.venv\Lib\site-packages\pip\_internal\distributions\__init__.py
.venv\Lib\site-packages\pip\_internal\distributions\base.py
.venv\Lib\site-packages\pip\_internal\distributions\installed.py
.venv\Lib\site-packages\pip\_internal\distributions\sdist.py
.venv\Lib\site-packages\pip\_internal\distributions\wheel.py
.venv\Lib\site-packages\pip\_internal\index\__init__.py
.venv\Lib\site-packages\pip\_internal\index\collector.py
.venv\Lib\site-packages\pip\_internal\index\package_finder.py
.venv\Lib\site-packages\pip\_internal\index\sources.py
.venv\Lib\site-packages\pip\_internal\locations\__init__.py
.venv\Lib\site-packages\pip\_internal\locations\_distutils.py
.venv\Lib\site-packages\pip\_internal\locations\_sysconfig.py
.venv\Lib\site-packages\pip\_internal\locations\base.py
.venv\Lib\site-packages\pip\_internal\metadata\_json.py
.venv\Lib\site-packages\pip\_internal\metadata\__init__.py
.venv\Lib\site-packages\pip\_internal\metadata\base.py
(Results are truncated. Consider using a more specific path or pattern.)
```

6. user (2026-06-10T16:07:59.150Z)

```
.githooks\pre-commit
.githooks\pre-push
.githooks\precommit_check.py
.github\workflows\ci.yml
.mypy_cache\3.11\cache.0.db
.mypy_cache\3.11\cache.1.db
.mypy_cache\3.11\cache.10.db
.mypy_cache\3.11\cache.11.db
.mypy_cache\3.11\cache.12.db
.mypy_cache\3.11\cache.13.db
.mypy_cache\3.11\cache.14.db
.mypy_cache\3.11\cache.15.db
.mypy_cache\3.11\cache.2.db
.mypy_cache\3.11\cache.3.db
.mypy_cache\3.11\cache.4.db
.mypy_cache\3.11\cache.5.db
.mypy_cache\3.11\cache.6.db
.mypy_cache\3.11\cache.7.db
.mypy_cache\3.11\cache.8.db
.mypy_cache\3.11\cache.9.db
.mypy_cache\3.13\cache.0.db
.mypy_cache\3.13\cache.1.db
.mypy_cache\3.13\cache.10.db
.mypy_cache\3.13\cache.11.db
.mypy_cache\3.13\cache.12.db
.mypy_cache\3.13\cache.13.db
.mypy_cache\3.13\cache.14.db
.mypy_cache\3.13\cache.15.db
.mypy_cache\3.13\cache.2.db
.mypy_cache\3.13\cache.3.db
```

.mypy_cache\3.13\cache.4.db
.mypy_cache\3.13\cache.5.db
.mypy_cache\3.13\cache.6.db
.mypy_cache\3.13\cache.7.db
.mypy_cache\3.13\cache.8.db
.mypy_cache\3.13\cache.9.db
.mypy_cache\.gitignore
.mypy_cache\CACHEDIR.TAG
.pytest_cache\v\cache\lastfailed
.pytest_cache\v\cache\nodeids
.pytest_cache\.gitignore
.pytest_cache\CACHEDIR.TAG
.pytest_cache\README.md
.ruff_cache\0.15.15\12765879588015877657
.ruff_cache\0.15.15\13428912160643764434
.ruff_cache\0.15.15\4373846357469854159
.ruff_cache\0.15.15\5647337317940576954
.ruff_cache\0.15.15\9380428756468918620
.ruff_cache\0.15.15\9919018729046908979
.ruff_cache\.gitignore
.ruff_cache\CACHEDIR.TAG
docs\CURRENT_STATE.md
docs\DEPENDENCIES.md
docs\encryption-at-rest.md
docs\feedbackJune2026.md
docs\ingesting-statements.md
docs\onboarding-a-bank.md
docs\parser-architecture.md
docs\prior-art-email-mining.md
docs\research-open-banking-india.md
docs\ROADMAP.md
docs\schema.md
docs\TESTING.md
src\muneem\ingestion__init__.py
src\muneem\ingestion\base.py
src\muneem\ingestion\gmail.py
src\muneem\ingestion\imap.py
src\muneem\ingestion\outlook.py
src\muneem\parsers__init__.py
src\muneem\parsers\au.py
src\muneem\parsers\axis.py
src\muneem\parsers\base.py
src\muneem\parsers\clustering.py
src\muneem\parsers\concepts.py
src\muneem\parsers\engine.py
src\muneem\parsers\hdfc.py
src\muneem\parsers\profiles.py
src\muneem\parsers\savings.py
src\muneem\parsers\validation.py
src\muneem__init__.py
src\muneem\cli.py
src\muneem\config.py
src\muneem\db_key.py
src\muneem\db.py
src\muneem\errors.py
src\muneem\extract.py
src\muneem\log.py
src\muneem\migrations.py
src\muneem\ocr.py
src\muneem\offers.py
src\muneem\password_rules.py
src\muneem\recovery.py
src\muneem\redaction.py
src\muneem\secrets.py
src\muneem\statement_unlock.py

```
src\muneem\unlock.py
src\muneem.egg-info\dependency_links.txt
src\muneem.egg-info\entry_points.txt
src\muneem.egg-info\PKG-INFO
src\muneem.egg-info\requires.txt
src\muneem.egg-info\SOURCES.txt
src\muneem.egg-info\top_level.txt
testdata\aubank-statements\Statement_2024MTH07_XXXXX859_146.pdf
testdata\aubank-statements\Statement_2026MTH01_XXXXX859_721.pdf
testdata\aubank-statements\Statement_2026MTH03_XXXXX859_108.pdf
testdata\aubank-statements\Statement_2026MTH04_XXXXX859_849.pdf
testdata\axis-statements\AXIS BANK  Statement for August 2024.pdf
testdata\axis-statements\AXIS BANK  Statement for August 2025.pdf
testdata\axis-statements\AXIS BANK  Statement for January 2026.pdf
testdata\axis-statements\AXIS BANK  Statement for June 2025.pdf
testdata\axis-statements\AXIS BANK  Statement for May 2025.pdf
testdata\axis-statements\AXIS BANK  Statement for October 2023.pdf
testdata\axis-statements\AXIS BANK  Statement for October 2025.pdf
testdata\credentials\client_secret_207319377548-
j0vbfnqoqak4cq6kqa4s9ikq6l5sgore.apps.googleusercontent.com.json
testdata\hdfc-statements\Avi_Dullu_28022026_183854522.pdf
testdata\hdfc-statements\Avi_Dullu_30042026_001858675.pdf
testdata\hdfc-statements\Avi_Dullu_30062025_105046061.pdf
testdata\hdfc-statements\Avi_Dullu_30092025_094157457.pdf
testdata\hdfc-statements\Avi_Dullu_30112025_110905081.pdf
testdata\hdfc-statements\Avi_Dullu_31082025_205409615.pdf
testdata\hdfc-statements\HDFC_Avi_Dullu_31072024_150007641.pdf
testdata\promo-emails\airasia-1.pdf
testdata\promo-emails\airasia-2.pdf
testdata\promo-emails\airasia-3.pdf
testdata\promo-emails\dataquest.pdf
testdata\promo-emails\expected.yaml
testdata\promo-emails\flipkart.pdf
testdata\promo-emails\makemytrip-1.pdf
testdata\promo-emails\makemytrip-2.pdf
testdata\promo-emails\README.md
testdata\promo-emails\swiggy.pdf
testdata\promo-emails\tataneu.pdf
testdata\promo-emails\unacademy.pdf
testdata\snapshots\April Credit Card Statement.json
testdata\snapshots\lder Credit Card Statement.json
tests\ingestion\test_gmail.py
tests\ingestion\test_providers.py
tests\parsers\test_au.py
tests\parsers\test_axis.py
tests\parsers\test_clustering.py
tests\parsers\test_engine.py
tests\parsers\test_hdfc.py
tests\parsers\test_local_au.py
tests\parsers\test_local_axis.py
tests\parsers\test_parse_errors.py
tests\parsers\test_registry.py
tests\parsers\test_savings.py
tests\parsers\test_validation.py
tests\conftest.py
tests\synthpdf.py
tests\test_cli.py
tests\test_db_crypto_ops.py
tests\test_db_crypto.py
tests\test_db_invariants.py
tests\test_db.py
tests\test_dependency_licenses.py
tests\test_errors.py
tests\test_eval_cli.py
tests\test_extract.py
```

```
tests\test_log.py
tests\test_migrations.py
tests\test_ocr.py
tests\test_offers.py
tests\test_password_rules.py
tests\test_precommit_hook.py
tests\test_promo_regression.py
tests\test_provenance.py
tests\test_recovery.py
tests\test_redaction.py
tests\test_regression_synth.py
tests\test_repo_hygiene.py
tests\test_secrets.py
tests\test_smoke.py
tests\test_statement_unlock.py
tests\test_unlock.py
tests\test_verifier_properties.py
tools\gmail_auth.py
tools\imap_auth.py
tools\outlook_auth.py
.coverage
.gitattributes
.gitignore
CLAUDE.md
DESIGN.md
Makefile
pyproject.toml
README.md
```

7. user (2026-06-10T16:08:35.083Z)

```
testdata/promo-emails/README.md
testdata/promo-emails/airasia-1.pdf
testdata/promo-emails/airasia-2.pdf
testdata/promo-emails/airasia-3.pdf
testdata/promo-emails/dataquest.pdf
testdata/promo-emails/expected.yaml
testdata/promo-emails/flipkart.pdf
testdata/promo-emails/makemytrip-1.pdf
testdata/promo-emails/makemytrip-2.pdf
testdata/promo-emails/swiggy.pdf
testdata/promo-emails/tataneu.pdf
testdata/promo-emails/unacademy.pdf
---- gmail repo ----
a764030 Codify the auditability invariant (Option A) + enforce no egress (#14)
90c85ba Merge pull request #13 from avidullu/docs/vision-and-architecture
4975254 docs: capture extraction-pivot vision and architecture
172b3b3 Validate setup input + traceable attachment-copy errors (#12)
964b58c Portable attachment names + robust content-type matching (#11)
5e934b1 Make the web app multi-user-ready for small-group / org deployment (#10)
e4362a3 Add hybrid history.list incremental sync (opt-in) (#9)
6d0b10e ci: trigger deploy to verify CLASPRC_JSON (v2 format)
098ddfa ci: validate clasp creds shape and pin clasp in deploy (#8)
c41630e ci: bump checkout/setup-node to v5 on Node 24 (#7)
```

8. user (2026-06-10T16:08:35.108Z)

```
1       # feedbackJune2026 ? Architectural Review: Response & Incorporation Plan
2
3       > **Status: Approved direction (2026-06).** Avi has signed off on this analysis/plan.
The §5 open
4       > decisions (flagged **? DECISION**) remain to be settled before *their* phases are
implemented and
5       > must not be silently locked by code. The low-risk Phase-A items that carry **no** open
decision
```

> (Windows CI, `docs/schema.md`, marking Outlook/IMAP experimental) are being implemented now; the
> rest waits on §5.
>
> Author's note (attribution): claims about current code are grounded in the repo as of `main`
> after PRs #34?#40. Where I infer or recommend, I say so.

---

0. Framing ? what "commercial-grade" means *here*

muneem is **paid, but single-user, local-first, and lifelong**: one person's entire financial life,
on their own machine, for years. So the bar is **correctness · zero-leak · auditability ·
recoverability · maintainability-by-one**. It is **not** multi-tenant SaaS. That distinction drives
several push-backs below: the right move is to be *bulletproof at single-user scale*, not to import
enterprise machinery (heavy ORMs, distributed anything, a web tier) that adds surface without
serving the actual risk. "High bar" = fewer moving parts, each provably correct and recoverable.

Two non-negotiables carry over from CLAUDE.md and stay load-bearing in everything below:
1. **No sensitive data leaves the machine** without explicit per-category, per-session approval.
2. **No open architectural decision gets silently locked** by code. This doc proposes; Avi disposes.

---

1. Corrections ? three premises in the review are already stale

The review is strong, but three items were **done in the recent code-quality pass** and should be
re-scoped from "needed" to "extend / verify":

Review claim	Reality (as of `main`)	What actually remains
"currently zero logging in src/"	**False now.** `muneem/log.py` (`get_logger` + `RedactingFilter`) + `muneem/redaction.py` landed in **#36**, wired into `cli`, with masking tests (`test_redaction`, `test_log`).	Structured (JSON/logfmt) output + **broad adoption** across modules + per-stage leak tests. The *foundation* exists.
"muneem db backup (proper Online Backup API, not naive cp)"	**Done in #39.** `backup_database()` uses the SQLite Online Backup API (WAL-safe), destination keyed ? encrypted backup.	A documented restore runbook; optional *recovery-key* backups (see §2).
"key rotation scaffolding (the deferred passphrase path)"	**Rotation done in #39.** `rekey_database()` + `muneem db rotate-key` (keychain-first with rollback).	The **passphrase path** specifically ? but reframed as **recovery**, not just "stronger" (see §2).

Also already in place (so the review's "table-stakes" list is largely satisfied): SQLCipher
encryption-at-rest (#34), schema-lock + DB-invariant tripwires (#34), pre-commit secret guard,
license allowlist, **mypy** + **ruff** + **80% coverage gate** in CI and the pre-push hook (#38/#40),
verifier property-fuzz (#40). The engineering-hygiene floor is commercial-grade *today*.

---

2. ? The biggest risk the review *under-weights*: key-loss / disaster recovery (P0)

Encrypting the ledger at rest (correctly!) created a **catastrophic single point of

failure** that
none of the ten bullets names directly:

> The 256-bit DB key lives only in the OS keychain. **`muneem db backup` encrypts the backup with the
> same key.** So if the keychain is lost ? OS reinstall, profile/Credential-Manager corruption, dead
> machine, new laptop ? the ledger **and every backup** are *permanently unrecoverable*.

For a tool whose entire pitch is "hold my whole financial life for years," this outranks most of the
listed features. It must be solved **before** we accumulate real, irreplaceable history.

The right way (reframe the deferred "passphrase mode" from security ? recovery):
- Keep the random DB key (fast, full-entropy) as the working key.
- Add a **recovery wrapper**: encrypt the DB key under a user passphrase (Argon2id, OWASP params)
  and store that wrapped blob in a `recovery.json` the user backs up **off-machine** (password
  manager / printed). Losing the keychain ? restore the key from passphrase + recovery file.
- And/or `muneem db export-key` ? prints the raw key **once**, with loud warnings, for the user's
  password manager. Simpler; less safe-by-default.
- Optionally let `db backup --recovery-key` write a backup under the passphrase-derived key, so a
  backup is restorable even with the keychain gone.
- Ship a **`docs/recovery.md` runbook**: "keychain lost," "move to a new machine," "verify a backup."

? DECISION: which recovery model ? (a) passphrase-wrapped recovery file *(recommended)*, (b)
one-time key export, (c) both. Pick before storing real numbers long-term.

---

3. Point-by-point: verdict · the right way · dependencies

Verdicts: **? Accept** · **? Accept-with-changes** · **? Push back** · **?? Largely done**.

P1 ? Structured, redacted logging (D.1) · ? Accept-with-changes
Premise stale (see §1). **Right way:** build on `log.py`/`redaction.py`; add a structured formatter
(logfmt or JSON ? ? minor decision), adopt `get_logger(__name__)` across `parsers/`, `db`, `unlock`,
`ingestion` at sensible levels; keep `RedactingFilter` as the *backstop*, not the primary control
(code passes already-redacted context). **Tests:** feed synthetic sensitive values through *each
stage* and assert masking (extend the #36 tests). The other half of D.1 ? *idempotent, resumable
re-runs* ? is a **pipeline** concern (staging dir + `documents.status`), partly separable; don't
conflate. **Priority: high, low-risk ? Phase A.**

P2 ? Two-layer regression CI · ? Accept (already the plan)
Layer 1 (synthetic full-pipeline via `synthpdf.py`) exists (`test_regression_synth.py`); expand it
for more HDFC/Axis/AU header variants + bookend drift (this also absorbs the deferred "adversarial
synthpdf fixtures" follow-up from the code pass). Layer 2 (self-hosted runner over real, gitignored
statements; passwords from `password_rules.yaml`, **never** GitHub secrets) is decided/deferred.

Caveat to record: a self-hosted runner executes CI on Avi's personal machine ? fine for a
no-external-contributor repo, but note it runs only when the machine is on, and never enable it for
forked PRs. **? DECISION:** confirm self-hosted-runner acceptance. **Phase D.**

P3 ? Storage shape + provenance/confidence · ? DECIDED (2026-06)
Decision (Avi): keep **flat, per-issuer source-of-truth tables** ? each parser writes its own
isolated table (today's `axis_*` / `hdfc_*` / `au_*`, extended per issuer/instrument). Rationale:
regression isolation (a bad parse refreshes *one* table), trivial backfill/rollback, parser-faithful
storage. Cross-source querying is served by a **derived serving layer** ? a SQL **view / materialised
table** now, graduating to a separate query-optimised "serving DB" only when latency/cost demand it;
the flat tables remain the rollback-able **source of truth**. This **supersedes** the earlier "core +
per-family canonical table" sketch, and fits the reconcile-oracle philosophy *better* (each source
stays independently verifiable). Endorsed ? no strong reason to override the flat-truth + serving-
layer (bronze?silver) split.

Provenance + confidence are first-class but NOT per-row (Avi): recorded in a **separate
`provenance` table keyed by `document_id`** ? `source_type` (pdf/aa/sms/manual), `parser`,
`parser_version`, `mapping_layer`, `confidence`. Every txn row already FKs `documents(id)`, so it
reaches its provenance by join with **zero redundancy**. The reconciliation oracle treats every
source (PDF / AA / SMS) as a **hint to validate**, never ground truth. **Phase B = the `provenance`
table (the first forward migration) + a serving view; instrument-family tables follow the same
per-issuer-flat pattern.**

P4 ? L3 vision fallback + golden evals · ? Accept-with-changes (sequence!)
Decision stands (DeepSeek-OCR-3B, MIT, 4-bit+SDPA on the 2070S, reconcile-gated, self-consistency
resampling). **Push-back on ordering:** L3 should come **after D.3** ? the eval writeup that
characterizes where the deterministic ladder actually breaks. Building a fallback before we know
its target failure modes is guessing. So: **D.3 eval first ? then L3.** Golden sets (local +
synthetic now; cloud eval-only, approval-gated) extend `muneem eval`. **Phase E (after D.3).**

P5 ? Resolve/time-box parked decisions (§8.3 LLM locality, §8.5 merchant canon) · ? Accept
These stay parked until a milestone *forces* them, and surface to Avi with the hardware/latency
constraints first. **Hard rule restated:** no cloud path for bank/CC/broker data, even
experimentally, without per-session approval + redaction + a test gate. Merchant canon (§8.5) is
downstream of P3+P8. **Governance item ? not a build; revisit at the phase boundary.**

P6 ? Versioned, tested schema migrations + `docs/schema.md` · ? Accept ? but ? **not Alembic**
Accept the need; **reject Alembic** (it drags SQLAlchemy ? heavy, ORM-shaped ? against muneem's
deliberate raw-`sqlite3`, stdlib-light design). **Right way:** a tiny hand-rolled runner

? numbered
    `migrations/NNN_*.py|sql` applied in order inside the keyed connection, recorded in a
    `schema_migrations` table, each with an **idempotency/up-checksum**, gated by the
`user_version`
    pragma. Pair with the existing schema-**lock** test (drift tripwire) ? versioned +
tested + enforced.
    Add a living `docs/schema.md`. Note: migrations run *inside* the encrypted connection ?
fine, but the
    runbook must cover "migrate then re-verify reconciliation." **Phase A (precedes P3's
migrations).**

    ### P7 ? Ingestion completeness + `muneem fetch` · ? Accept ? but ? re-sequence
    Push-back on "before heavy parser work": the parser core is already *proven* (HDFC
6/6, AU 4/4,
    Axis 6/7) and the user supplies statements manually today. `fetch` is
**automation/convenience, not
    correctness** ? it doesn't unblock parser accuracy or the whole-finance vision;
**storage
    unification (P3) does.** So: **P3 before P7.** What *is* cheap and should happen
immediately:
    explicitly mark Outlook/IMAP attachment paths **experimental/deferred** in code + docs
(the mypy
    override already treats them as such) so no one mistakes scaffolding for support. Then
build `fetch`
    (incremental `historyId`/`deltaLink`, sender allowlist + "financial document"
heuristics, staging ?
    `documents` rows as `locked`/`unparsed`, robust dedup) as **Phase F**. The fast bit
(mark
    experimental) is **Phase A**.

    ### P8 ? Tier-2 transaction semantics + merchant canonicalization · ? Accept
(downstream)
    Strictly **after P3** (needs the canonical schema) and needs **real receipt data** to
expose the mess
    (don't design merchant canon in the abstract). Counterparty/merchant extraction + fuzzy
match +
    a user-editable **override table**; itemized receipts (Swiggy/Amazon) likewise. **Phase
G.**

    ### P9 ? Observability / ops surface · ? Accept (partly done; pull recovery forward)
    Done: `db backup` (Online Backup API), `db rotate-key`, `db status` (#39). **Right way
for the rest:**
    `muneem verify` (re-run reconciliation over *stored* rows ? a cheap, high-value
integrity audit that
    also exercises provenance), `export` (CSV/JSON for the user's own analysis ? explicitly
**redaction-
    exempt** since it's the user's own data to their own disk, but never to stdout/logs),
`txns` filters
    (date/amount/account/status), light analytics. **Crucially, the recovery model (§2)
belongs here but
    is P0 ? pull it into Phase A, not "later."** Document backup/restore + "keychain lost."
**Phase A
    (recovery) + Phase H (the rest).**

    ### P10 ? Packaging / distribution / reproducibility · ? Accept-with-changes
    - **Lockfile** (uv.lock or pip-tools `requirements*.txt`): ? yes ? reproducibility is
real for a tool
      with a compiled dep (sqlcipher3). **Phase A.**
    - **Cross-platform CI matrix** (Windows + macOS, not just ubuntu): ? **high value,
under-rated** ?
      the dev box is Windows, and the secret backends genuinely differ (DPAPI vs Secret
Service vs
      Keychain) and sqlcipher3 wheels are per-OS. This directly de-risks "works on my
machine" for the
      one machine that matters. **Phase A.**

- **SBOM in CI** (CycloneDX): ? reasonable for paid software but **time-box / low-urgency** ? it
  serves a distribution/audit story we don't have yet. Do it near first external distribution, not
  now. **Phase H+.**
- **Local web UI:** ? defer ? CLI stays authoritative; a web tier is new attack surface + scope for
  no current need. Revisit only if a real UX gap appears.

---

4. Proposed sequencing (dependency-ordered)

> Rule of thumb: **recoverability + foundation first; then the one big schema decision; then
> everything that depends on it.** Each phase is shippable and individually PR-able.

- **Phase A ? Foundation & recovery (low-risk, high-leverage):** §2 **recovery model (P0)** ·
  finish structured-logging adoption (P1) · hand-rolled migration runner + `docs/schema.md` (P6) ·
  lockfile + cross-platform CI matrix (P10) · mark Outlook/IMAP experimental (P7-fast).
- **Phase B ? Provenance table (first forward migration) + serving view (P3, DECIDED).** Flat
  per-issuer source of truth stays; add the separate `provenance` table + a cross-source serving
  view. Unblocks the instrument family.
- **Phase C ? Instrument family** (equity/MF/ETF/bond events) on the unified schema + new
  `FamilySpec`/oracle (P3 cont.).
- **Phase D ? Regression CI Layer 2 (self-hosted) + the D.3 eval writeup** (P2) ? also tells us where
  L3 is actually needed.
- **Phase E ? L3 vision fallback + golden evals (P4),** informed by D.3.
- **Phase F ? `muneem fetch` + ingestion completeness (P7).**
- **Phase G ? Tier-2 semantics + merchant canonicalization (P8),** on real receipt data.
- **Phase H ? Remaining ops surface (verify/export/filters/analytics) (P9) + SBOM (P10).**
- **Parked/governance throughout:** §8.3, §8.5 (P5) ? surface at their forcing milestones.

---

5. Decisions ? settled vs still open

Settled (2026-06):
1. **Recovery / key-loss model** ? **passphrase-wrapped recovery file** (Argon2id + AES-GCM). ? shipped (#43).
2. **Storage shape** ? **flat per-issuer source-of-truth tables + a derived serving layer; provenance/
   confidence in a separate `provenance` table** (P3). Endorsed; Phase B implements the provenance table.
3. **Migration mechanism** ? **hand-rolled runner** (not Alembic). ? shipped (#44).
8. **Open Banking / Account Aggregator** ? **AA deferred; PDF-first**, with explicit re-adopt triggers
   (see §7 and [`research-open-banking-india.md`](./research-open-banking-india.md)).

Still open:
4. **Structured log format** (P1) ? logfmt vs JSON.
5. **Self-hosted runner** (P2) ? accept the run-CI-on-my-machine trade-off?
6. **macOS first-class** (P10) ? Windows + Linux are first-class now (CI matrix, #42); macOS best-effort?
7. **Parked:** §8.3 sensitive-doc LLM locality, §8.5 merchant canonicalization ? confirm still parked.

```
---

## 6. Push-back summary (explicit, with reasons)

- **Alembic ? hand-rolled migrations.** Dep-minimization + raw-`sqlite3` design + the schema-lock
  already exists; Alembic's ORM weight buys nothing here.
- **"Ingestion before heavy parser work" ? storage unification (P3) first.** Parser core is proven;
  `fetch` is convenience, not correctness. (But mark Outlook/IMAP experimental *now* ? cheap.)
- **L3 before D.3 ? reversed.** Characterize the deterministic ladder's failure modes *first*; don't
  build a fallback blind.
- **SBOM / web UI ? time-box / defer.** Low urgency vs correctness + recoverability; web UI is
  unjustified surface for a single-user CLI tool.
- **Don't over-engineer for scale muneem doesn't have.** Single-user, local-first. Bulletproof beats
  big.
- **And the addition the review missed: key-loss recovery is P0** ? above most of the feature list.

---

## 7. Open Banking / Account Aggregator (researched 2026-06) ? decided: **defer, PDF-first**

Full writeup + sources: [`research-open-banking-india.md`](./research-open-banking-india.md). Summary:

- **Verdict:** a single-user, local-first, no-paid-API tool **cannot** use India's Account Aggregator
  (AA) today. **FIU status requires a regulated entity** (no individual/hobbyist path); the only
  reachable integration rides under a licensed FIU via a **paid TSP that decrypts your whole feed in
  its cloud**. So AA-primary is both *impossible* and *undesirable* under muneem's principles.
- **On the security fear:** AA is *safer in transit* (E2E-encrypted, data-blind aggregator, no
  credential sharing) ? but **"self-custody" is not an available AA mode**; decryption is institutional
  (FIU/TSP cloud). For muneem's threat model (one local honeypot, zero third parties) that's a **net
  privacy regression** ? so the wariness is justified, for a subtle reason (a new institutional cloud
  tier, not weak crypto).
- **Fidelity:** AA wins on live bank rows + demat equity/MF, but **can't cover CC, bonds, broker
  detail, or non-demat MF** (muneem's exact scope), caps history ~2y, and still needs muneem's
  enrichment/oracle layer. A boost on a *subset*, not a replacement.
- **Stance:** **PDF-first core; AA a deferred, opt-in *complement* (import from a regulated FIU's
  export, oracle-validated, provenance-tagged) ? never primary.** Re-adopt triggers: a TSP documents
  **true on-device, self-custody-key decryption** (the key gate); CC/bonds go live on AA; a free
  personal tier appears; or muneem's PDF coverage saturates.
- **New near-term candidate it surfaced ? local SMS/email transaction-alert parsing (P11):** RBI
  mandates per-transaction bank SMS; mature OSS parsers exist; **zero third party, fully local,
  near-real-time.** Limits: no history/balances/holdings, brittle formats, no
```

```
investments ? so a
258       *freshness complement* to PDF, not a replacement, and it violates **no** muneem
principle (echoes
259       the predecessor's email-mining DNA). Slots into Phase A (optional) / Phase F.
260
261     ---
262
263     *Next step: §5's settled decisions are being implemented (recovery #43, migrations #44,
Windows CI
264     #42; provenance table next). Remaining open: log format, self-hosted runner, macOS. One
PR per item,
265     CI green and ?80% coverage throughout.*
266
```

9. user (2026-06-10T16:08:35.109Z)

```
1       # Current State
2
3       > **Last Updated:** 2026-06 (concept-core engine + issuer refocus: HDFC / Axis / AU
Bank)
4       > Summarizes current capabilities, completed features, and near-term next steps for
`muneem`.
5       > See [`ROADMAP.md`](./ROADMAP.md) for milestone tracking and [`parser-
architecture.md`](./parser-architecture.md) for the parser design.
6
7       > **Scope reminder ? muneem is a _whole-finance_ bookkeeper, not a bank tool.** Bank &
credit-card
8       > statements are the first vertical slice; **equities, mutual funds, ETFs, and bonds are
core
9       > targets**, parsed into the same local ledger, with transaction/instrument-event
classification to
10      > follow. The parser core is deliberately **family-agnostic** (see §2), so adding an
instrument
11      > family is *new data + a reconciliation rule*, not a new pipeline.
12
13      ## 1. Core Ingestion Pipeline
14      The ingestion pipeline supports a generic `SearchCriteria` interface and multiple
providers (Track B plumbing):
15
16      * **Gmail** (`muneem.ingestion.gmail`): Full OAuth2 read-only + search + attachment
download implemented.
17      * **IMAP** (`muneem.ingestion.imap`): Basic auth + search implemented; `get_headers` /
attachment methods are NotImplemented (experimental).
18      * **Outlook** (`muneem.ingestion.outlook`): Device-flow auth + basic search implemented;
attachment/header/download NotImplemented (experimental).
19
20      Companion one-time auth tools live in `tools/`. No high-level `muneem fetch` CLI command
yet (see Roadmap B.4).
21
22      ## 2. Statement Parsers (concept-core architecture)
23      Deterministic parsers live under `muneem.parsers`, registered via decorator and selected
by `get_parser` on text heuristics + sender. Submodules are **auto-loaded on package import**
(`_autoload_parsers`), so the registry is populated in the production path (not just under
tests).
24
25      The parser core is **concept-first and verification-first** (see [`parser-
architecture.md`](./parser-architecture.md)): an extractor maps an issuer's columns onto a bank-
agnostic **concept schema** (`concepts.py`); a generic **engine** (`engine.py`) does the
column?concept mapping (L0 header lexicon ? profile column-order ? L2 content search); and a
shared **verifier** (`validation.py`) decides trust by *arithmetic* ? per-row balance walk +
total reconcile + SUMMARY bookend + a recorded confidence verdict. The engine is **family-
agnostic**: parameterized by a `FamilySpec` + an injected reconciliation oracle, so a future
**instrument family** (equity/MF/ETF/bond holdings + buy/sell/dividend/fee events) reuses the
same control flow with its own concepts + oracle. Issuer knowledge lives in `profiles.py` as
**data, not code**. Bank **savings** parsers share one engine-driven path ? `savings.py` (per-
```

table + coordinate-clustered candidates ? engine ? oracle), so adding a savings issuer is mostly a profile; HDFC keeps bespoke coordinate clustering (the documented `extract_tables` exception).

Active issuers ? we have real statements + passwords, so these get validated on real data:

* **HDFC Bank** (`hdfc.py`): Savings / transaction-account statements via **coordinate clustering** (deliberately *not* `extract_tables()` ? HDFC's table is semi-ruled). **Self-validating:** running-balance reconciliation + a cross-check against the bank's own SUMMARY (Dr/Cr counts, opening/closing balance); records `documents.status` = `reconciled`/`unreconciled`. Refactored onto the concept core as the **reference implementation**. ? **Verified on the user's 6 real statements** ? all reconcile *and* bookend, without inspecting any value.
* **Axis Bank** (`axis.py`): Savings + Credit Card. Savings runs the shared engine-driven path (`savings.py`) ? per-table + coordinate-clustered candidates ? engine mapping ? reconciliation oracle ? plus a **balance-delta reconstruction**: when columns don't split into a clean debit/credit (a merged amount column, an interleaved value-date, a sparse deposit column), it derives direction from the running-balance sign and the oracle cross-checks `amount == |?balance|`. Oracle-gated, so it never persists unreconciled rows. ? **6/7 real statements reconcile + persist**; the 1 miss (May 2025) has a transaction line the clusterer drops ? incomplete ledger ? the oracle quarantines it (stores **nothing**). Credit-card has no running balance (count only).
* **AU Bank** (`au.py`): engine-driven savings via the shared `savings.py` path ? same recipe, just `AUBANK_PROFILE`. ? **4/4 real statements reconcile + persist** (the cleanest issuer so far).

> _Dropped: **ICICI** ? no statements on hand to validate against, so it was removed to focus effort on the issuers we can actually verify (HDFC/Axis/AU). It can return as a profile when data exists._

All balance-bearing parsers share the verifier and split parsing from DB-insert, so row logic is unit-testable without a PDF/DB and a parse is validated before it's trusted. **Concept-core steps 1?3 have landed** (the verifier + concept schema; the generic mapping engine + profiles; the shared savings path with Axis + AU Bank on it). Parsers use `pdfplumber` (with optional password). **Password resolution is now wired into `muneem ingest --doc-type bank_statement`** ? the hybrid unlock (keychain-cached + `password_rules.yaml` candidates ? interactive prompt; `unlock.py` + `statement_unlock.py`) runs before extraction and forwards the password to the parser, in-memory only. See [ingesting-statements.md](./ingesting-statements.md).

See `password_rules.py` (prefers user config dir) and `cli.py:ingest`.

3. Database & CLI

* Schema v4 with `documents`, `offers`, and per-bank txn tables (`axis_*`, `hdfc_transactions`) ? documented in [`schema.md`](./schema.md). Dedup on sha256. **Versioned migrations** via a hand-rolled runner (`muneem/migrations.py`): the v4 schema is the frozen baseline, forward migrations layer on (recorded in `schema_migrations` with checksums), drift is rejected ? see [`schema.md`](./schema.md) and feedbackJune2026 P6. A unified `transactions` table (plus `holdings` for instruments) with per-row provenance/confidence is the planned direction once the design decision lands (parser-architecture.md §9; feedbackJune2026 P3).
* **The on-disk ledger is encrypted at rest** (decision D.2) ? **SQLCipher** transparent AES-256 via `sqlcipher3`, opened through `db.connect_encrypted()`; the 256-bit key lives in the OS keychain. SQL queries are unaffected. `:memory:` and the CI fixtures stay on plaintext stdlib `sqlite3`. See [encryption-at-rest.md](./encryption-at-rest.md).
* CLI: `ingest <pdf>` (promo, or `--doc-type bank_statement` ? **auto-unlocks encrypted PDFs**, see [ingesting-statements.md](./ingesting-statements.md)), `offers list`, `txns list <bank>`, `eval <folder>` (reconcile-rate report ? the seed?golden onboarding loop, see [onboarding-a-bank.md](./onboarding-a-bank.md)), and `db` encryption ops (`status`, `encrypt`, `rotate-key`, `backup`, `setup-recovery`, `recover` ? see [encryption-at-rest.md](./encryption-at-rest.md)).
* Config uses platformdirs for local dirs; secrets (OAuth tokens, cached statement passwords, **the DB encryption key**) via keyring.

4. Testing & Hygiene

* Strong offline regression on `testdata/promo-emails/` + synthetic parser/engine

fixtures (default CI).
47   * Security guards in `test_repo_hygiene.py` + license allowlist test.
48   * **DB invariant tripwires** (`test_db_invariants.py`, default CI): a **schema lock**
(pins `SCHEMA_VERSION` + every table/column shape ? an accidental drift fails the merge; a
deliberate change is "explicitly allowed" by updating the lock in the same PR) plus **encryption
guards** (production ingest must write an *encrypted* ledger; a keyed open never silently
downgrades to plaintext; foreign keys enforced + cascade; `documents.sha256` dedup).
49   * **Local git hooks** (`.githooks/`, enable via `make hooks`): `pre-commit` blocks
staging sensitive files; `pre-push` runs ruff + the offline suite (incl. the invariant
tripwires) so only green code reaches GitHub. The pre-push gate is the **local** stand-in for
server-side branch protection, which GitHub gates behind Pro / public repos (muneem stays
private). Both are bypassable with `--no-verify` (the conscious "explicitly allowed" escape).
50   * Local-only tests for real statements (gitignored data; passwords via env/keyring,
never committed). The suite never touches the real OS keychain (autouse fixture in
`tests/conftest.py`) ? headless-CI safe.
51
52   ## 5. Next Steps
53   Concept-core **steps 1?3 are done**, and **password unlock is now wired into `muneem
ingest`** (C.1 ? hybrid candidates ? prompt, keychain-cached; see [ingesting-
statements.md](./ingesting-statements.md)). Real-statement precision: **HDFC 6/6, AU 4/4, Axis
6/7** (the 1 Axis miss stores nothing ? oracle-gated). In rough order from here:
54
55   * **Recover the May 2025 vintage**: the clusterer drops one transaction line ?
incomplete ledger; needs smarter line recovery. Isolated and safe (it currently stores nothing;
June 2025 was recovered via balance-delta reconstruction).
56   * **Step 4 ? L3 local-vision fallback** (pending the local-VLM deep-research run) for
layouts the deterministic ladder can't reconcile.
57   * **Step 5 ? unify storage** (`transactions` + `holdings`) and begin the **instrument
family** (equity/MF/ETF/bond ? holdings + buy/sell/dividend/fee classification) ? the next big
piece of the whole-finance vision.
58   * **Regression CI for statements** (decided 2026-06, *deferred ? not yet built*): a
smart mix of synthetic full-pipeline fixtures in cloud CI + a self-hosted runner exercising the
real HDFC/AU statements (which stay local, off GitHub). Grounded plan + rationale in
[ROADMAP.md](./ROADMAP.md) (§ "Planned ? Regression CI for statement parsing").
59
60   Parked: DESIGN §8.3 (local LLM for sensitive categories), §8.5 (merchant
canonicalization). _(Encryption-at-rest timing, D.2, is now resolved & implemented ? SQLCipher;
see [encryption-at-rest.md](./encryption-at-rest.md).)_
61
62   The project stays **local-first**, secrets in the OS keychain, promo corpus safe for
CI/LLM experiments.
63

10. user (2026-06-10T16:08:35.274Z)

1   # Vision ? Email as a smart way to organize your life
2
3   > Status: living document. The overall product direction. Architectural detail
4   > and the decision log live in [`ARCHITECTURE.md`](./ARCHITECTURE.md).
5
6   ## North star
7
8   Turn a person's email into a **high-fidelity, structured, queryable record of
9   their life** ? receipts, bank statements, flights & travel, coupons, shopping ?
10   built automatically, privately, and reliably. Email becomes the source of truth
11   that organizes the rest.
12
13   ## What this repo is ? and is not
14
15   This repository (`gmail-attachments-to-gdrive`) is the **extraction / ingest
16   layer**. It is the reliable, quota-aware "front door" that:
17
18   - watches a user's Gmail **in their own account**,
19   - archives raw artifacts (attachments) ? still valuable on its own,
20   - **classifies** messages and **extracts** a clean, normalized signal,

```
21      - **redacts/minimizes** that signal, and **hands it off** to downstream
22        processors.
23
24      It is **not** the domain parser, the datastore, or the analytics engine. That
25      logic is deliberately **isolated in separate downstream services**:
26
27      | Concern | Owner |
28      | --- | --- |
29      | Gmail watching, archiving, classification, extraction, redaction, handoff | **this
repo** |
30      | Bank-statement & financial parsing, indexing, encrypted storage, analytics |
**Muneem** (`avidullu/muneem`) |
31      | Grocery / receipts, coupons, travel processing | future siblings of Muneem |
32
33      The boundary is intentional: **extraction (this repo) is decoupled from
34      processing (downstream).** This repo never imports domain-parsing logic;
35      downstream services never read Gmail. They meet only at a versioned handoff
36      contract.
37
38      ## Foundation first
39
40      Attachment archiving is the **hardened core**. Everything else is built on its
41      proven strengths, which we preserve and extend, never bypass:
42
43      - per-user isolation (`executeAs: USER_ACCESSING`),
44      - a per-user lock and self-healing trigger fan-out,
45      - the hybrid `history.list` + date-window sync,
46      - "mark a message done only on clean completion,"
47      - pure-logic / GAS-shell separation with heavy unit testing,
48      - ruthless respect for Apps Script quotas and the 6-minute boundary.
49
50      ## Audience & rollout
51
52      - We **build for public** ? the long-term goal is a wide launch.
53      - **For now, stay under 100 users** (OAuth "Testing" mode) to harden the
54        system, learn the value proposition, and make an *educated* call on whether to
55        financially invest (the annual CASA security assessment, hosted infra) **once
56        there is a substantial user base**.
57      - Until that call, we stay in Google's no-verification zone (see
58        [`../README.md`](../README.md) ? Multi-user deployment / publishing).
59
60      ## Privacy north star
61
62      - **Minimize egress.** Only **non-PII** signal ever leaves the user's Google
63        account, and only to **our own isolated, hosted service** ? **never third
64        parties**.
65      - **Scrub + encrypt** as required, applied *before* anything leaves.
66      - Continuously **reduce** what leaves while maintaining product health.
67      - The in-account baseline needs **zero** external dependency; external
68        processing is a deliberate, **separately-consented** upgrade with a privacy
69        policy that names exactly what is sent, where, and why.
70
71      ### The auditability invariant
72
73      **This repo is the only code that ever touches a user's Gmail, and it stays
74      auditable and self-deployable by a power user at any time.** It is intended to be
75      open-sourced precisely so that a privacy-conscious user can read ? and run ? the
76      exact code that reads their mail. Muneem and every downstream service operate
77      only on minimized, non-PII signal and never hold the Gmail grant. This is a
78      binding constraint on all future work; the enforceable form (and a per-change
79      checklist) lives in [`ARCHITECTURE.md`](./ARCHITECTURE.md#auditability-invariant-
binding).
80
81      ## First verticals
82
```

```
1. **Grocery receipts** ? high volume, forgiving fidelity bar; proves the whole
   ingest ? classify ? extract ? index pipeline cheaply.
2. **Bank statements** ? high-fidelity path; output feeds **Muneem**, proving the
   redact ? handoff ? downstream-processing seam.

## Success criteria (proposed)

- The user's index auto-populates with correct entities at **high precision**;
  low-confidence items are **surfaced for review**, never silently dropped.
- Re-running an improved extractor over history **corrects past entities without
  duplicates** and without re-copying blobs.
- Egress stays **minimal and non-PII**; the in-account baseline works with no
  external service available.

## Phased plan (high level)

Detailed status is tracked in [`../ROADMAP.md`](../ROADMAP.md); the technical
shape of each phase is in [`ARCHITECTURE.md`](./ARCHITECTURE.md).

0. **Ingest model** ? a rich, stable `EmailMessage`; the copier consumes it
   (no behavior change).
1. **Classify + extract + index (GAS-only, in-account)** ? classifier, pluggable
   extractors, normalized entities, a user-owned index. No egress.
2. **Coverage + re-extraction** ? more extractors, body/OCR handling, extraction
   cursors & status, "re-extract for new rules," review queues.
3. **Handoff to hosted services** ? redact/minimize, then feed Muneem & siblings;
   separate consent + privacy update; extraction/egress budgets.
4. **Surfaces & ops** ? dashboards, Calendar/Tasks automation, a richer review &
   configuration UI, Shared Drives, schema migration.
```

11. user (2026-06-10T16:08:35.737Z)

```
# Roadmap

Next steps for the modernised project, grouped by effort and grounded in the
current Apps Script / Gmail platform constraints.

## Platform constraints that shape the design

For consumer (gmail.com) accounts:

- **6 minutes** max per execution; **90 minutes/day** total trigger runtime;
  **20 triggers** per script per user; ~**20,000** Gmail read calls/day.
  ([Apps Script quotas](https://developers.google.com/apps-
script/guides/services/quotas))
- There is **no clearly documented daily Drive file-create quota**, so
  `MAX_FILES_ADD_PER_DAY` (240) is a self-imposed heuristic, not an API limit.
- Gmail's
[`users.history.list`](https://developers.google.com/workspace/gmail/api/guides/sync)
  with a stored `startHistoryId` is the purpose-built incremental sync (~2 quota
  units vs 5 for a list call) and returns only changes. A `historyId` can expire
  (HTTP 404 ? fall back to full sync), so it complements the date-window
  backfill rather than replacing it.
- clasp can deploy from CI using a `CLASPRC_JSON` secret (contents of
  `~/.clasprc.json`); service accounts do **not** work with the Apps Script API,
  so use a dedicated owner account's token.
  ([clasp #225](https://github.com/google/clasp/issues/225))

## Quick wins

- [x] **Cover the orchestration shells with tests.** `sync.ts` / `ui.ts` had no
  coverage ? the trickiest logic (trigger fan-out, quota reset, catch-up loop,
  `processInput` state reset). *(done in this branch; see `tests/sync.test.ts`,
  `tests/ui.test.ts`)*
```

```
- [x] **Raise the per-run wall-clock budget.** Was capped at 1 minute against a
  6-minute ceiling, making backfill ~6× slower than necessary. Now
  `MAX_RUNTIME_MS` (5 min) in `src/config.ts`, threaded through `hasTime`.
- [x] **Guard against trigger leakage.** A `LockService` script lock prevents
  concurrent runs from stacking triggers, deletion is scoped to the sync
  handler, and a cap prunes any excess. *(done in this branch; `src/sync.ts`)*
- [x] **Add a Jest `coverageThreshold`** so CI fails if coverage regresses.
  *(done; `jest.config.js`, gated at 85% stmts/lines/funcs, 75% branches,
  `entry.ts` bundler glue excluded)*

## Medium

- [x] **CI/CD auto-deploy on merge to `master`.** A gated `deploy` job writes
  `CLASPRC_JSON` ? `~/.clasprc.json` and runs `clasp push` / `clasp deploy`
  against the committed `.clasp.json`. *(done; `.github/workflows/ci.yml`.
  Requires only the `CLASPRC_JSON` repo secret; skips cleanly if unset.)*
- [x] **Cross-run dedup by message/attachment ID.** A bounded FIFO of processed
  Gmail message ids (`src/processed.ts`, capped at `MAX_PROCESSED_IDS`) is
  persisted in user properties; `updateDrive` skips messages already in the set
  and reports newly-completed ids back to `sync.ts`. A message is only marked
  done when all its attachments were copied cleanly (not truncated by the daily
  cap or a transient error), so re-scanned windows skip the Drive
  `getFilesByName` probe. *(done; `src/processed.ts`, `src/drive.ts`,
  `src/sync.ts`, `src/state.ts`)*
- [x] **Observability.** Level-prefixed structured logging (`src/logger.ts`),
  per-day run stats accumulator (`src/stats.ts` ? runs/files/errors/last error,
  persisted via `PROP_RUN_STATS`), and an opt-in daily summary
  (`src/notify.ts`) logged on each new-day rollover. The summary email is OFF by
  default; enabling it needs `ENABLE_SUMMARY_EMAIL` + the `script.send_mail`
  scope. *(done)*

## Larger / architectural

- [x] **Hybrid incremental sync.** Keep the date-window scan for the initial
  backfill; once caught up (`isToday`), switch the steady state to the Gmail
  advanced service + `history.list`/`startHistoryId`, storing `historyId` in
  properties and falling back to full sync on 404. *(done; pure logic in
  `src/historySync.ts`, advanced-service shell in `src/gmailHistory.ts`, wired
  into `src/sync.ts`. Gated behind `ENABLE_INCREMENTAL_SYNC` (OFF by default)
  for gradual rollout over the proven backfill. The Gmail advanced service is
  declared in `appsscript.json`; no new OAuth scope ? `gmail.readonly` covers
  history reads. Backfill captures the mailbox `historyId` before processing so
  mail arriving mid-run is still caught; expiry (HTTP 404) clears the cursor and
  rewinds to a bounded date-window re-scan.)*
- [ ] **More structure options & Shared Drive support** ? by-sender, nested
  `Year/Month`, label-driven folders; currently My Drive only.

## Open-source & auditability readiness

Concrete prep so the [auditability invariant](docs/ARCHITECTURE.md#auditability-
invariant-binding)
is real, not aspirational, before/when the repo is opened up.

- [x] Root `LICENSE` file (MIT, matching the source headers).
- [x] Egress guard ? a test that fails CI if `src/` makes any network call
  (`tests/auditability.test.ts`). Update it deliberately when the user-endpoint
  egress lands (assert the allowlisted destination, don't remove the guard).
- [ ] `SECURITY.md` ? how to report issues + the current data-flow guarantees
  (today: reads Gmail, writes the user's own Drive, **no egress**).
- [ ] **"Deploy your own from source"** guide ? self-host the GAS project end to
  end, so the trust claim is "run the code you read."
- [ ] **Reproducible / verifiable build** ? pin the toolchain and document how to
  diff a deployed `dist/Code.js` against a build of a given commit.
- [ ] `CONTRIBUTING.md` carrying the auditability PR checklist.
```

12. user (2026-06-10T16:13:27.715Z)

```
Workflow launched in background. Task ID: wctcr4nab
Summary: Deep review of muneem + gmail-attachments-to-gdrive: code, security, architecture,
roadmap, and market fit
Transcript dir: C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-
muneem\02b9e9c9-3806-45ca-9b24-2ecc35a81efd\subagents\workflows\wf_27df53c8-8e0
Script file: C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-
muneem\02b9e9c9-3806-45ca-9b24-2ecc35a81efd\workflows\scripts\muneem-deep-review-
wf_27df53c8-8e0.js
(Edit this file with Write/Edit and re-invoke Workflow with {scriptPath:
"C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-
muneem\02b9e9c9-3806-45ca-9b24-2ecc35a81efd\workflows\scripts\muneem-deep-review-
wf_27df53c8-8e0.js"} to iterate without resending the script.)
Run ID: wf_27df53c8-8e0
To resume after editing the script: Workflow({scriptPath: "C:\Users\avidu\.claude\projects\C--
Users-avidu-Projects-muneem\02b9e9c9-3806-45ca-9b24-2ecc35a81efd\workflows\scripts\muneem-deep-
review-wf_27df53c8-8e0.js", resumeFromRunId: "wf_27df53c8-8e0"}) ? completed agents return
cached results.

You will be notified when it completes. Use /workflows to watch live progress.
```

13. user (2026-06-10T16:13:35.719Z)

```
1       # muneem ? Design Document
2
3       > Status: **draft, pre-implementation**. This document captures the vision, the pipeline
shape, the security posture, and the open decisions to resolve before code lands. It is
intentionally opinionated where the author has a strong view, and explicitly undecided where the
trade-offs need a human call.
4
5       ---
6
7       ## 1. What muneem is
8
9       **muneem** is a personal-finance bookkeeper that ingests financial documents from the
user's email accounts, unlocks the password-protected ones, parses them into a normalized
schema, stores them in a local database, and surfaces analytics across the user's full financial
life.
10
11      The name is the Hindi word for the traditional household/business bookkeeper ? a trusted
figure who maintained the family ledger by hand. This project is the same role, automated.
12
13      **Single user. Local-first. Privacy-critical.**
14
15      ## 2. Why
16
17      The user's financial life today is fragmented across many silos:
18
19      - 3?5 bank accounts (statements arrive monthly as password-protected PDFs)
20      - Multiple credit cards (monthly bills, also password-protected PDFs)
21      - Brokerage and mutual-fund accounts (quarterly statements, portfolio snapshots)
22      - Grocery and shopping receipts (Swiggy, Zomato, Amazon, Flipkart, BigBasket, Blinkit ?
mostly plain HTML/PDF in email body)
23      - Utility bills, insurance premiums, EMI schedules
24      - Promotional offers and coupons (often forgotten)
25
26      Each silo has its own portal, login, and format. There is no consolidated view. **The
inbox is the only place where they already converge.**
27
28      Concrete use-cases the user wants once this exists:
29
30      1. **Grocery & shopping price intelligence** ? track what was bought, where, and at what
price, over time. Detect when a merchant is no longer competitive on staples.
31      2. **Investment portfolio timeline** ? a single time-series of holdings across brokers,
```

so allocation drift and rebalancing decisions are obvious.
32   3. **Bank/card comparison** ? fees paid, interest earned, reward redemption efficiency,
surfaced as objective numbers.
33   4. **Item catalog** ? every significant purchase linked to its receipt/invoice, so
warranty windows, return windows, and replacement cycles are queryable.
34   5. **Forgotten-deals reminder** ? surface the unused coupons and offers already sitting
in the inbox (this is the original goal of the predecessor project; see § 11).
35
36   ## 3. Origin
37
38   muneem is the successor to `~/Projects/Email-Mining`, a small prototype the user built
earlier that extracted promo codes from manually-saved email PDFs using PyMuPDF + Gemini for
image OCR. Same insight ("the inbox contains structured value I'm forgetting"), generalized
scope.
39
40   See [`docs/prior-art-email-mining.md`](./docs/prior-art-email-mining.md) for what it
did, what to inherit, and what not to.
41
42   ## 4. The pipeline (five stages)
43
44   Each stage is independent enough to develop and test in isolation. They communicate
through well-defined data boundaries (raw files on disk ? unlocked files on disk ? structured
records in DB ? derived views).
45
46   ### Stage 1 ? Email ingestion
47
48   **Goal:** pull messages and attachments from Gmail and Outlook into a local staging
area.
49
50   - **Gmail:** [Gmail API](https://developers.google.com/gmail/api) over OAuth2. Read-only
scope (`gmail.readonly`). Supports server-side search queries (`has:attachment filename:pdf
from:hdfc newer_than:90d`) which is far cheaper than pulling everything and filtering locally.
Has good rate limits for personal use.
51   - **Outlook:** [Microsoft Graph API](https://learn.microsoft.com/en-
us/graph/api/resources/mail-api-overview) over OAuth2. Read-only scope (`Mail.Read`). Similar
search capability.
52   - **Why not IMAP:** worse search, no labels, harder OAuth, and we lose Gmail's powerful
query syntax. Use the native APIs.
53
54   Mechanically straightforward. The main design questions are: **how is "financial
document" detected** (sender allowlist? subject patterns? attachment heuristics? hybrid?), and
how is incremental sync state tracked (Gmail `historyId` / Graph `deltaLink`).
55
56   ### Stage 2 ? Attachment unlocking
57
58   **Goal:** decrypt password-protected PDFs.
59
60   The decryption itself is trivial ? every mainstream PDF library handles the standard
security handler. In Go, `github.com/pdfcpu/pdfcpu`; in Python, `pikepdf` or `pypdf`.
61
62   The real design question is **where the passwords come from.** Indian bank and CC
statement passwords are almost always derived from user attributes:
63
64   - "First 4 letters of name (uppercase) + DDMM of DOB" ? common at HDFC, Axis
65   - "PAN number" ? common at some brokers
66   - "Last 4 digits of card number" ? common for CC statements
67   - "Customer ID" ? some banks
68   - "Mobile number" ? utilities
69
70   Three plausible approaches, each with trade-offs:
71
72 | Approach | Pro | Con |
73 |---|---|---|
74 | **Per-sender rule template** the user configures once (e.g., `from:alerts@hdfcbank.net
? first4(name)+ddmm(dob)`) | Deterministic, no guessing, auditable | Requires upfront setup;

rules must be maintained as banks change formats |
| **Candidate dictionary** (try a small list of derived strings per PDF) | Zero config; "just works" | Brute-forceable feel; fails noisily on unknown senders |
| **Interactive prompt** (ask the user the first time, cache for that sender) | Safe, simple | Friction during initial backfill of years of statements |

A hybrid is likely best: candidate dictionary first, fall back to rule template, fall back to interactive prompt. **Decision parked.** See § 8.

Stage 3 ? Document parsing

This is the hard 80% of the project. Every issuer's PDF layout is different and changes over time.

The space of approaches, in increasing robustness and cost:

1. **Per-issuer regex/template parsers.** Precise; brittle. One parser per format, breaks when the issuer redesigns. Realistic for the top ~5 senders that dominate volume.
2. **Text-layer extraction + structural heuristics.** Pull text with `pdfminer.six` / `pdfplumber` (both MIT); rely on coordinates and font runs to recover tables. Works well for "born digital" PDFs (most bank statements); fails on scanned/image PDFs. (We avoid PyMuPDF/`fitz` ? it is AGPL; see § 5 rule 8.)
3. **OCR (Tesseract) for image-based PDFs and embedded raster content.** Slower; need layout reconstruction.
4. **LLM-assisted structured extraction.** Hand page text (+ optionally page rasters) to a model with a target JSON schema. Generalizes across layouts; expensive per page; **must be local for sensitive document categories**. Options: Ollama-hosted models (Llama 3, Qwen, Mistral), or a small purpose-built model.

In practice this will be **a hybrid**: deterministic template parsers for the top ~5 high-volume senders (where they pay for themselves), LLM-assisted extraction as the long-tail fallback. Promo emails and other non-sensitive categories can use cloud LLMs; bank/card/investment statements **must not** without explicit per-category approval.

Embedded image deduplication (Email-Mining used MD5 hash + SSIM) is a real win here: bank statements repeat the same logo/letterhead on every page, and we don't want to OCR that 30 times per document.

Stage 4 ? Normalization & storage

Goal: map heterogeneous parser outputs into one schema, persist locally.

Storage: **SQLite**. Single file, no server, easy to back up, easy to encrypt at rest (SQLCipher), good enough performance for years of personal data. The schema needs to span:

- `documents` ? every ingested file with sender, date, type, hash, path
- `accounts` ? bank accounts, cards, broker accounts, identified by issuer + last-4 or similar
- `transactions` ? line items from statements, normalized (date, amount, currency, counterparty, category, source_document)
- `holdings` ? point-in-time snapshots of investment positions per account
- `merchants` ? canonicalized merchant identities (so "SWIGGY*BANGALORE", "Swiggy Instamart", and "SWIGGY BNGLR" collapse to one)
- `items` ? line-items from itemized receipts (grocery, shopping)
- `offers` ? promotional offers/coupons with expiry, source, redemption status

The merchant canonicalization step is non-trivial ? it deserves its own thought (fuzzy match + manual override table is the usual answer).

Stage 5 ? Analytics & insights

Goal: turn the normalized DB into the use-cases in § 2.

Once the data is in the schema, the analytics are mostly SQL + light application logic. Likely surface: a local CLI initially, possibly a small local web UI later. Notebook-style

exploration is fine for prototyping queries before they're promoted to first-class views.

---

5. Security posture (first class)

The threat model deserves to be stated plainly: **muneem aggregates the user's entire financial life ? every account, every transaction, every holding ? into one local store. That store is a honeypot.** A leak would be catastrophic.

Standing rules:

1. **All raw statement data stays on the local machine.** No cloud sync of the staging dir or the DB.
2. **No cloud LLM APIs for sensitive document categories** (bank, CC, broker, MF, insurance) without explicit per-category user approval discussed in the current session. The default extraction path for these is local (Tesseract + local LLM via Ollama, or deterministic parsers).
3. **Non-sensitive categories** (promo emails, generic newsletters, public receipts) may use cloud LLMs after a privacy review of the specific content type.
4. **DB encrypted at rest** ? *implemented* (decision D.2, 2026-05) with **SQLCipher**: transparent AES-256 page encryption via the permissive `sqlcipher3` binding, so SQL queries work unchanged. The 256-bit key is random, lives only in the OS keychain (never in the repo, config, args, or logs), passed in raw-key form. OS full-disk encryption (BitLocker/LUKS) is a recommended complement, not a substitute. See [`docs/encryption-at-rest.md`](./docs/encryption-at-rest.md).
5. **OAuth tokens** stored in the OS keychain (Windows Credential Manager / macOS Keychain), not on disk.
6. **Logs must not contain extracted statement text.** Sanitize aggressively; redact account numbers, balances, line-items.
7. **Nothing sensitive ever in git.** `.gitignore` blocks real PDFs, the DB file, `.env`, and password files. Do not weaken it.
8. **Commercial-license safety.** Only dependencies under permissive, commercial-safe licenses (MIT, BSD, Apache-2.0, ISC, HPND). No copyleft (GPL/LGPL/**AGPL**) or source-available licenses. **PyMuPDF/`fitz` is AGPL-3.0 and is banned** ? use pdfminer.six (MIT) / pypdfium2 (Apache/BSD) instead. Enforced by `tests/test_dependency_licenses.py`; tracked in [`docs/DEPENDENCIES.md`](./docs/DEPENDENCIES.md).
9. **Minimize paid dependencies.** Prefer local / self-hosted / open-source (and, where needed, own-trained) models and tools. Paid APIs ? cloud LLMs, paid OCR/vision ? are a **fallback only**, gated behind explicit per-use approval. This extends rule 2 from privacy to cost.

6. Non-goals (for v1)

- **Multi-user / SaaS / hosted product.** Single user, single machine.
- **Cloud sync of raw data.** Period.
- **Cracking unknown passwords.** muneem decrypts PDFs the user is authorized to read, using passwords the user knows or can derive.
- **Browser scraping bank portals.** Email APIs only. Portal scraping is a fragile, ToS-fraught rabbit hole; we deliberately avoid it.
- **Real-time alerting / push.** Batch-pull on a schedule is sufficient.
- **Tax preparation, investment advice, or anything regulated.** muneem surfaces data; the user decides.

7. Recommended first vertical slice

Before generalizing, prove the end-to-end pipeline on the smallest meaningful scope:

> **Gmail ? one specific bank's monthly statements ? unlocked ? parsed into `transactions` ? stored in SQLite ? a CLI command that lists last month's transactions.**

This exercises every stage exactly once, on a real input the user cares about, with no detours. It will surface the actual hard parts (which we can only guess at now) and force the open decisions in § 8 to get made on real evidence.

Concretely: pick **one** Indian bank statement format the user has many examples of (HDFC, given the user's geography and statement history), build the parser for that one format only, and put the rest of the pipeline around it. Then evaluate before generalizing.

8. Open architectural decisions

These were parked for explicit user discussion (**don't pick silently**). Items **8.1, 8.2, and 8.4 are now resolved** ? see the inline resolution notes below and [`docs/ROADMAP.md`](./docs/ROADMAP.md) § 1. Items **8.3 and 8.5 remain open**.

8.1 Language

Option	Pro	Con
Go only	Single binary, easy distribution, great concurrency for the ingestion side, the user already uses Go for the badminton-highlight-indexer project	Weak PDF parsing / OCR / ML ecosystem; rolling our own is painful
Python only	Best-in-class PDF, OCR, and ML libraries (pdfplumber, PyMuPDF, Tesseract bindings, Ollama clients, every parser under the sun); fastest to prototype	Heavier deployment story; less ergonomic for the long-running ingestion daemon
Go ingestion + Python parsing	Plays to each language's strengths	Two-language project complexity; inter-process boundary to maintain

Recommended default: **Python-only for v1**, accept the deployment-story cost in exchange for moving fast on the hard parts. Revisit if the ingestion side outgrows Python.

> **Resolved (2026-05-30): Python-only for v1.** Adopted the recommended default. See [`docs/ROADMAP.md`](./docs/ROADMAP.md) § 1.

8.2 Password sourcing strategy

See § 4 Stage 2. The hybrid (dictionary ? template ? prompt) is the natural answer, but the design of the rule-template config format and the candidate dictionary needs a user call.

> **Resolved (2026-05-30): Hybrid ? candidate dictionary ? per-sender rule template ? interactive prompt**, caching the winning method per sender; password material never in the repo or logs. The rule-template config format and dictionary design are deferred to implementation (ROADMAP Milestone C.1). See [`docs/ROADMAP.md`](./docs/ROADMAP.md) § 1.

8.3 LLM locality, per document category

The default is "local for sensitive, cloud allowed for non-sensitive after review." But which models? Ollama with Llama 3 / Qwen / Mistral on the user's hardware? A small fine-tuned model? Cloud (Anthropic / OpenAI / Gemini) with strict redaction for the non-sensitive categories?

This decision is closely coupled to the user's hardware capacity (GPU? RAM?) and willingness to wait on inference latency.

8.4 First-slice scope confirmation

§ 7 proposes "Gmail + one bank ? CLI". Confirm with user before building.

> **Resolved (2026-05-30): Build both tracks in parallel** ? the promo-corpus parse?store?query spine *and* the Gmail ingestion plumbing, as two decoupled tracks that converge at the first bank slice; first bank = **HDFC**. This refines § 7 (which proposed a single end-to-end bank slice first). See [`docs/ROADMAP.md`](./docs/ROADMAP.md) §§ 1 and 4.

8.5 Merchant canonicalization approach

Fuzzy string matching has well-known failure modes on merchant names. Do we use a curated rules file (user maintains)? A library? A learned model? Defer until real data exposes the actual mess.

8.6 Ingestion source ? Open Banking (Account Aggregator) vs PDF parsing ? **RESOLVED

(2026-06): PDF-first, AA deferred**

    Researched in depth ([`docs/research-open-banking-india.md`](./docs/research-open-banking-india.md)). India's "open banking" is the RBI **Account Aggregator** framework. A single-user, local-first, no-paid-API tool **cannot** consume AA today ? **FIU status requires a regulated entity** (no individual path), and the only reachable integration rides under a licensed FIU via a **paid TSP that decrypts the entire feed in its cloud** (on-device/self-custody-key decryption is *not* an available AA mode). AA is *safer in transit* than emailed PDFs (E2E-encrypted, data-blind aggregator), but for muneem's threat model (one local honeypot, zero third parties) the realistic integration is a **net privacy regression**. AA also can't cover muneem's full scope (no credit cards / bonds / broker detail / non-demat MF) and still needs the enrichment/reconcile layer. **Decision:** PDF-first core; AA only ever a **deferred, opt-in complement** (import from a regulated FIU's export, oracle-validated, provenance-tagged), with explicit re-adopt triggers (chiefly: a TSP offering true on-device decryption). A new local-only candidate surfaced ? **SMS/email transaction-alert parsing** ? as a freshness complement to PDF.

    ### 8.7 Storage shape ? **RESOLVED (2026-06): flat per-issuer source of truth + derived serving layer**

    Each parser writes its own **flat, isolated per-issuer table** (the source of truth ? regression-isolated, trivially backfilled/rolled back). Cross-source querying is a **derived serving layer** (a SQL view/materialised table now; a separate serving DB only when latency/cost demand). **Provenance + confidence** are first-class but live in a **separate `provenance` table keyed by `document_id`** (not duplicated per row): source_type (pdf/aa/sms), parser, parser_version, mapping_layer, confidence. The reconciliation oracle treats every source as a hint to validate. See [`docs/schema.md`](./docs/schema.md) and [`docs/feedbackJune2026.md`](./docs/feedbackJune2026.md) P3.

    ## 9. Proposed (not built) file layout

    This is a *suggestion* for once the language decision is made. Don't materialize it prematurely.

    ```
    muneem/
    ??? README.md
    ??? CLAUDE.md
    ??? DESIGN.md
    ??? docs/
    ?   ??? prior-art-email-mining.md
    ?   ??? (future: parser-notes-per-issuer.md, schema.md, threat-model.md)
    ??? testdata/
    ?   ??? promo-emails/         # non-sensitive corpus (in repo)
    ?   ??? statements/           # GITIGNORED ? real statements, never committed
    ??? (source tree TBD based on § 8.1)
    ??? (DB file, GITIGNORED)
    ```

    ## 10. Glossary

    - **Muneem** (?????) ? traditional Indian household/business bookkeeper.
    - **Issuer** ? the bank, card network, broker, or merchant that produced a document.
    - **Statement** ? a periodic financial summary from an issuer (bank statement, CC bill, broker statement, MF report).
    - **Holding** ? a position in an investment account at a point in time.
    - **Line item** ? a single transaction or itemized purchase row within a statement or receipt.
    - **Born-digital PDF** ? a PDF generated from a software source with a real text layer (most bank statements). Contrast with scanned/image PDFs.

    ## 11. Vision continuity from Email-Mining

    The predecessor project (`~/Projects/Email-Mining`) extracted promo codes and expiry dates from manually-saved promo email PDFs, using PyMuPDF for text+image extraction and Gemini

for image OCR and summarization. Its scope was just "find the coupon I forgot about."
231
232   muneem **subsumes** that scope. The deals/coupons feature survives as **one of several extraction targets** ? one of the analytics surfaces, fed by the same five-stage pipeline that handles bank statements and everything else. The promo-email PDFs from that project are preserved in [`testdata/promo-emails/`](./testdata/promo-emails/) as the first regression corpus.
233
234   What was learned there and worth keeping: the PDF text+image extraction *shape* is right (but PyMuPDF/fitz itself is AGPL-3.0 ? muneem uses pdfminer.six (MIT) instead; see § 5 rule 8); image dedup by hash + SSIM is a real win for repeated chrome; the shape "page text + per-image OCR ? unified text ? structured extraction" generalizes well. What we change: programmatic Gmail/Outlook fetching instead of manual save-as-PDF; local extraction by default for sensitive categories; persistent normalized DB instead of one-shot stdout.
235
236   Full details in [`docs/prior-art-email-mining.md`](./docs/prior-art-email-mining.md).
237

14. user (2026-06-10T16:13:42.948Z)

```
1       # Architecture
2
3       > Status: living document. Captures the **target architecture** for the
4       > extraction pivot and the **decisions** behind it. Product framing is in
5       > [`VISION.md`](./VISION.md). This supersedes the original "just copy
6       > attachments" mental model; the copier survives as one extractor at the leaf.
7
8       ## Auditability invariant (binding)
9
10      **This repository is the _only_ component that reads or processes a user's
11      Gmail, and it must remain auditable and self-deployable by a power user at any
12      time.** This is the product's core privacy guarantee and a deliberate constraint
13      on every future decision (see ADR-008). It is why the project is built to be
14      open-sourced: *don't trust us ? read, and run, the code.*
15
16      Every change MUST preserve all of the following. A change that cannot is, by
17      definition, a decision to break the invariant and needs explicit owner sign-off.
18
19      1. **Sole Gmail surface.** Gmail is read/processed *only* here, in the user's own
20         account (`executeAs: USER_ACCESSING`). No other component ? Muneem included ?
21         ever holds the Gmail OAuth grant or sees raw mail. Muneem may *orchestrate*
22         this app; it is never *itself* the Gmail reader.
23      2. **Egress == the audited boundary.** Nothing derived from a user's mail leaves
24         their Google account except through the in-repo **REDACT/MINIMIZE** stage,
25         carrying only minimized, non-PII signal, only to the user's own hosted
26         service, never to third parties. The set of egress destinations is explicit
27         and reviewable in source.
28      3. **Run-your-own-from-source.** A power user can build and deploy their own
29         instance from the published source and obtain an equivalent app; the build is
30         reproducible and the verification steps are documented ? "trust by running the
31         code you read," not merely "read the code."
32      4. **No secrets in source.** Credentials exist only as deploy-time secrets; the
33         repository contains nothing sensitive.
34      5. **Transparent over clever** at the read and egress/redaction boundaries ?
35         prefer obviously-correct, reviewable code even at some cost to brevity.
36
37      **PR review checklist (apply to every change):**
38
39      - [ ] Gmail access stays entirely within this repo, in-account?
40      - [ ] If any data leaves the account, does it pass through REDACT/MINIMIZE and go
41            only to the user's own service (non-PII)?
42      - [ ] Build still reproducible and self-deployable from source?
43      - [ ] No new secrets, third-party endpoints, or network calls? (Default: none ?
44            enforced by `tests/auditability.test.ts`.)
45
```

```
## Principles (preserved from today's codebase)

1. **Shells around pure functions.** GAS-facing code is a thin shell; all logic
   is pure and unit-tested.
2. **Quota discipline.** Respect the 6-minute execution limit, the 90-min/day
   trigger budget, and per-user caps. Self-healing trigger fan-out; per-user
   lock; "mark done only on clean completion."
3. **Per-user isolation.** Runs as the accessing user; state is per-user.
4. **Versioning & provenance everywhere.** Every persisted shape carries a
   `schemaVersion`; every extracted entity carries where it came from and which
   extractor (and version) produced it.

## The pipeline

This repo owns the left half; downstream services own the right half. They meet
at the **handoff contract** only.

```
                ?????????????????????????? this repo ??????????????????????????   ?? downstream
??
 Gmail ???  INGEST ??? CLASSIFY ??? EXTRACT ??? REDACT/MINIMIZE ??? HANDOFF ???  Muneem
/
                (rich      (type +     (normalize  (strip PII,         (sink)        grocery
/
                EmailMsg)  confidence) light)      encrypt)                          coupon
svc
                   ?                                                                 ??
parse,
                   ???? RAW ARCHIVE (Drive folders ? optional per type later)
encrypted
                   ???? LOCAL INDEX (Store interface; Sheets now, remote later)
store,

analytics
```

- **INGEST** builds a durable `EmailMessage` **once** per message (body, subject,
  labels, attachment metadata ? via the Gmail advanced service). Extraction
  re-runs over this record without re-touching Gmail or re-copying blobs.
- **CLASSIFY** assigns a type + confidence (sender/subject/body rules now; richer
  models downstream later).
- **EXTRACT** is a registry of pluggable, versioned extractors. The current
  attachment copier becomes the `RawArchive` extractor.
- **REDACT/MINIMIZE** enforces the privacy north star before any egress.
- **HANDOFF** routes a normalized payload to the right downstream **sink**.

Data model

All pure shapes, versioned from day one.

```
EmailMessage {                          // INGEST output ? the stable substrate
  schemaVersion, id, threadId, historyId?,
  from, to[], cc[], subject, date, labels[],
  snippet, plainBody?, htmlBody?,
  attachments: [{ index, name, contentType, sizeBytes,
                  attachmentId/partId, sha256?, driveFileId? }],
}

ExtractedEntity {                       // EXTRACT output ? the queryable payload
  schemaVersion, entityKey,             // stable dedup key, e.g.
                                        //   receipt:{merchant}:{total}:{date}
  type, subtype, confidence,
  fields,                               // typed per kind (receipt, statement, ?)
  provenance: { sourceMsgIds[], attachmentIndex?,
```
```

```
                    extractor, extractorVersion, extractedAt, model? },
      status: pending|extracted|low_confidence|needs_review|corrected,
      links: { driveFileId?, ? },
    }

    HandoffPayload {                        // REDACT/MINIMIZE output ? what leaves
      schemaVersion, entityKey, type, extractorVersion,
      nonPiiFields,                         // minimized, scrubbed signal only
      // encrypted in transit; PII stripped or tokenized
    }
    ```

    - **Entity keys** give content-stable dedup (re-mailed receipts; cross-source
      fusion of an email confirmation + a PDF ticket ? one entity).
    - **Attachment `sha256`** enables content-addressable dedup of re-sent files.
    - **Provenance + `extractorVersion`** make "this extraction is stale ? re-run"
      possible and auditable.

    ## The extraction ? processing boundary

    - This repo produces a normalized, classified, **redacted** `HandoffPayload` per
      detected document/entity and routes it to a downstream **sink**.
    - Downstream services (Muneem for finance; siblings for groceries/coupons) own
      domain parsing, indexing, **encrypted storage**, and analytics.
    - **Contract is owned jointly, defined from our side first.** The exact
      Muneem-facing schema and transport are **TBD and aligned with that repo**
      (this repo's GitHub tooling is scoped to itself, so the contract is specified
      here and reconciled with Muneem separately).
    - Hard rule: this repo imports **no** domain-parsing logic; downstream reads
      no Gmail.

    ## Storage abstraction (decision: extensible, not baked-in)

    A single `Store` / `Sink` interface; code depends only on the interface.

Impl	When	Properties
`GoogleSheetsStore` (user-owned "LifeIndex" sheet)	v1	in-account, queryable, zero
infra, doubles as a first dashboard
`RemoteStore` (our hosted DB) / Muneem-API sink	later	the durable system of record

    `PropertiesService` stays only for small per-user **control state** (cursors,
    flags, counters) ? never the entity index.

    ## Privacy & egress controls

    - **Egress only to our own isolated, hosted service. No third parties.**
    - A **Redactor/Minimizer** stage runs before any egress: strip/tokenize PII,
      keep only non-PII signal, encrypt in transit (encrypted at rest downstream).
    - North star: continuously **shrink** what leaves while keeping the product
      healthy.
    - The in-account tier (Phases 0?2) has **no egress** and needs no privacy
      change. The external tier (Phase 3) is **flag-gated + separately consented**,
      and updates the privacy policy to name exactly what is sent and where.
    - **Cascade to be aware of:** sending restricted-scope Gmail data outside Google
      is what makes Google's **CASA security assessment mandatory for a public
      launch**. Deferred ? consistent with "stay <100 until the user base justifies
      the investment."
    - **Open tension to resolve (Phase 3 design):** bank-statement and receipt
      signal (merchant, amounts, account identifiers) is often itself sensitive.
      "Non-PII only" needs a concrete per-vertical definition of what is sent vs.
      tokenized vs. kept fully in-account.

    ## State & cursors
```
```

```
166
167     - Keep the hybrid `history.list` + date-window skeleton; **extend, don't
168       duplicate.**
169     - Add an **extraction-aware cursor** ("fully extracted up to X with
170       extractor-set V") distinct from the copy cursor, plus per-message extraction
171       status ? so a "re-extract window for new rules" reprocesses without
172       re-copying (content hash) or duplicating entities (entity keys).
173
174     ## Quota, cost & pacing
175
176     - **Separate budgets:** raw-copy cap (existing) vs. extraction cap vs.
177       egress/handoff cap.
178     - Per-item time budgets, **prioritization** (recent + high-confidence/trusted
179       senders first), and **deferral** of heavy items.
180     - Heavy ML/PDF work happens **downstream**, not in Apps Script ? GAS stays the
181       lightweight, reliable watcher/archiver/handoff.
182
183     ## Observability
184
185     - Per-**classifier** and per-**extractor** success / failure / ambiguity rates ?
186       not just "files added."
187     - Surfaced **review queues:** "matched no extractor" and "low confidence."
188     - Structured logs carrying `messageId`; extraction & handoff metrics.
189
190     ## Platform spectrum
191
192     1. **Pure-GAS conservative extractors** (regex, known formats `.ics`/`.csv`,
193        native Drive OCR via Doc conversion) for the easy ~70%, **in-account**.
194     2. **Hosted, isolated service** (Muneem pattern) for heavy/ML extraction,
195        storage, and analytics ? fed only the minimized, non-PII handoff.
196
197     ## Decision log
198
199     | # | Decision | Rationale |
200     | --- | --- | --- |
201     | ADR-001 | Pipeline (ingest ? classify ? extract ? redact ? handoff), not a copy loop |
Treat mail as structured data; make extraction re-runnable over a durable record |
202     | ADR-002 | Extraction (this repo) is decoupled from processing (Muneem & siblings) |
Isolate domain logic + encrypted storage + analytics downstream; clean contract seam |
203     | ADR-003 | Storage via a `Store`/`Sink` interface ? Sheets v1, remote/Muneem later |
Extensible by design; never baked into the code |
204     | ADR-004 | Egress: non-PII only, to our own isolated hosted service, no third parties;
scrub + encrypt; minimize | Privacy north star; preserves trust; in-account baseline needs no
egress |
205     | ADR-005 | Audience: <100 (Testing) now, public later; CASA deferred | Harden + learn
value before investing in assessment/infra |
206     | ADR-006 | First verticals: grocery receipts + bank statements (? Muneem) | Volume +
the high-fidelity handoff path |
207     | ADR-007 | Preserve current strengths (pure/shell, quotas, triggers, per-user,
versioning/provenance) | The recent rewrite is exactly why this pivot is feasible |
208     | ADR-008 | **Option A:** this repo is the sole Gmail-touching component ? in-account,
auditable, self-deployable; Muneem orchestrates but never holds the Gmail grant. Codified as the
binding [auditability invariant](#auditability-invariant-binding). | Verifiable privacy is the
product's moat and shrinks future verification scope; it concentrates the trust-critical surface
in one open, in-account place. |
209
210     ## Open items for the next iteration
211
212     - **Muneem handoff contract:** schema + transport (e.g. `UrlFetch` to a signed
213       endpoint vs. write-to-a-shared-store) ? define jointly with `avidullu/muneem`.
214     - **Per-vertical "non-PII" definition** for receipts and bank statements (what is
215       sent vs. tokenized vs. kept in-account).
216     - **First extractor slice** to build in Phase 1 (recommend a narrow
217       "receipt-smelling" detector) and its golden-email test fixtures.
218
```

15. user (2026-06-10T16:14:09.966Z)

Next wakeup scheduled for 22:10:00 (in 1550s). Nothing more to do this turn ? the harness re-invokes you when the wakeup fires or a task-notification arrives.

16. user (2026-06-10T16:32:42.521Z)

<task-notification>
<task-id>wctcr4nab</task-id>
<tool-use-id>toolu_014Kjje7FJhK2YXaUkpAonwG</tool-use-id>
<output-file>C:\Users\avidu\AppData\Local\Temp\claude\C--Users-avidu-Projects-muneem\02b9e9c9-3806-45ca-9b24-2ecc35a81efd\tasks\wctcr4nab.output</output-file>
<status>completed</status>
<summary>Dynamic workflow "Deep review of muneem + gmail-attachments-to-gdrive: code, security, architecture, roadmap, and market fit" completed</summary>
<result>{"code":[{"key":"muneem-core","summary":"Deep review of muneem's 16 core (non-parser) modules plus their tests. The crypto design is fundamentally sound ? SQLCipher raw-key open with key-before-any-access ordering and verification, no silent plaintext fallback, fresh salt+nonce per recovery-file wrap with sane Argon2id parameters, and value-free keychain error messages ? but the key LIFECYCLE has serious atomicity gaps: `db rotate-key` destroys the only durable copy of the current key before rekeying and its rollback misses non-EncryptedDatabaseError failures (critical, permanent-lockout window), and a transient keychain read failure causes `get_or_create_key` to mint a new key over the real one (high). `db backup` unlinks the destination before validating it is not the live ledger, so `muneem db backup &lt;ledger-path&gt;` silently replaces the ledger with an empty DB (high). Medium items include un-validated key material spliced into PRAGMA strings (non-hex keys silently fall back to SQLCipher passphrase-KDF semantics), forward-migration checksums that are recorded but never re-verified, a legacy-DB backfill that stamps partial/older schemas as current without DDL, an Axis-CC listing bug printing '+Cr' instead of the amount, redaction patterns that miss PAN and spaced card/Aadhaar formats, and rotation silently invalidating the recovery file and prior backups. Low findings cover Windows chmod being a no-op, ATTACH path quoting, recovery-file KDF param validation, a missing mkdir in unlock's plaintext path, and the redaction filter covering only cli.py's logger.","verified":[{"title":"Key rotation overwrites the only copy of the current key before the file is rekeyed; rollback misses realistic failures","severity":"critical","area":"crypto / key lifecycle","file":"src/muneem/cli.py","line":"438-446","description":"`db rotate-key` writes the fresh key into the keychain FIRST (`db_key.set_key(fresh)`), destroying the only durable copy of the current key (it survives only in process memory as `current`), and only then rekeys the file. Two realistic failure modes permanently lock the ledger: (1) a crash/power loss/Ctrl-C between `set_key(fresh)` and a completed `PRAGMA rekey` leaves keychain=fresh, file=old-key, old key gone; (2) the rollback `except` only catches `dbmod.EncryptedDatabaseError`, but `rekey_database` (src/muneem/db.py:274-279) can raise `sqlcipher3.OperationalError` (e.g. 'database is locked' ? very plausible on Windows with another process/AV holding the file) or any other sqlcipher3 error from `PRAGMA rekey`, which escapes uncaught, skips the rollback, and exits with keychain=fresh / file=old. Unless the user happens to have a recovery file (which wraps the OLD key), the entire financial ledger and all same-key backups are unrecoverable. The inline comment ('Keychain-first with rollback: if the file rekey fails, the keychain still matches the file') asserts the opposite of what the code does ? keychain-first guarantees a mismatch window.","evidence":"# Keychain-first with rollback: if the file rekey fails, the keychain still matches the file.\n   db_key.set_key(fresh)\n   try:\n   dbmod.rekey_database(db_path, current, fresh)\n   except dbmod.EncryptedDatabaseError as exc:\n   db_key.set_key(current) # roll back; the file is still under the old key","suggestion":"Never have a moment where only one key exists in durable storage: write the fresh key under a secondary keychain name (e.g. 'db-key-pending') first, rekey the file, then promote it to 'db-key' and delete the pending entry; on open failure, fall back to trying the pending key. At minimum: rekey the file BEFORE touching the 'db-key' entry, broaden the except to catch all exceptions (re-raising after rollback), and stash the old key under 'db-key-prev' before overwriting.","confidence":0.92,"verdict":{"confirmed":true,"votesReal":3,"votesTotal":3,"adjustedSeverities":["high","high","critical"],"notes":["CONFIRM: Traced cli.py:438-446, db.py:266-280 (rekey_database), db_key.py, secrets.py, errors.py, and tests/test_db_crypto_ops.py. Confirmed: the keychain's single 'db-key' entry is overwritten with the fresh key BEFORE the file rekey; the rollback catches only EncryptedDatabaseError (a MuneemError subclass) while PRAGMA rekey/commit raise raw sqlcipher3 errors that escape it; crash/Ctrl-C in the window also bypasses rollback ? all leaving keychain=fresh/file=old with no durable copy of the old key (backups share the same key, so they're lost too). Adjusted critical?high: the trigger is a rare

user-initiated op with a narrow failure window, an optional recovery file (PR #43) wraps the old key, and the ledger is re-derivable from the user's retained source PDFs; also the inline comment is accurate for the one caught path, so 'asserts the opposite' mildly overstates.","CONFIRM: Confirmed in code: cli.py:440 overwrites the keychain 'db-key' (the only durable copy of the current key) before the file rekey, violating the explicit docstring contracts in db.py:271-272 and db_key.py:45, and the inline comment asserts the opposite of the behavior. The uncaught-exception path is real and reachable: EncryptedDatabaseError is a MuneemError unrelated to sqlcipher3 exceptions, and 'database is locked' from a concurrent SHARED-lock holder surfaces exactly at PRAGMA rekey (db.py:276, only try/finally) because the earlier verification SELECT succeeds ? so the rollback is skipped and the process exits with keychain=fresh/file=old; Ctrl-C and disk-full during the page-rewrite window behave the same. I downgrade critical?high because the catastrophe requires a failure during a rare manual operation, the shipped P0 recovery file (db setup-recovery, PR #43) wraps the OLD key and fully recovers a failed rotation if the user set it up, and source statements remain re-ingestable ? but the lockout is deterministic (no crash timing needed) for the locked/disk-full cases, so it stays high.","CONFIRM: Verified in code: cli.py:440-442 overwrites the sole keychain copy of the old key before PRAGMA rekey (keyring.set_password replaces in place, no versioning), and the rollback at cli.py:443 only catches EncryptedDatabaseError while rekey_database (db.py:274-279) leaves PRAGMA rekey/commit unwrapped, so sqlcipher3.OperationalError (e.g. database-is-locked) escapes uncaught and skips rollback. I searched for mitigations and found none: tests cover only the happy path, no pending/prev-key mechanism exists, and the opt-in recovery file is neither required nor refreshed by rotate-key (and goes stale after a successful rotation). The implementation also contradicts the explicit file-first contracts in rekey_database's and set_key's own docstrings, and docs/encryption-at-rest.md repeats the inverted comment ? critical stands given total ledger+backup loss, tempered only slightly by rotate-key being a rare command."]}},{"title":"Transient keychain read failure mints a new key and silently overwrites the only DB key","severity":"high","area":"crypto / key lifecycle","file":"src/muneem/db_key.py","line":"24-31","description":"`secrets.get_secret` (src/muneem/secrets.py:25-30) swallows every `KeyringError` and returns `None`, deliberately treating 'backend unavailable' the same as 'key absent'. `get_or_create_key()` then interprets that `None` as 'no key has ever been minted', generates a fresh key, and `set_secret` OVERWRITES the existing 'db-key' entry. If a read fails transiently (locked/temporarily failing Credential Manager session, keyring backend hiccup) while the subsequent write succeeds, the only copy of the real ledger key is destroyed; the next `connect_encrypted` fails with 'wrong key' and the data is gone unless a recovery file exists. `connect_encrypted` (src/muneem/db.py:207) calls `get_or_create_key()` unconditionally ? even when an encrypted ledger file already exists on disk, which is exactly the case where minting is never the right answer.","evidence":"def get_or_create_key() -> str:
... (truncated 232067 chars, full result in C:\Users\avidu\AppData\Local\Temp\claude\C--Users-avidu-Projects-muneem\02b9e9c9-3806-45ca-9b24-2ecc35a81efd\tasks\wctcr4nab.output)</result>
<usage><agent_count>35</agent_count><subagent_tokens>2567029</subagent_tokens><tool_uses>680</tool_uses><duration_ms>1154787</duration_ms></usage>
</task-notification>

17. user (2026-06-10T16:33:01.152Z)

=== CODE REVIEWERS ===

18. user (2026-06-10T16:33:13.018Z)

LENGTH: 258056
{
  "summary": "Deep review of muneem + gmail-attachments-to-gdrive: code, security, architecture, roadmap, and market fit",
  "agentCount": 35,
  "logs": [
    "Launching 4 code/security reviewers (with verification), 3 doc/test reviewers, and the market-research agent",
    "muneem-security: 10 fi
...
ures/email-receipts/",
        "WellyBox ? automatic email receipt collection ? https://www.wellybox.com/expense-tracker-software/",
        "Zoho Expense for Gmail ? Workspace Marketplace listing ? https://workspace.google.com/marketplace/app/zoho_expense_for_gmail/927860167710"

```
      ]
    }
  }
}
```

19. user (2026-06-10T16:33:27.201Z)

```
=== LOGS ===
Launching 4 code/security reviewers (with verification), 3 doc/test reviewers, and the market-
research agent
muneem-security: 10 findings (0 high/critical to verify)
muneem-core: 14 findings (3 high/critical to verify)
gmail-code: 19 findings (3 high/critical to verify)
muneem-parsers: 12 findings (3 high/critical to verify)
Verification done: 9 high/critical findings confirmed, 0 refuted
=== CODE REVIEWERS ===
```

muneem-core

SUMMARY: Deep review of muneem's 16 core (non-parser) modules plus their tests. The crypto design is fundamentally sound ? SQLCipher raw-key open with key-before-any-access ordering and verification, no silent plaintext fallback, fresh salt+nonce per recovery-file wrap with sane Argon2id parameters, and value-free keychain error messages ? but the key LIFECYCLE has serious atomicity gaps: `db rotate-key` destroys the only durable copy of the current key before rekeying and its rollback misses non-EncryptedDatabaseError failures (critical, permanent-lockout window), and a transient keychain read failure causes `get_or_create_key` to mint a new key over the real one (high). `db backup` unlinks the destination before validating it is not the live ledger, so `muneem db backup <ledger-path>` silently replaces the ledger with an empty DB (high). Medium items include un-validated key material spliced into PRAGMA strings (non-hex keys silently fall back to SQLCipher passphrase-KDF semantics), forward-migration checksums that are recorded but never re-verified, a legacy-DB backfill that stamps partial/older schemas as current without DDL, an Axis-CC listing bug printing '+Cr' instead of the amount, redaction patterns that miss PAN and spaced card/Aadhaar formats, and rotation silently invalidating the recovery file and prior backups. Low findings cover Windows chmod being a no-op, ATTACH path quoting, recovery-file KDF param validation, a missing mkdir in unlock's plaintext path, and the redaction filter covering only cli.py's logger.

-- VERIFIED high/critical --
[critical -> confirmed=True votes=3/3 adj=high,high,critical] Key rotation overwrites the only copy of the current key before the file is rekeyed; rollback misses realistic failures | src/muneem/cli.py:438-446
[high -> confirmed=True votes=3/3 adj=medium,low,medium] Transient keychain read failure mints a new key and silently overwrites the only DB key | src/muneem/db_key.py:24-31
[high -> confirmed=True votes=3/3 adj=high,high,high] `db backup` deletes the destination before any validation ? `muneem db backup <ledger-path>` destroys the ledger | src/muneem/db.py:292-295

-- UNVERIFIED high/critical (overflow) --

-- MEDIUM/LOW --
[medium] Key is spliced into PRAGMA strings with no 64-hex validation ? non-hex keys silently switch SQLCipher to passphrase-KDF mode, and quotes break out of the SQL literal | src/muneem/db.py:109-111
[medium] Forward-migration checksums are recorded but never verified ? only the baseline is drift-checked | src/muneem/migrations.py:133-139
[medium] Legacy/partial-DB backfill records the baseline without running DDL ? older or partially-created DBs are stamped current with missing tables | src/muneem/migrations.py:124-131
[medium] `txns list` prints '+Cr' instead of the amount for Axis credit-card credit rows | src/muneem/cli.py:356-359
[medium] RedactingFilter misses PAN, spaced/hyphenated card numbers, and spaced Aadhaar formats | src/muneem/redaction.py:14-17
[medium] `db rotate-key` silently invalidates the recovery file and all prior backups | src/muneem/cli.py:428-447
[low] encrypt_database: ATTACH path spliced into a SQL literal breaks on apostrophes, and a failed final rename deletes the verified encrypted copy | src/muneem/db.py:234-235, 255-262
[low] unlock(): plaintext-copy path skips parent-directory creation that the encrypted path

performs | src/muneem/unlock.py:69-71
[low] unwrap_key parses attacker-influenced KDF params outside the malformed-payload guard, breaking the RecoveryError contract | src/muneem/recovery.py:90-96
[low] _harden_permissions / chmod 0o600 is a no-op on Windows, the primary platform | src/muneem/db.py:127-134
[low] Redacting logger is opt-in per module and only cli.py opts in; third-party and future module loggers bypass it | src/muneem/log.py:34-43

muneem-parsers

SUMMARY: The parsing core's architecture (concept schema + arithmetic oracle gating persistence) is genuinely strong, and I verified the sensitive testdata dirs (statements, snapshots, credentials) are not git-tracked in any commit. However, deep review found the safety guarantee has real holes exactly where arithmetic cannot see: the savings reconcile-oracle short-circuits on the per-row walk and never consults the printed opening/closing bookends, so a statement missing its leading or trailing transactions persists as 'reconciled' (empirically confirmed); stripping Cr/Dr suffixes without sign semantics lets the L2 mapping search endorse a fully direction-inverted parse of an overdraft statement as verified (empirically confirmed); and HDFC persists all rows even when unreconciled while `txns list` never filters by status. The tolerance equals the currency quantum with a strict-greater comparison, so ~42% of exact one-paisa corruptions pass the walk (measured). Routing is substring-based with alphabetical first-match precedence (an HDFC statement containing a '@aubank' UPI handle routes to AUParser ? confirmed), and sha256 dedup permanently blocks re-ingesting a quarantined document after a parser fix. The Axis credit-card path persists with no verification at all; imap/outlook providers are confirmed experimental and unreachable from the CLI (no fetch command; only manual tools/ scripts), and Gmail uses the minimal read-only scope with keychain token storage.

-- VERIFIED high/critical --
[high -> confirmed=True votes=3/3 adj=high,high,high] Savings oracle ignores available opening/closing bookends when the balance walk passes ? missing leading/trailing transactions persist as 'reconciled' | src/muneem/parsers/savings.py:78-84, 194-205
[high -> confirmed=True votes=3/3 adj=high,high,high] Cr/Dr suffix sign-stripped in parse_amount ? an overdraft (Dr-balance) statement reconciles with every debit/credit INVERTED and persists as verified | src/muneem/parsers/validation.py:32, 45
[high -> confirmed=True votes=3/3 adj=high,high,high] HDFC parser persists all rows even when the parse is UNRECONCILED, and `txns list` never filters by document status | src/muneem/parsers/hdfc.py:435-458

-- UNVERIFIED high/critical (overflow) --

-- MEDIUM/LOW --
[medium] Reconciliation tolerance equals the currency quantum with strict-greater compare ? ~42% of exact one-paisa corruptions pass the balance walk | src/muneem/parsers/validation.py:70-71, 91, 118
[medium] Parser routing by bare substring + alphabetical first-match: a statement mentioning 'aubank' in a narration routes to AUParser | src/muneem/parsers/au.py:33-37
[medium] sha256 dedup permanently blocks re-ingesting a quarantined statement after a parser fix | src/muneem/cli.py:121-124
[medium] Axis credit-card parser persists with zero verification, fixed column indices, sign-stripping, and silent 0.0 coercion | src/muneem/parsers/axis.py:127-149, 169-201
[medium] Dates are never parsed, validated, or normalized ? raw locale strings persisted; arithmetic cannot catch date-column or date-content errors | src/muneem/parsers/engine.py:37-40, 261-263
[medium] Clustering drops whole transactions (the Axis May-2025 mechanism) and all narration continuation lines | src/muneem/parsers/clustering.py:30-40, 82-89
[low] Gmail provider: refresh-failure crash path, unbounded in-memory attachments, sender-controlled filenames returned for future path joins | src/muneem/ingestion/gmail.py:33-37, 92-115
[low] Outlook provider replays a cached access token forever (no expiry/refresh) and silently drops the newer_than_days filter; experimental status of imap/outlook confirmed | src/muneem/ingestion/outlook.py:39-44, 66-67
[low] Dead `safe_amount` helper that maps '0.00' to None is still exported from the parsers package | src/muneem/parsers/base.py:8-21

gmail-code

SUMMARY: Deep correctness review of gmail-attachments-to-gdrive (Apps Script/TS). The date-window backfill core is well-designed (deliberate ±1-day window overlap absorbs timezone skew; fan-out-before-delete trigger replacement is genuinely self-healing; the FIFO + getFilesByName probe prevents duplicate copies). However, the opt-in incremental history sync has several real data-loss/wedge paths: the historyId cursor is advanced even when messages were truncated by the daily cap or failed transiently, GmailApp.getMessageById throws (rather than returning null) for deleted messages which wedges the incremental loop until cursor expiry, pagination truncation at 20 pages silently skips history, and no spam/trash/draft filtering is applied (unlike the backfill query). In backfill mode, the 'mark done only on clean completion' invariant is undermined because lastSynced advances past a window even when attachment-level errors occurred, so failed messages are never retried. Additional defensible issues: getOrCreateFolderInFolder iterates ALL folders in the user's Drive (DriveApp.getFolders() is not 'children of root'), state is persisted only at run end so a 6-minute hard kill undercounts the daily cap, setup re-runs race in-flight syncs (no lock in processInput), CI creates a brand-new deployment per push instead of updating one in place, and execution logs containing Gmail message ids land in developer-visible Cloud Logging, in tension with the privacy policy's 'author has no access' claim.

-- VERIFIED high/critical --
[high -> confirmed=True votes=3/3 adj=high,medium,medium] Incremental sync advances historyId past messages that were not fully processed (silent data loss) | src/sync.ts:175-189
[high -> confirmed=True votes=3/3 adj=medium,medium,medium] GmailApp.getMessageById throws on deleted messages, wedging incremental sync until the cursor expires | src/gmailHistory.ts:70-77
[high -> confirmed=True votes=3/3 adj=high,high,high] Backfill advances lastSynced past windows that had attachment-level errors, so failed messages are never retried | src/sync.ts:213-218

-- UNVERIFIED high/critical (overflow) --

-- MEDIUM/LOW --
[medium] getOrCreateFolderInFolder(DriveApp, ...) iterates ALL folders in the user's Drive, not root children | src/drive.ts:87, 12-15, 34-44
[medium] No runtime-budget check inside updateDrive; all bookkeeping persisted only at run end, so the 6-minute hard kill loses it | src/sync.ts:199-203, 233-244
[medium] Incremental path applies no spam/trash/draft/sent filtering, diverging from the backfill query | src/gmailHistory.ts:37-42
[medium] History pagination truncated at MAX_HISTORY_PAGES silently skips changes (cursor jumps to current mailbox id) | src/gmailHistory.ts:12, 37-48
[medium] History-expiry recovery rewinds a fixed 30 days; lastSynced is never advanced during incremental mode, so longer outages lose mail | src/sync.ts:170-173
[medium] Setup re-run races an in-flight sync: processInput takes no lock, so the running sync's end-of-run writes clobber fresh setup state | src/ui.ts:58-72
[medium] Distinct attachments sharing a filename are silently dropped while the message is marked done | src/drive.ts:128, 148, 177-180
[medium] Execution logs (Gmail message ids, raw error text) flow to developer-visible Cloud Logging, in tension with PRIVACY.md claims | src/logger.ts:8-10
[medium] CI creates a brand-new Apps Script deployment on every push instead of updating the existing one | .github/workflows/ci.yml:54
[medium] Unpaged GmailApp.search caps results (~500 threads); denser 30-day windows can permanently skip mail | src/sync.ts:208
[low] getFilesAddedToday parses JSON unguarded, outside the run's try/catch ? a malformed value causes a permanent crash-loop | src/state.ts:49-55
[low] logError severity claim is wrong: Logger.log always lands at INFO in Cloud Logging | src/logger.ts:4-9, 24-27
[low] Backfill query 'to:me' misses cc/bcc-received mail, and diverges from the incremental path | src/query.ts:23
[low] Properties-wipe recovery path re-copies up to a year of attachments as duplicates into the root folder | src/sync.ts:96-100
[low] SUMMARY_EMAIL_RECIPIENT is a global constant: in a multi-user deployment every user's summary is mailed (as them) to one hardcoded address | src/config.ts:71-72
[low] Single script timeZone (America/Los_Angeles) governs day boundaries, cap resets and summaries for all users | appsscript.json:2

muneem-security

SUMMARY: muneem's security posture is unusually strong for a personal project: git ground truth

is verified clean (nothing sensitive tracked now or ever in history, including the full --all log over statements, credentials, snapshots, env and DB patterns), the SQLCipher encryption seam is enforced by tested invariants that make plaintext-ledger regressions a failing build, secrets live exclusively in the OS keychain with value-free error messages, and CI uploads no artifacts and physically cannot see real statements. The real residual risk is concentrated in two places. First, information-at-rest inside the repo working directory: real statement PDFs, parser snapshots containing real credit-card rows, and a real OAuth client secret all sit unencrypted in testdata/ ? untracked and hook-blocked, but exposed to sync/backup/malware and one forced add away from GitHub, and the strongest guard layer (the pre-commit hook) is opt-in and undocumented for the Windows dev platform, with the CI backstop firing only after a bad commit has already reached GitHub. Second, supply chain: every dependency is unpinned with no lock or hash checking, including the native sqlcipher3 wheel that provides the at-rest encryption itself. Specific smaller gaps: client_secret_*.json escapes all three guard layers once outside testdata/, the recovery passphrase floor is 8 characters on an offline-attackable file, local-statement tests write real parsed rows to plaintext temp SQLite files, and the redaction backstop misses spaced card numbers and unredacted filenames in eval output. All findings have concrete, low-effort fixes; none indicate sensitive data has actually leaked.

-- VERIFIED high/critical --

-- UNVERIFIED high/critical (overflow) --

-- MEDIUM/LOW --
[medium] Real-statement derivatives sit unencrypted in the repo working dir (snapshots, statements, OAuth client) | tests/conftest.py:27-48
[medium] client_secret_*.json escapes every guard layer once outside testdata/ | .githooks/precommit_check.py:25,48-51
[medium] Pre-commit guard is opt-in and undocumented for Windows; CI backstop fires only after the secret is already on GitHub | Makefile:20-22
[medium] Supply chain: no version pinning, no lock file, no hash checking; native crypto binaries pulled unpinned | pyproject.toml:13-28
[low] Recovery passphrase floor of 8 characters on an offline-attackable file | src/muneem/recovery.py:41,61-62
[low] local_statements tests persist real parsed transactions into plaintext SQLite files in pytest temp dirs | tests/parsers/test_local_au.py:39-44
[low] Redaction backstop has pattern gaps; eval output prints unredacted statement filenames | src/muneem/redaction.py:15-17
[low] unlock.write_decrypted is an unused footgun that writes decrypted statement copies to disk | src/muneem/unlock.py:41-53
[low] Transient keychain read failure can mint and store a divergent DB key (availability, not confidentiality) | src/muneem/db_key.py:24-31
[low] Strengths: layered hygiene with verified-clean history, hard encryption seam, value-free diagnostics, sterile CI | tests/test_repo_hygiene.py:

20. user (2026-06-10T16:33:38.816Z)

<persisted-output>
Output too large (66.9KB). Full output saved to: C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-muneem\02b9e9c9-3806-45ca-9b24-2ecc35a81efd\tool-results\bxoew9pc1.txt

Preview (first 2KB):
########## muneem-tests-ci ##########
SUMMARY: muneem's test/CI architecture is genuinely strong for a pre-1.0 personal project: the offline suite (390 tests) passes in 6.35s at 86.17% coverage against the 80% CI gate, CI really does run ruff+mypy+pytest on a Linux 3.11-3.13 + Windows 3.13 matrix, the schema-lock and plaintext-downgrade tripwires exist and work, the license allowlist is enforced as a test, and sensitive paths are defended in depth (gitignore default-deny, tested pre-commit guard, repo-hygiene backstop; git history verified clean). The biggest gaps: the key-rotation CLI's failure/rollback path has zero test coverage and its "keychain-first" ordering contradicts its own docstrings (a crash mid-rotation orphans the ledger key); real parsed credit-card transactions sit in plaintext JSON in testdata/snapshots/, written by a now-dead fixture; the real-statement test layer has no CI story at all today (Layer-2 self-hosted runner is design-only, local tests skip silently without passwords); synthetic full-pipeline fixtures cover only 2 of 4 parser families with idealized 3-row single-page layouts; and Google's default OAuth filename pattern (client_secret_*.json) is not blocked by any guard outside testdata/.

MAP: ARCHITECTURE (verified by reading + running):
- CI (.github/workflows/ci.yml): job "test" on push/PR/dispatch ? matrix ubuntu {3.11,3.12,3.13}
+ windows-latest 3.13; steps: pip install -e .[dev], ruff check, ruff format --check, mypy,
pytest --cov=muneem --cov-fail-under=80. Job "ocr" (Tesseract, pytest -m ocr) runs ONLY on
workflow_dispatch.
...
</persisted-output>

21. user (2026-06-10T16:33:44.045Z)

<system-reminder>[Truncated: PARTIAL view ? showing lines 1-133 of 163 total (25884 tokens, cap 25000). Call Read with offset=134 limit=133 for the next page, or Grep to find a specific section. Do NOT answer from this page alone if the answer may be further in the file.]</system-reminder>

1
2 ########## muneem-tests-ci ##########
3 SUMMARY: muneem's test/CI architecture is genuinely strong for a pre-1.0 personal project: the offline suite (390 tests) passes in 6.35s at 86.17% coverage against the 80% CI gate, CI really does run ruff+mypy+pytest on a Linux 3.11-3.13 + Windows 3.13 matrix, the schema-lock and plaintext-downgrade tripwires exist and work, the license allowlist is enforced as a test, and sensitive paths are defended in depth (gitignore default-deny, tested pre-commit guard, repo-hygiene backstop; git history verified clean). The biggest gaps: the key-rotation CLI's failure/rollback path has zero test coverage and its "keychain-first" ordering contradicts its own docstrings (a crash mid-rotation orphans the ledger key); real parsed credit-card transactions sit in plaintext JSON in testdata/snapshots/, written by a now-dead fixture; the real-statement test layer has no CI story at all today (Layer-2 self-hosted runner is design-only, local tests skip silently without passwords); synthetic full-pipeline fixtures cover only 2 of 4 parser families with idealized 3-row single-page layouts; and Google's default OAuth filename pattern (client_secret_*.json) is not blocked by any guard outside testdata/.
4
5 MAP: ARCHITECTURE (verified by reading + running):
6 - CI (.github/workflows/ci.yml): job "test" on push/PR/dispatch ? matrix ubuntu {3.11,3.12,3.13} + windows-latest 3.13; steps: pip install -e .[dev], ruff check, ruff format --check, mypy, pytest --cov=muneem --cov-fail-under=80. Job "ocr" (Tesseract, pytest -m ocr) runs ONLY on workflow_dispatch.
7 - Local gates (.githooks/, active on the dev box: core.hooksPath=.githooks): pre-commit runs precommit_check.py (blocks staging PDFs, anything under testdata/ except promo-emails|synthetic, DBs, key material, password_rules.*, .env, recovery files); pre-push runs ruff+format+mypy+full offline pytest (no coverage gate). Hooks are the substitute for branch protection (private repo, none available). Makefile is Linux/macOS-only convenience.
8 - pytest config (pyproject): addopts deselect markers ocr/llm/local_statements ? default suite is offline/deterministic/secretless. mypy covers src/muneem but ingestion.* has ignore_errors=True (documented as temporary scaffolding).
9 - Test tiers: (1) offline default ? unit + synthetic full-pipeline (tests/test_regression_synth.py + tests/synthpdf.py: stdlib hand-rolled born-digital PDFs with Helvetica metrics for right-aligned amount columns, run through real pdfminer/pdfplumber?parser?reconcile; covers HDFC savings + AU savings layouts only); (2) ocr marker ? promo corpus T2, manual-dispatch CI only; (3) local_statements marker ? tests/parsers/test_local_axis.py, test_local_au.py, plus local tests in test_hdfc.py and test_unlock.py run real encrypted statements from gitignored testdata/{axis,aubank,hdfc}-statements/; they skip unless a password is available (password_rules.yaml or MUNEEM_TEST_*_PW env); never in CI; ROADMAP Layer-2 self-hosted runner is designed but NOT implemented; (4) llm marker ? never in CI.
10 - Tripwires: tests/test_db_invariants.py pins EXPECTED_SCHEMA_VERSION=5 + exact table/column shapes (explicit-allow = edit the constants in the same PR), plus security invariants: CLI ingest must write an encrypted ledger (is_plaintext_sqlite check), keyed open must raise (never plaintext-fallback) when sqlcipher3 missing, FK enforcement/cascade, sha256 dedup unique. tests/test_repo_hygiene.py asserts git ls-files contains no sensitive paths/PDFs outside promo-emails. tests/test_dependency_licenses.py allowlists every pyproject dep (MIT/BSD/Apache/Zlib/HPND) and bans GPL/AGPL substrings.
11 - Verifier fuzz: tests/test_verifier_properties.py ? 180 seeded-random cases (consistent ledgers accepted; single perturbation past tolerance caught by balance walk and total oracle). Deterministic, no hypothesis dep.
12 - Isolation: tests/conftest.py autouse _isolate_keychain replaces muneem.secrets

get/set/delete with an in-memory dict ? the real OS keychain (incl. DPAPI on the Windows runner) is NEVER touched by any test anywhere. conftest also defines a `snapshot` fixture (writes/reads testdata/snapshots/*.json, UPDATE_SNAPSHOTS=1 to regenerate) that NO test currently uses.
13   - RUN RESULT (C:/Users/avidu/Projects/muneem/.venv, python 3.13.13, CI invocation): 390 passed, 21 deselected (ocr + local_statements incl. parametrized real-PDF ids), 6.35s, coverage 86.17% (gate 80% passed). Per-module: cli.py 70%, ingestion/outlook.py 38%, ingestion/imap.py 50%, ocr.py 57%, parsers/base.py 58%; parsers/db/migrations/validation all 92-100%.
14   - Sensitive-data ground truth (existence/tracking only): testdata/{aubank,axis,hdfc}-statements (4/7/7 PDFs), testdata/credentials/ (one real client_secret_*.apps.googleusercontent.com.json), testdata/snapshots/ (two JSONs that ARE real parsed CC transactions ? list of rows keyed account_number/amount/merchant_category/particulars/txn_date/type; 12 and 28 rows) ? ALL untracked, gitignored, and `git log --all` over those paths is empty (never committed). Only promo-emails PDFs are tracked.
15
16   STRENGTHS:
17   * The offline suite is real and fast: 390 tests, 6.35s, genuinely offline/deterministic/secretless (keychain autouse-isolated, no network, seeded RNG, tmp_path everywhere) ? verified by running it; 86.17% coverage clears the 80% CI gate.
18   * CI matrix matches the claim: Linux 3.11/3.12/3.13 + Windows 3.13 (the dev platform, exercising the sqlcipher3 win wheel and Windows paths), with ruff lint+format, mypy, and the coverage gate all actually wired.
19   * Defense-in-depth on sensitive data actually holds: .gitignore default-denies testdata/* with explicit allowlist, the pre-commit guard is itself unit-tested (32 tests via PRECOMMIT_TEST_FILES injection, portable, no git mutation), test_repo_hygiene.py is the post-commit backstop, and git history is verifiably clean of statements/credentials/snapshots.
20   * The DB tripwires are well designed: schema lock with an explicit-allow mechanism (edit EXPECTED_SCHEMA in the same PR), plus always-hold invariants ? CLI ingest writes a genuinely encrypted ledger end-to-end, keyed opens never silently downgrade to plaintext, FK cascade and sha256 dedup enforced.
21   * tests/synthpdf.py is a clever zero-dependency solution: hand-rolled valid PDFs with real Helvetica metrics so right-aligned amount columns land in the clusterer's bands, letting the full pdfminer/pdfplumber ? clustering ? engine ? reconcile pipeline run in cloud CI with no committed statements.
22   * The verifier (the project's truth oracle) gets 180 seeded property-fuzz cases including single-perturbation detection ? proportionate rigor for the component whose failure means silently wrong financial data.
23   * Local-only real-statement tests assert the right invariant (store-nothing or store-reconciled, surfacing only counts/booleans/parser names, never statement content), and pytest output discipline keeps contents out of logs.
24   * Leak-prevention is tested as behavior, not policy: redaction filters, ParseError wrapping (no path/PII digits in messages), encryption key never in error text, recovery file leaks neither key nor passphrase.
25   * License allowlist enforced as a failing test synced to pyproject (every dep must be allowlisted; GPL/AGPL substrings banned), directly encoding the commercial-safety memory rule.
26   * The pre-push hook runs the full local gate (ruff, format, mypy, offline suite) as a deliberate substitute for unavailable branch protection, with a documented --no-verify escape hatch, and it is actually enabled on the dev machine.
27
28   ISSUES:
29   [high] Key-rotation failure path has zero test coverage and the implementation contradicts its own documentation
30   muneem db rotate-key sets the NEW key in the keychain FIRST, then rekeys the file. The cli.py comment claims 'Keychain-first with rollback: if the file rekey fails, the keychain still matches the file' ? backwards: between set_key(fresh) and a successful rekey, the keychain does NOT match the file. A crash, Ctrl-C, or any exception other than EncryptedDatabaseError in that window leaves the ledger under the old key with only the new key stored ? locked out unless the recovery file exists. db_key.set_key's docstring says it is used 'after the file has been rekeyed', which the code does not do. Coverage confirms the except/rollback branch (cli.py lines 443-446) is never executed by any test; only the happy path (test_db_rotate_key_cli) and the no-ledger refusal are tested. The rollback set_key(current) call can itself fail (keychain error) with no defined behavior.
31   [medium] Real parsed credit-card transactions sit in plaintext JSON on disk, written by a now-dead snapshot fixture
32   testdata/snapshots/ contains two JSON files that are lists of real parsed CC

transaction rows (12 and 28 rows; keys: account_number, amount, merchant_category, particulars, txn_date, type ? values not inspected). They are gitignored, untracked, and verifiably never committed, so there is no repo leak ? but they are cleartext on the same disk the project encrypts the ledger to protect, directly undermining the at-rest posture. The `snapshot` fixture in tests/conftest.py that produced them is no longer used by ANY test (grep confirms only conftest references), yet it remains as a loaded footgun: UPDATE_SNAPSHOTS=1 writes real parsed data to a cwd-relative path with no redaction.
33 [medium] No CI story for real statements today; local_statements tests skip silently and nothing tracks freshness
34 The Layer-2 self-hosted runner (ROADMAP § Layer 2) is design-only ? no .github/workflows/real-statements.yml exists. The local_statements tests skip unless a bank password is resolvable, the default pytest addopts exclude them, and the pre-push hook runs the default suite only ? so a parser regression against real statement vintages (the product's actual job) is detected only if the developer remembers to run `pytest -m local_statements` manually. There is no record of when they last passed, and a skip looks identical to never-run.
35 [medium] Migration runner has an acknowledged-but-untested partial-failure window
36 migrations.py applies each migration via executescript (which commits implicitly) and records it in schema_migrations afterwards; the module docstring admits a crash between DDL and recording 'could require manual cleanup' and defers per-migration transactions. No test exercises a migration failing mid-script (partial DDL applied, nothing recorded) or failure between apply and record ? test_migrations.py covers baseline, idempotency, back-fill, forward apply, drift detection, and encrypted connections, all happy-path or clean-rejection. With v5 (provenance) now shipped and more migrations planned (unified schema P3), this window is live on an encrypted production ledger where 'manual cleanup' is harder.
37 [medium] Synthetic Layer-1 fixtures are far cleaner than real statements and cover only 2 of 4 parser families
38 test_regression_synth.py builds single-page PDFs with exactly 3 perfectly-aligned rows. There is no multi-page table (repeated headers, page-break row splits), no wrapped/multi-line narration, no merged or interleaved columns at the real-PDF level (those are tested only on hand-built cell matrices via pdfplumber mocks in test_savings/test_axis), no password-protected PDF through the real pipeline (unlock is tested separately with pypdf-made blanks; CLI ingest password flow is monkeypatched), and no Axis savings or Axis CC synthetic fixture at all ? only HDFC and AU layouts route through the full pdfplumber path in CI. The real-world failure the suite itself names (the Axis May 2025 dropped-row case) is reproduced only at matrix level, not as a born-digital PDF.
39 [medium] OCR tier is effectively unmonitored: the CI job only runs on manual dispatch
40 The ocr job in ci.yml is gated on `github.event_name == 'workflow_dispatch'`, so the T2 image-code tier (about half the promo corpus per docs/TESTING.md § 2) and ocr.py (57% line coverage in the default suite) get no automatic regression signal ? a Tesseract-interaction or ocr.py regression ships silently unless someone remembers to trigger the job. No schedule trigger exists.
41 [medium] Guard rails miss Google's default OAuth client filename pattern outside testdata/
42 The real OAuth client file on disk is named client_secret_<id>.apps.googleusercontent.com.json ? Google's default download name. Inside testdata/ it is protected by the blanket testdata rules, but if it were copied to the repo root or src/ (a common tutorial pattern), nothing blocks it: .gitignore lists only credentials.json/token.json/token.pickle/oauth_token*, precommit_check.py SECRET_NAMES has only 'credentials.json', and test_repo_hygiene FORBIDDEN_NAMES likewise. A `git add -A` of a root-level client_secret_*.json would pass all three guards. (Also minor: FORBIDDEN_PREFIXES in test_repo_hygiene references stale dirs testdata/statements/ and testdata/private/ that do not exist, though the broader noncorpus-testdata rule compensates.)
43 [low] Aggregate 86% coverage masks weak spots: CLI error paths 70%, ingestion 38-76% and double-exempted
44 The 80% gate measures the right package (--cov=muneem resolves to src/muneem via the editable install ? verified in the run output) but is aggregate-only. Untested regions include the rotate-key rollback (above), db backup error branch (cli.py 462-464), recovery CLI partial paths (486-497, 528-536), offers OCR-merge branch (258-270), offers/txns list formatting (307-361), and the unlock interactive paths. ingestion/outlook.py (38%) and imap.py (50%) are simultaneously exempt from mypy (ignore_errors=True) ? scaffolding with neither type nor behavioral pressure, yet its lines inflate denominator dynamics of the single gate. Erosion in cli.py can be offset by well-covered parser code without tripping anything.
45 [low] The real OS keychain backend is never exercised anywhere ? including on the Windows CI runner
46 The autouse _isolate_keychain fixture (correct for hygiene) means no test on any

platform ever touches a real keyring backend; test_secrets.py drives the wrappers by mocking the keyring library. So the production path ? DPAPI via keyring on the dev box, get_or_create_key minting on first run, keychain-unavailable behavior on headless setups ? has zero integration coverage; its first real failure will be discovered with real data at stake. The Windows CI matrix cell, justified in ci.yml partly by 'DPAPI secret backend', does not actually test DPAPI.
47 [low] Pytest IDs of local tests embed real statement filenames
48 test_local_axis.py parametrizes with ids=lambda p: p.name, so collected test IDs include real statement filenames (bank + statement month). These appear in local pytest/collection output and would appear in any future Layer-2 runner logs. Filenames only ? no account data ? but the project's own redaction discipline (redact_filename masks such names in errors/logs) is bypassed by test IDs.
49
50 RECOMMENDATIONS:
51 [short-term] Test and harden the rotate-key failure window: Add CLI tests that monkeypatch rekey_database to raise (a) EncryptedDatabaseError ? assert the keychain is rolled back to the old key ? and (b) a generic exception/KeyboardInterrupt ? decide and assert the behavior. Fix the ordering or the docs: either rekey-the-file-first then set_key (matching db_key.set_key's docstring), or keep both keys reachable during rotation (e.g. store the outgoing key under a db-key-previous entry until the rekey verifies) so no crash window can orphan the ledger. Also define behavior when the rollback set_key itself fails.
52 [short-term] Remove the dead snapshot fixture and securely delete the real-data snapshots: Delete the unused `snapshot` fixture from tests/conftest.py (no test uses it) and remove the two real-transaction JSON files from testdata/snapshots/. If parse-snapshotting of real statements is wanted later, redesign it to store only non-sensitive summaries (counts, reconcile booleans, hashes) or write into the encrypted ledger, never cleartext JSON.
53 [short-term] Block the client_secret_*.json filename pattern everywhere: Add client_secret*.json (and *.apps.googleusercontent.com.json) to .gitignore, to a pattern check in .githooks/precommit_check.py (alongside SECRET_NAMES), and to tests/test_repo_hygiene.py FORBIDDEN_NAMES ? Google's default download name is the most likely real-world filename for this secret. While there, update the stale FORBIDDEN_PREFIXES to the directory names actually in use.
54 [short-term] Add a migration partial-failure test and per-migration transactions: Write a test with a two-statement migration whose second statement fails, asserting the DB is left in a defined state (ideally fully rolled back and unrecorded). Wrap each forward migration's DDL + schema_migrations insert in one transaction (the module docstring already flags this as the known limitation) before migration v6 lands ? closing it is cheap now and expensive after a production ledger hits the window.
55 [short-term] Broaden the synthetic Layer-1 corpus: Extend tests/test_regression_synth.py with: an Axis savings and an Axis CC born-digital fixture (so all four parser families route through real pdfplumber in CI), a multi-page statement with repeated headers, a wrapped two-line narration row, a dropped-row fixture reproducing the Axis May 2025 must-not-reconcile case at the PDF level, and a password-protected synthetic PDF pushed through the actual `muneem ingest` CLI (synthpdf could gain RC4/AES encryption via pypdf, already a dependency).
56 [short-term] Put the OCR job on a schedule: Add `schedule: cron` (e.g. weekly) to the ocr job's triggers in ci.yml so the T2 tier and ocr.py get automatic regression signal instead of relying on someone remembering workflow_dispatch.
57 [long-term] Implement the Layer-2 real-statement runner (or an interim freshness mechanism): Stand up the decided-but-deferred self-hosted runner (ROADMAP § Layer 2) running `pytest -m local_statements` on pushes. Until then, add a low-tech freshness signal: a local scheduled task or a `make test-local` habit that runs the local tier and writes a last-green timestamp the default suite can warn about (e.g. a non-failing report when real-statement tests haven't passed in N days), so silent skips can't quietly become months of untested real-vintage parsing.
58 [long-term] Split the coverage gate so the aggregate can't hide erosion: Keep the 80% global gate but add per-area floors ? e.g. fail-under on muneem.parsers+db+migrations (currently ~95%) at a high bar, a separate explicit (lower) expectation for cli.py error paths, and either exclude muneem.ingestion from coverage until the fetch command lands or, when it does, remove both the mypy ignore_errors override and the coverage slack in the same PR.
59 [long-term] Add an opt-in real-keychain integration tier: Introduce a `keychain` marker (mirroring ocr/local_statements) with a handful of tests that exercise the real keyring backend ? get_or_create_key mint/reuse, set/delete, rotation end-to-end against DPAPI ? run manually on the dev box and later on the self-hosted runner, using a dedicated test service name so the production db-key entry is never touched.
60 [long-term] Extend property fuzzing beyond the verifier: The seeded-fuzz pattern in test_verifier_properties.py is cheap and effective; apply it to the clustering and engine layers

(random column geometries, jittered x-positions, randomized row counts feeding synthpdf) so the layout-inference code ? the most likely source of silent mislabeling after the verifier ? gets the same adversarial pressure. Consider hypothesis (MIT, allowlist-compatible) if the hand-rolled generators grow unwieldy.
61
62 ########## muneem-docs ##########
63 SUMMARY: muneem's doc suite is unusually honest and mostly code-accurate at the operational layer (schema.md, ingesting-statements.md, encryption-at-rest.md match the code line-for-line; Outlook/IMAP are marked experimental in code exactly as promised; sensitive testdata is untracked and was never in git history). The debt is at the strategic layer: (1) sequencing authority is split between ROADMAP.md's Milestones/Tracks and feedbackJune2026 §4's Phases A?H, with colliding letter namespaces and neither doc fully owning the plan; (2) the designated "live status" doc (CURRENT_STATE.md) and parser-architecture.md both lag shipped reality (schema v5/provenance, Axis 6/7); (3) Phase C (instrument family) is the riskiest, least-specified phase ? no broker corpus evidenced, no instrument reconciliation oracle designed, sequenced before the regression-CI safety net the project itself decided it needs; (4) the roadmap has no phase delivering daily user value (query/reporting), no cash/manual-entry plan, no multi-year backfill/archive strategy, and a stale v1 definition of done; and (5) a head-on, unacknowledged contradiction with the sister repo gmail-attachments-to-gdrive, whose three-day-old "binding auditability invariant" declares muneem must never hold the Gmail grant ? while muneem already ships full Gmail OAuth ingestion and plans `muneem fetch` (Phase F), and bank-statement PDFs are PII that cannot flow through the sister's "non-PII minimized signal" contract anyway. The still-open decisions (log format, self-hosted runner, macOS) are genuinely still open in code ? no silent locks found, with one partial lapse around the L3 model choice never being back-recorded in DESIGN §8.3/ROADMAP §1.
64
65 MAP: PROJECT: muneem (C:/Users/avidu/Projects/muneem), Python 3.11+, single-user local-first personal-finance bookkeeper for Avi (Bangalore; Indian banks/brokers). Successor to ~/Projects/Email-Mining (promo-code extractor prototype).
66
67 VISION (DESIGN.md): ingest financial docs from email -> unlock password PDFs -> parse -> normalized local SQLite -> analytics over the whole financial life (groceries price-intel, portfolio timeline, bank/card comparison, item catalog, forgotten deals). Five-stage pipeline: (1) Email ingestion (Gmail/Outlook APIs, OAuth read-only), (2) Attachment unlocking (passwords derived from user attributes), (3) Document parsing ("the hard 80%": template parsers + text-layer heuristics + OCR + LLM fallback), (4) Normalization & storage (SQLite; documents/accounts/transactions/holdings/merchants/items/offers), (5) Analytics (CLI first). Security posture: everything local; no cloud LLM on sensitive categories without per-session approval; SQLCipher encryption at rest (implemented); permissive licenses only (PyMuPDF/AGPL banned); no paid APIs by default. Non-goals: SaaS, cloud sync, portal scraping, real-time.
68
69 OPEN DECISIONS (DESIGN §8 + feedbackJune2026 §5): 8.1 language=RESOLVED Python-only (2026-05-30). 8.2 passwords=RESOLVED hybrid dictionary->template->prompt (implemented in `muneem ingest`). 8.3 LLM locality for sensitive docs=OPEN (but the L3 vision model is researched-and-chosen in parser-architecture.md: DeepSeek-OCR-3B MIT, 4-bit+SDPA on RTX 2070S, build deferred ? not back-recorded in DESIGN §8.3, and ROADMAP §9 still says "unconfirmed"). 8.4 first slice=RESOLVED both tracks parallel, HDFC first. 8.5 merchant canonicalization=OPEN (parked until real receipt data). 8.6 Account Aggregator=RESOLVED 2026-06 deferred/PDF-first (research-open-banking-india.md: FIU regulatory gate; TSPs decrypt in cloud = net privacy regression; SMS-alert parsing surfaced as local freshness complement). 8.7 storage shape=RESOLVED 2026-06 flat per-issuer source-of-truth tables + derived serving layer + separate provenance table keyed by document_id. feedback §5 still-open: log format (logfmt vs JSON), self-hosted-runner acceptance, macOS first-class. Verified in code: none silently locked (log.py has no formatter; only ci.yml exists, ubuntu+windows matrix, no real-statements.yml, no macOS).
70
71 SEQUENCING DOCS: ROADMAP.md (created 2026-05-30) tracks Milestone 0 (done), Track A promo spine (done), Track B Gmail ingestion (OAuth+search+download done; `muneem fetch` B.4 NOT built), Milestone C HDFC slice (parsers+unlock done; unified-schema C.3 pending), Milestone D harden/evaluate (D.2 encryption done; D.1 logging and D.3 eval writeup pending), §6 generalize backlog, plus a "Planned ? Regression CI" section (two-layer: synthetic cloud CI [Layer 1 exists: tests/synthpdf.py + test_regression_synth.py, all-branch triggers live] + self-hosted runner over real statements [decided, NOT built]). feedbackJune2026.md §4 (approved 2026-06) defines a DIFFERENT phase plan: A foundation/recovery (recovery #43, migrations #44, Windows CI #42, Outlook/IMAP-experimental #41 done; lockfile and structured-logging NOT done) -> B provenance table + serving view (provenance table shipped #46; serving view NOT built) -> C

instrument family (equity/MF/ETF/bond) -> D regression CI Layer 2 + D.3 eval -> E L3 vision -> F `muneem fetch` -> G merchant canon/Tier-2 -> H ops surface (verify/export/filters) + SBOM. CURRENT_STATE.md is named the live status doc; its §3 still says "Schema v4" and calls a unified table "with per-row provenance/confidence ... the planned direction once the design decision lands" ? both superseded by the §8.7 decision and the shipped v5 provenance migration.
72
73   CODE STATE (verified): CLI = info, ingest (promo + --doc-type bank_statement with hybrid unlock, --ocr, --no-prompt), offers list, txns list {hdfc,axis,axis_cc,au}, eval <folder> (reconcile-rate report), db {status,encrypt,rotate-key,backup,setup-recovery,recover}. No fetch. db.py: SCHEMA_VERSION=4 baseline (documents, offers, axis/axis_cc/au/hdfc txn tables) + migrations.py v5 provenance (document_id UNIQUE FK, source_type/parser/parser_version/mapping_layer/confidence) ? schema.md mirrors this exactly. record_parse/record_provenance called from hdfc.py/axis.py/au.py. Parser core: concepts.py + engine.py (FamilySpec-parameterized L0 lexicon -> profile order -> L2 content search, oracle-selected) + profiles.py (AXIS/AUBANK/HDFC as data) + savings.py shared path + validation.py verifier ? matches parser-architecture.md §§3?6. Real-data precision per ROADMAP/CURRENT_STATE: HDFC 6/6, AU 4/4, Axis 6/7 (May 2025 quarantined, stores nothing); parser-architecture.md header/§10/§10a still say Axis 2/4 and HDFC 4/4 (stale). Axis credit-card parser unverified on real data. Ingestion: gmail.py full OAuth2 gmail.readonly (token in keychain) + tools/gmail_auth.py; imap.py/outlook.py carry "EXPERIMENTAL / INCOMPLETE" docstrings as promised. Hygiene: testdata/* deny-by-default gitignore (only promo-emails/ and synthetic/ whitelisted); real statement dirs (hdfc/axis/aubank-statements), snapshots/ (2 credit-card JSON snapshots), credentials/ (1 real OAuth client JSON) exist locally, are untracked, and `git log --all` over those paths is empty (never committed); precommit hook + test_repo_hygiene.py enforce (the hook docstring references an earlier statement-leak near-miss as motivation). testdata/synthetic/ contains no committed files ? synthetic fixtures are generated at test time, contradicting TESTING.md §7 / onboarding-a-bank.md §3 which say they are committed.
74
75   SISTER REPO (C:/Users/avidu/Projects/gmail-attachments-to-gdrive): Apps Script attachment archiver pivoting to an ingest->classify->extract->redact->handoff pipeline. docs/VISION.md + ARCHITECTURE.md (codified 2026-06-03, PR #14, "binding auditability invariant" / ADR-008): that repo is THE ONLY code that touches Gmail; "Muneem and every downstream service operate only on minimized, non-PII signal and never hold the Gmail grant"; Muneem is framed as "our own isolated, hosted service"; handoff contract/schema is explicitly TBD ("define jointly with avidullu/muneem") and handoff is its Phase 3 (unbuilt); per-vertical "non-PII" definition for bank statements is an acknowledged open item. muneem's docs contain ZERO references to this repo, the invariant, or any handoff.
76
77   PRIOR-ART LESSONS (prior-art-email-mining.md): keep the pipeline shape (page text + per-image OCR -> unified text -> structured extraction), image dedup MD5+SSIM, the promo corpus (now testdata/promo-emails/, 10 PDFs + expected.yaml answer key); drop manual save-as-PDF, cloud-Gemini-by-default, one-shot stdout, no-tests/no-schema; PyMuPDF is AGPL and banned (pdfminer.six/pypdfium2 instead). TESTING.md grounds tiers T1?T4 in actual corpus inspection (ligatures, image-only codes, relative expiry, filename?issuer, third-party recipient PII noted honestly).
78
79   STRENGTHS:
80   * Operational docs are code-accurate: schema.md matches db.py/migrations.py DDL column-for-column including the v5 provenance migration (PR #46); ingesting-statements.md matches cli.py behavior and exact error strings; encryption-at-rest.md matches the implemented db subcommands (status/encrypt/rotate-key/backup/setup-recovery/recover) and states the threat model honestly ('does not defend against malware running as the logged-in user').
81   * Promises kept in code: Outlook/IMAP are explicitly marked '?? EXPERIMENTAL / INCOMPLETE' in module docstrings exactly as feedbackJune2026 P7/Phase A required (PR #41); the migration runner is hand-rolled (not Alembic) per the explicit push-back; provenance is a separate document_id-keyed table per the §8.7 decision, not per-row.
82   * Open-decision discipline largely holds: log format (no formatter in log.py), self-hosted runner (no real-statements.yml workflow), and macOS (CI matrix is ubuntu+windows only) are all verifiably still open in code ? nothing silently locked.
83   * Sensitive-data hygiene is defense-in-depth and verified: real statement dirs, snapshots, and the OAuth client are untracked and absent from all git history; .gitignore is deny-by-default on testdata/*; a pre-commit hook plus test_repo_hygiene.py (which is actually stronger than TESTING.md describes ? it forbids ALL non-corpus testdata and ALL non-corpus PDFs) enforce it as failing builds.
84   * Honest-status culture: ROADMAP openly flags its own checkbox rot and points to

CURRENT_STATE; TESTING.md marks 6 of 10 answer-key rows 'unverified' rather than inventing ground truth and discloses the third-party recipient PII in the corpus; research-open-banking-india.md labels every claim FACT / note-sourced / INFERENCE.
85 * feedbackJune2026.md is a model review-response artifact: it corrects the reviewer's stale premises with PR citations, pushes back with reasons (Alembic, fetch-before-storage, L3-before-eval), and catches the genuinely highest-stakes gap itself (key-loss makes the ledger AND all backups unrecoverable ? fixed as P0 in #43).
86 * The verification-first parser architecture ('extraction proposes, reconciliation disposes') is both well-documented and faithfully implemented: engine.py/profiles.py/validation.py match parser-architecture.md §§3?6, and the oracle-gating claim ('stores nothing rather than mislabeled rows') is real in the Axis May-2025 case.
87
88 ISSUES:
89 [high] Unreconciled head-on contradiction with sister repo gmail-attachments-to-gdrive over who touches Gmail
90 The sister repo's binding auditability invariant (docs/VISION.md + ARCHITECTURE.md ADR-008, codified 2026-06-03) states it is THE ONLY code that ever touches a user's Gmail and that 'Muneem ... never hold[s] the Gmail grant', receiving only 'minimized, non-PII signal'. muneem already violates this as a statement of fact: src/muneem/ingestion/gmail.py implements full gmail.readonly OAuth (token cached in keychain), tools/gmail_auth.py drives consent against a real OAuth client in testdata/credentials/, ROADMAP Track B marks OAuth+search+download done, and 'muneem fetch' is planned (ROADMAP B.4 / feedback Phase F). Neither repo references the other's constraint ? muneem's docs contain zero mentions of the sister repo. Two further axes: (a) the sister VISION frames Muneem as a HOSTED service, while muneem's DESIGN declares 'Multi-user / SaaS / hosted product' a non-goal and forbids cloud sync of raw data, period; (b) bank-statement PDFs are inherently PII (names, account numbers, transactions), so they categorically cannot flow through the sister's 'non-PII minimized signal' egress contract ? the sister repo itself lists 'per-vertical non-PII definition for bank statements' as unresolved, and the handoff contract/transport is TBD (its Phase 3, unbuilt). Reconciliation options: (1) muneem keeps its own ingest ? costs: breaks the sister invariant (needs explicit scoping, e.g. 'the invariant binds the public product, not the owner's personal instance'), duplicates Gmail sync/dedup logic, two OAuth surfaces; benefits: zero cloud processing of statements, full raw-PDF fidelity, passwords stay local, no Apps Script quota fragility. (2) muneem consumes the handoff ? costs: blocked on an undefined contract several phases away, the PII problem above, statement classification running in Google's cloud runtime, and password-protected PDFs cannot be unlocked in-account without putting password material into Apps Script (a real regression); benefits: single auditable Gmail surface, muneem drops google-api deps. (3) The unexplored middle: the sister archives raw attachments to Drive in-account (no egress, invariant-intact ? archiving is already its hardened core), and muneem ingests from the locally-synced Drive folder; the Gmail grant stays solely with the sister, unlock stays local, no handoff contract needed ? at the cost of a Drive-sync dependency and replacing historyId sync with folder watching. Whichever is chosen, today the sister VISION is stale as a description of reality, and muneem's Phase F is in unflagged conflict with a document calling itself binding.
91 [high] Two competing sequencing sources of truth with colliding namespaces (ROADMAP Milestones/Tracks vs feedbackJune2026 Phases A-H)
92 ROADMAP.md declares 'this doc owns sequencing', but the actually-current plan is feedbackJune2026 §4's Phases A-H ? a different ordering vocabulary where the letters collide ('Track A' = promo spine vs 'Phase A' = foundation/recovery; ROADMAP 'Milestone C/D' = HDFC slice / harden vs feedback 'Phase C/D' = instrument family / regression CI). ROADMAP's checkbox lists are self-admittedly rotten (B.1-B.3 unchecked though done), its §6 'Generalize' backlog overlaps Phases C/E/F/G without mapping, and its §10 v1 definition of done predates the whole-finance rescope. For a project whose docs are explicitly the memory across sessions, a restart cannot tell which plan governs without reading four docs in the right order and mentally diffing them.
93 [high] Phase C (instrument family) is the riskiest, most under-specified phase ? and is sequenced before the regression-CI safety net
94 Phase C is the next big build after B, but: (a) no broker/MF/CAS corpus is evidenced anywhere ? testdata holds only hdfc/axis/aubank statements, while the bank parsers' entire credibility came from '6 real statements + passwords' validation; (b) the instrument reconciliation oracle is undesigned ? the engine is FamilySpec-parameterized in code, but no doc states what arithmetic identity replaces the balance walk for holdings statements (units walk? units x NAV = value? CAS statements often lack a clean per-row identity, and dividend/fee classification has no oracle at all), which is exactly the load-bearing component of the whole 'verification-first' philosophy; (c) which artifact is the source (CDSL/NSDL CAS vs broker statement vs tradebook export) is neither chosen nor parked as an open decision; (d) Phase D

(regression CI Layer 2 + the D.3 eval writeup) comes AFTER C ? yet feedbackJune2026 itself argued 'characterize failure modes first, don't build blind' to justify D.3-before-L3, and the same logic applies to building a whole new family before the safety net exists; (e) cheaper unfinished work is skipped over: the Axis credit-card parser is still unverified on real data and HDFC CC bills (the other half of the first slice) are absent.
95 [high] The roadmap delivers no daily user value: no query/reporting surface in any phase, and Phase B's serving view silently went missing
96 Every DESIGN §2 use-case (price intelligence, portfolio timeline, bank/card comparison, item catalog) is the product's point, yet the only read surface after ~46 PRs is 'muneem txns list <bank>' (last 50 raw rows per single bank) and 'offers list'. Cross-issuer querying was explicitly promised as part of Phase B ('Phase B = the provenance table + a serving view') ? the provenance table shipped in #46 but no SQL view exists anywhere in src/ or tests/, and no tracker records the gap; feedback §5 marks decision 2 as settled with 'Phase B implements the provenance table', quietly dropping the view. Filters/analytics/export are deferred to Phase H, dead last. For a single-user tool, a weekly-usable 'show me last month across all accounts' query is what makes the user keep feeding it data through the long Phase C-G middle.
97 [medium] Whole categories missing from the roadmap: cash/manual entry, multi-year backfill & raw-PDF archive strategy, quarantine UX, restore runbook, refreshed definition of done
98 (1) Cash/manual data entry: migration v5 already reserves source_type='manual', but no phase plans an entry/correction path ? a personal ledger without cash spends can never reconcile with reality. (2) Multi-year archive strategy: nothing covers backfilling years of Gmail history (query design, quota, ordering), what happens to documents.path when staged files move, or the retention policy for raw locked PDFs in the staging dir (SQLCipher protects the DB; the PDF staging area is protected only by optional full-disk encryption ? undiscussed). (3) Quarantine UX: parser-architecture §11's 'what does the CLI do with a quarantined statement' is the user-facing handling of the real 1/7 Axis failure and is unowned by any phase. (4) feedbackJune2026 §2 prescribed a docs/recovery.md runbook ('keychain lost', 'new machine', 'verify a backup') and P6 a 'migrate then re-verify' runbook ? neither exists; encryption-at-rest.md covers fragments. (5) ROADMAP §10's v1 'definition of done' is still the old HDFC-slice wording and no phase has exit criteria, so 'done' for the whole-finance vision is undefined. (6) SMS-alert parsing was surfaced as a decided-worthy candidate (P11) but is assigned vaguely to 'Phase A (optional) / Phase F' with no owner.
99 [medium] CURRENT_STATE.md ? the designated live-status doc ? is wrong about the DB layer it points restarts at
100 Its §3 says 'Schema v4' and that 'a unified transactions table ... with per-row provenance/confidence is the planned direction once the design decision lands'. Reality: the decision landed (2026-06, DESIGN §8.7: flat per-issuer tables are permanent by design, provenance is a separate table, explicitly NOT per-row) and partially shipped (migration v5 provenance, PR #46). A restart reading CURRENT_STATE would re-litigate a settled decision in the opposite direction and miss that the schema is at v5. Everything else in the doc (CLI list, parser precision, ingestion status) verifies as accurate against code.
101 [medium] parser-architecture.md lags shipped reality the most among design docs: stale precision numbers and a superseded data-model section
102 Header and §10 step 3 say 'Axis 2/4' and §10a says 'HDFC 4/4 and AU 4/4 qualify today' ? reality is HDFC 6/6, AU 4/4, Axis 6/7 (June 2025 recovered via balance-delta reconstruction, which this doc never mentions). §9 'Data-model evolution' still points at 'a unified transactions table ... plus provenance + confidence columns' ? superseded by the §8.7 decision (flat per-issuer is permanent; provenance is a separate table); §11's 'storage unification timing' open question is answered but unannotated. Since this is the doc the L3/instrument-family work will be built against, the stale direction in §9 is the kind of drift that gets silently re-implemented.
103 [medium] Open-decision discipline lapse around §8.3: the L3 model was decided but never back-recorded, leaving DESIGN and ROADMAP contradicting parser-architecture
104 DESIGN §8.3 (LLM locality/model for sensitive categories) carries no resolution note, and ROADMAP §9 still says 'Ollama + Llama/Qwen/Mistral is the likely shape, but unconfirmed' ? while parser-architecture.md §4 states 'The research resolved the model: DeepSeek-OCR-3B (MIT)' with hardware specifics, and the auto-memory records it as decided. This is a partial resolution (the model is chosen, the locality rule is unchanged, the build is deferred), but the project's own decision-log discipline (ROADMAP §7: 'append resolutions to §1 and mark the corresponding DESIGN.md §8 item resolved') was followed for 8.1/8.2/8.4/8.6/8.7 and skipped here. A restart obeying 'do not pick silently' would either re-research the model or wrongly treat DeepSeek as unapproved.
105 [low] Phase A declared 'foundation first' but is quietly incomplete while Phases B/C proceed

Of Phase A's five items, two are unfinished: no dependency lockfile exists (uv.lock / requirements*.txt absent from the repo, despite P10 marking it 'Phase A' and 'reproducibility is real for a tool with a compiled dep (sqlcipher3)'), and structured-logging adoption (P1) is stalled on the open log-format decision ? log.py is a redaction filter with no formatter and adoption beyond cli/parsers is thin. Neither gap is tracked anywhere as remaining Phase-A work; the closing note of feedbackJune2026 implies Phase A is done modulo the three open decisions.

[low] Synthetic-fixture story: docs say committed PDFs in testdata/synthetic/, reality is runtime generation ? three docs and a gitignore comment overstate

TESTING.md §7 ('Stored under testdata/synthetic/ (committed; clearly fake)'), onboarding-a-bank.md step 3 ('Commit it to testdata/synthetic/'), and the .gitignore comment ('committed synthetic fixtures are tracked') all describe committed fixture PDFs. In reality git tracks nothing under testdata/synthetic/ ? tests/test_regression_synth.py generates born-digital PDFs into tmp_path at test time via tests/synthpdf.py (which is arguably the better design, per ROADMAP's Layer-1 description 'generates ... at test time'). A new-bank onboarding session following onboarding-a-bank.md verbatim would try to commit PDFs into a directory pattern the hygiene tooling treats as a whitelisted-but-empty path. Minor related staleness: TESTING.md §9 describes the hygiene guard as checking testdata/statements/ and testdata/private/, while test_repo_hygiene.py actually enforces the stronger 'nothing under testdata/ except promo-emails/ and synthetic/' rule ? the docs understate the real guard. Also TESTING.md's golden-file layer (testdata/promo-emails/golden/) does not exist.

[low] README understates project status and asserts untested macOS support while macOS first-class is an explicitly open decision

README 'Status' says the skeleton/CLI/offline-CI 'are in place' ? three months of shipped substance (verified real-statement parsers, hybrid unlock, SQLCipher encryption + recovery, migrations, provenance) is invisible, which matters because README is the entry doc. It also states 'Runs on Linux, macOS, and Windows' while feedbackJune2026 §5 decision 6 leaves 'macOS first-class?' open and the CI matrix tests only ubuntu+windows ? the claim is plausible (keyring/platformdirs) but unverified, against the project's own attribution discipline. Cosmetic but same genre: cli.py's module docstring still says 'Pipeline subcommands (fetch, txns) land with their milestones' though txns has landed.

RECOMMENDATIONS:

[short-term] Write the cross-repo ADR: decide muneem-fetch vs sister-handoff vs Drive-folder middle path, and scope the auditability invariant: One page, referenced from both repos. Decide explicitly: (a) does the sister repo's binding invariant apply to Avi's personal muneem instance or only to the future public product? (b) does muneem keep direct Gmail ingest (then amend the sister VISION's 'only code that touches Gmail' claim or scope it), consume the future handoff (then resolve that statement PDFs are PII and cannot ride a 'non-PII' contract ? the per-vertical carve-out must be designed jointly), or adopt the middle path (sister archives attachments to Drive in-account, muneem ingests the locally-synced folder; Gmail grant stays single, unlock stays local, no handoff contract needed)? This blocks Phase F and is currently a silent contradiction between two 'binding' documents written in the same week.

[short-term] Collapse sequencing into one owner: fold feedbackJune2026 Phases A-H into ROADMAP.md (or formally demote ROADMAP's milestone sections to history): Rename to avoid the Track-A/Phase-A and Milestone-C/Phase-C letter collisions, mark each phase's real status (A: incomplete ? lockfile + structured logging pending; B: half-shipped ? serving view missing), give each phase an exit criterion, and refresh §10's definition of done for the whole-finance scope. Simultaneously fix the two stale status docs: CURRENT_STATE §3 (schema v5, provenance shipped, per-row direction superseded) and parser-architecture (Axis 6/7, HDFC 6/6, balance-delta recovery; annotate §9/§11 with the §8.7 resolution). These are hours of work that de-risk every future restart.

[short-term] Gate Phase C behind an instrument-family entry checklist, and consider swapping it with Phase D: Before building: (1) corpus in hand ? N real broker/MF/CAS statements with passwords, in a gitignored testdata/<issuer>-statements/ dir, same as the bank slice demanded; (2) a one-page oracle design ? what arithmetic identity verifies a holdings statement (units continuity, units x NAV = value, fee/dividend cross-checks) and what 'reconciled' means when there is no balance walk; (3) the artifact decision (CAS vs broker statement vs tradebook) parked as an explicit open decision. Apply the project's own D.3-before-L3 logic: land regression CI Layer 2 + the D.3 eval writeup BEFORE the biggest expansion of parser surface, and verify the Axis CC parser on real data first ? it is the cheapest unverified thing already written.

[short-term] Ship the missing Phase-B serving view plus a minimal daily query surface, ahead of schedule: The serving view was decided and promised alongside the provenance table ? implement the cross-issuer SQL view (normalizing the four flat tables into date/amount/direction/narration/issuer + provenance join) as migration v6, then a 'muneem txns

all --since/--month' and a one-screen monthly summary. This is days of work on settled decisions, makes the tool worth opening weekly during the long C-G middle, and exercises the provenance join for real. Track 'muneem verify' (re-run reconciliation over stored rows) as the next cheap ops win rather than leaving it in Phase H.
117 [long-term] Add the missing roadmap categories: cash/manual entry, backfill & archive strategy, quarantine UX, recovery runbook: (1) Manual entry/correction path using the already-reserved source_type='manual' (cash spends, fixing a mis-parsed row) ? without it the ledger can never be complete. (2) A multi-year backfill plan: Gmail history query design and ordering, documents.path lifecycle when staged files move, and an explicit retention/protection policy for raw locked PDFs in the staging dir (currently outside SQLCipher's protection, covered only by optional FDE). (3) Decide the quarantine UX (parser-architecture §11): what 'muneem txns'/'eval' shows for the 1-in-7 statement that stores nothing. (4) Write docs/recovery.md (keychain lost / new machine / verify a backup / migrate-then-reverify) as feedbackJune2026 §2 and P6 prescribed.
118 [long-term] Close the small decision-hygiene loops: Back-record the L3 model choice as a partial resolution in DESIGN §8.3 and ROADMAP §1's decision log (model: DeepSeek-OCR-3B decided; locality rule unchanged; build deferred). Resolve the three cheap open decisions when next forced: log format (logfmt vs JSON ? unblocks finishing Phase A's P1), self-hosted runner acceptance (unblocks regression CI Layer 2), macOS stance (then fix or keep README's claim). Commit the lockfile (P10, trivially overdue for a compiled-dep project). Align the synthetic-fixture wording in TESTING.md §7 / onboarding-a-bank.md / .gitignore comment with the actual (better) runtime-generation design, and update TESTING.md §9 to describe the stronger guard that actually exists.
119
120 ########## gmail-docs ##########
121 SUMMARY: gmail-attachments-to-gdrive has unusually disciplined docs for a side project ? a binding auditability invariant with a CI-enforced egress guard, an honest quota-constraints section, and a PRIVACY.md that matches the shipped code. But the vision layer is ahead of (and in one place contradicted by) reality: muneem already holds its own Gmail OAuth grant with an implemented fetch track, directly violating this repo's "sole Gmail surface" invariant; the "only non-PII signal leaves" rule is irreconcilable with a meaningful bank-statement handoff (and redaction of password-locked Indian bank PDFs is infeasible in Apps Script); the "versioned handoff contract" the whole decoupling rests on is defined in no artifact in either repo; and ROADMAP.md is a rear-view mirror that tracks none of the Phase 0/1 work or the grocery-receipts vertical the vision names first. Phases 0?1 are realistic on GAS; Phase 2 OCR and the EmailMessage durable store are under-specified; Phase 3 as written breaks either the privacy rule or the invariant. The 100-user Testing-mode posture is financially coherent but undercut by 7-day refresh-token expiry (background sync dies weekly for testers) and a hard wall at user 101 (CASA, both gmail.readonly and full drive being restricted scopes) ? with self-deployment, the distribution path most aligned with the invariant, existing only as an unchecked roadmap stub.
122
123 MAP: VISION (docs/VISION.md): north star = "turn a person's email into a high-fidelity, structured, queryable record of their life." This repo is the extraction/ingest front door only: watches Gmail in-account, archives attachments, classifies, extracts, redacts/minimizes, hands off. Domain parsing/storage/analytics live downstream: muneem (finance), future siblings (grocery/coupons/travel). First verticals: (1) grocery receipts, (2) bank statements ? muneem. PHASES: 0 ingest model (stable EmailMessage; copier consumes it) ? 1 GAS-only classify+extract+index (no egress; Sheets "LifeIndex") ? 2 coverage/re-extraction (more extractors, body/OCR, extraction cursors, review queues) ? 3 handoff to hosted services (redact/minimize, separate consent, egress budgets) ? 4 surfaces/ops. AUDITABILITY INVARIANT (ARCHITECTURE.md, "binding", ADR-008): this repo is the ONLY code that touches Gmail; muneem never holds the Gmail grant ("Muneem may orchestrate this app; it is never itself the Gmail reader"); egress only via in-repo REDACT/MINIMIZE, non-PII only, to "the user's own hosted service", never third parties; self-deployable/reproducible from source; enforced by tests/auditability.test.ts (regex ban on UrlFetchApp in src/*.ts) + a PR checklist. PRIVACY (PRIVACY.md, 2026-05-31): app runs entirely in user's account (executeAs: USER_ACCESSING, verified in appsscript.json), copies attachments Gmail?user's own Drive, no third-party transmission, "the author of the app has no access to your data", only bookkeeping state in PropertiesService. CURRENT IMPLEMENTED SURFACE (src/, verified): attachment copier only ? doGet setup form, syncUserDrives trigger, 30-day window backfill + flag-gated history.list incremental sync, per-user lock, processed-id dedup, structured Logger.log to Stackdriver (exceptionLogging: STACKDRIVER), opt-in MailApp daily summary (off by default). NO ingest model, classifier, extractor registry, Sheets store, redactor, or handoff code exists (zero grep hits for EmailMessage/classify/extractor/HandoffPayload). Scopes: gmail.readonly, full drive, script.scriptapp; webapp access ANYONE. ROADMAP.md: platform constraints documented (6 min/run, 90 min/day triggers, 20 triggers, ~20k Gmail reads/day); all "quick win"/medium/large items

checked done except: Shared Drive/structure options, SECURITY.md, "deploy your own" guide, reproducible build, CONTRIBUTING.md. No items for Phase 0/1, the handoff contract, or grocery receipts. PUBLISHING (README): deliberately in OAuth "Testing" no-verification zone ? ?100 manually-added test users, unverified-app warning, ~weekly re-consent (7-day refresh-token expiry); or Workspace Internal (org-only). Fully public requires brand + restricted-scope + domain verification + annual CASA (~$1?5k+, 6?16 weeks); README recommends narrowing drive?drive.file first; gmail.readonly stays restricted regardless. CONTRADICTION WITH MUNEEM (verified in muneem/DESIGN.md + docs/ROADMAP.md): muneem Track B "Gmail OAuth2 + search + attachment download ?" with gmail.readonly, `muneem fetch` CLI (B.4) pending; muneem's docs contain zero references to gmail-attachments-to-gdrive or any handoff contract; muneem is local-first (SQLCipher on the user's machine), not a hosted service.
124
125   STRENGTHS:
126   * The auditability invariant is codified as a binding constraint with a per-PR checklist AND a CI test (tests/auditability.test.ts) ? a privacy claim with actual enforcement teeth, which is rare.
127   * ROADMAP.md opens with a cited, accurate platform-constraints section (6 min/run, 90 min/day, 20 triggers, ~20k Gmail reads/day, historyId expiry semantics) ? the design is genuinely quota-aware, not aspirational.
128   * PRIVACY.md precisely matches what the code does today (verified: no UrlFetchApp anywhere in src/, executeAs USER_ACCESSING in the manifest, state only in PropertiesService).
129   * Clean phase gating: Phases 0?2 are explicitly in-account with no egress and no privacy-policy change; Phase 3 is flag-gated and separately consented ? correct sequencing of consent with capability.
130   * ARCHITECTURE.md honestly flags its own weakest point as an open tension ("bank-statement and receipt signal? is often itself sensitive? 'non-PII only' needs a concrete per-vertical definition") rather than hand-waving it.
131   * The pure-logic/GAS-shell separation with an 85% coverage gate, trigger-leak guards, and 'mark done only on clean completion' is a solid hardened core to build the pivot on ? ADR-007's claim is borne out by the source layout.
132   * README's publishing section is unusually clear-eyed about the CASA wall, the Testing-mode caveats, and the drive?drive.file narrowing ? the cost/process reality is documented, not discovered later.
133

22. user (2026-06-10T16:33:51.896Z)

134   ISSUES:
135   [critical] Muneem already violates the binding 'sole Gmail surface' invariant
136   ARCHITECTURE.md invariant #1 states no other component ? 'Muneem included ? ever holds the Gmail OAuth grant or sees raw mail', and VISION.md says 'downstream services never read Gmail'. But muneem has its own Gmail OAuth ingestion IMPLEMENTED (Track B: OAuth2 + search + attachment download done, `muneem fetch` B.4 pending) using gmail.readonly, and muneem's DESIGN/ROADMAP contain zero references to this repo or any handoff. The binding invariant is contradicted by shipped sibling code on day one, and neither repo records the conflict or a resolution path. 'Muneem may orchestrate this app' also has no defined mechanism (how does a local Python CLI orchestrate a GAS web app?).
137   [critical] 'Only non-PII signal leaves' cannot coexist with a meaningful bank-statement handoff to muneem
138   Muneem's entire pipeline requires the FULL raw statement: it unlocks password-protected Indian bank PDFs locally (name+DOB / PAN+DDMM conventions), parses every transaction row, and reconciles against the bank's summary. A redacted, 'non-PII' payload is useless to it ? merchant names, amounts, dates, and account identifiers ARE the signal and ARE sensitive/PII. Redaction in GAS is also infeasible for this vertical: Apps Script has no native libs, no PDF-decryption capability, and Drive's OCR-via-Doc-conversion cannot open password-locked PDFs at all, so the REDACT/MINIMIZE stage cannot even read the document it is supposed to scrub. ARCHITECTURE.md flags this as an 'open tension', but the phase plan builds Phase 3 on the unresolved premise, and VISION.md states the rule absolutely ('Only non-PII signal ever leaves'). The honest options are: raw-blob handoff under separate consent (breaking the rule), muneem keeping its own Gmail path (breaking the invariant ? the current de-facto state), or finance staying fully out of this pipeline. Also, VISION/ARCHITECTURE describe egress to 'our own isolated, hosted service', but muneem is a local-first desktop tool (SQLCipher on the user's machine), not hosted ? the destination category itself is mischaracterized.
139   [high] The 'versioned handoff contract' ? the seam the whole architecture rests on ? is defined in no artifact

VISION.md says the repos 'meet only at a versioned handoff contract'; ARCHITECTURE.md sketches a 3-field HandoffPayload shape and then says schema and transport are 'TBD and aligned with that repo' (UrlFetch to a signed endpoint vs write-to-shared-store undecided). No schema file, no contract doc, no version, no transport decision exists in either repo, and muneem doesn't know the contract is supposed to exist. Until this is a concrete artifact, the extraction?processing decoupling is prose, not architecture ? and Phase 3 cannot be designed, costed, or consent-scoped.

[high] Phase 2 (OCR/body handling) and the durable EmailMessage substrate are under-specified for GAS quotas

Phase 0's premise is 'extraction re-runs over this record without re-touching Gmail' ? but where the durable EmailMessage (with plainBody/htmlBody) LIVES is never stated. PropertiesService is explicitly excluded ('never the entity index') and is anyway capped (~9KB/value, ~500KB total); a Sheets row caps at 50k chars/cell and a spreadsheet at 10M cells, making body storage for tens of thousands of messages awkward and slow via SpreadsheetApp; Drive-hosted JSON shards are unmentioned. For Phase 2 OCR, the only named mechanism is 'native Drive OCR via Doc conversion' ? workable for images/simple PDFs but mediocre on statements, generates intermediate Docs needing cleanup, and is useless on password-locked PDFs. No quota math exists for a multi-year backfill under 6 min/run, 90 min/day triggers, and ~20k Gmail reads/day (a 50k-message mailbox at even 2s/message of OCR+write is weeks of trigger budget). 'Heavy ML/PDF work happens downstream' is the right instinct, but Phase 2's 'body/OCR handling' sits in the in-account tier without a feasibility analysis. Phases 0?1 (rich EmailMessage via the Gmail advanced service, regex/sender classification, .ics/.csv extraction, Sheets index) are realistic on GAS.

[high] Auditability is weaker than stated for centrally-deployed users; Cloud Logging is already gray-zone egress

Three gaps: (1) End users of the central web-app deployment cannot view the deployed source, and the owner can push new code that all users execute on their next trigger run with no re-consent (scopes unchanged) ? so 'auditable at any time' truly holds only for self-deployers, and the reproducible-build + deploy-your-own items that would make verification possible are still unchecked. (2) Execution logs and exceptions go to Stackdriver/Cloud Logging in the DEVELOPER's GCP project for a centrally deployed script: src/drive.ts logs message IDs and exception text ('Attachment copy failed (message ?): ' + e, which can embed attachment names), and ARCHITECTURE.md plans 'structured logs carrying messageId'. That is data derived from a user's mail leaving the user's control ? in tension with invariant #2 ('Nothing derived from a user's mail leaves their Google account') and with PRIVACY.md's absolute 'The author of the app has no access to your data'. (3) The egress guard is a single regex for the literal 'UrlFetchApp' over src/*.ts only ? it doesn't scan html/index.html (client-side fetch from the served page), dist/ output, MailApp (SUMMARY_EMAIL_RECIPIENT is config-settable to any address), or trivially obfuscated access. Fine as a tripwire today; not the 'enforceable form' the docs imply.

[medium] Testing-mode 7-day refresh-token expiry kills the always-on value prop for the very users the plan wants to learn from

The posture is 'stay under 100 users to harden the system and learn the value proposition'. But for External/Testing apps, refresh tokens expire after 7 days ? each test user's background sync silently dies weekly until they revisit and re-consent. The README lists this as a one-line caveat, but it is fatal to evaluating a set-and-forget background archiver with real users: what testers experience is a product that stops working every week. The plan has no mitigation (e.g., expiry detection + nudge email, or steering testers to self-deploy where they own the OAuth client). Conclusions drawn from Testing-mode retention will be noise.

[medium] User 101 is a hard wall and the bridge plan (self-deploy distribution) exists only as an unchecked stub

At user 101: the Testing-mode test-user list is full; the only sanctioned path is External+Production = brand verification + restricted-scope verification + domain verification + annual CASA (~$1?5k+/yr, 6?16 weeks) ? and BOTH requested scopes are restricted (gmail.readonly, and full drive, which README already concedes should narrow to drive.file). 'Decide later with data' is financially coherent (ADR-005), but the plan is silent on the bridge: the one distribution channel that scales past 100 with zero verification AND maximally honors the auditability invariant is 'every power user deploys their own instance from source' ? exactly the unchecked 'Deploy your own from source' + 'reproducible build' roadmap items. The posture and the invariant point at the same answer; the roadmap doesn't prioritize it.

[medium] ROADMAP.md does not track the pivot: no Phase 0/1 items, no handoff contract, no grocery-receipts vertical

The roadmap is almost entirely checked-done copier hardening plus a thin tail (Shared Drives, SECURITY.md, self-deploy guide, reproducible build, CONTRIBUTING.md). Nothing tracks the EmailMessage ingest model (Phase 0), the classifier/extractor registry or the Sheets

LifeIndex store (Phase 1), the per-vertical non-PII definition, the handoff contract, or grocery receipts ? the vertical VISION.md names FIRST and calls the cheap proof of the whole pipeline. The vision and the roadmap describe two different projects right now; ARCHITECTURE.md's 'Open items' section is the only forward-looking work list and it isn't mirrored into the tracked roadmap.
151 [low] Decision-log hygiene: undated, status-less ADRs with unrecorded alternatives
152 The eight ADRs are one-line table rows: no dates, no status (proposed/accepted/superseded), no alternatives-considered, no links to discussion. ADR-008 is literally labeled 'Option A', implying Options B/C were weighed but they are recorded nowhere ? the exact context a future maintainer (or the CASA assessor) will need. Minor today; compounds as the pivot generates real decisions (handoff transport, store choice, non-PII matrix). Also cosmetic but real: appsscript.json timeZone is America/Los_Angeles while the user is in Bangalore and Gmail before:/after: boundaries follow it (README tells deployers to fix it, but the committed default is wrong for the owner's own deployment).
153
154 RECOMMENDATIONS:
155 [short-term] Reconcile Gmail ownership with muneem NOW ? one ADR recorded in both repos: Decide explicitly: (a) muneem keeps its implemented Gmail track and this repo's invariant is rescoped (e.g. 'sole Gmail surface for the multi-user product; the owner's local tools are exempt'), or (b) muneem's Track B is frozen/deprecated and this repo becomes its ingest front door (which forces the raw-blob handoff question immediately). Today the binding invariant is contradicted by shipped code in the sibling repo and neither repo acknowledges it. Whichever way it lands, record it as a dated ADR in ARCHITECTURE.md here and in muneem's DESIGN.md decision log.
156 [short-term] Write the handoff contract v0 as a real artifact and pick a transport: Promote the HandoffPayload sketch into a versioned schema file (JSON Schema or a markdown contract doc) plus a transport decision. A pragmatic v0 consistent with both repos: a Drive 'handoff' folder + manifest JSON (write-to-shared-store) that muneem's local CLI polls ? no hosted service, no UrlFetch, no change to the egress guard, works with muneem being local-first. Explicitly state, per vertical, whether the payload is structured fields or the raw blob, because that choice IS the privacy decision.
157 [short-term] Replace 'non-PII only' with a per-vertical data-minimization matrix: Bank statements are PII through and through; the absolute rule in VISION.md is unsatisfiable for the finance vertical. Reframe the privacy north star as minimization + explicit consent: a per-vertical table of sent / tokenized / kept-in-account (already an ARCHITECTURE.md open item ? give it a roadmap entry). For finance, the honest options are raw-blob-under-separate-consent or finance-stays-local; say which. Update VISION.md's 'Privacy north star' wording to match.
158 [short-term] Add the Phase 0/1 work to ROADMAP.md, grocery receipts first: Mirror ARCHITECTURE.md's open items into tracked roadmap entries: (1) EmailMessage ingest model consumed by the copier (Phase 0, no behavior change); (2) the narrow 'receipt-smelling' classifier + golden-email fixtures; (3) a GoogleSheetsStore spike that answers, with numbers, where the durable EmailMessage substrate lives (Sheets cell/spreadsheet limits, SpreadsheetApp write cost per run, Drive-JSON alternative) before Phase 0 code locks it in; (4) the handoff contract doc. The grocery vertical VISION names first currently has zero tracked work.
159 [short-term] Close the auditability gaps that are cheap today: (1) Ship SECURITY.md and the deploy-your-own guide (already roadmap stubs ? they are the enforcement of the invariant, not nice-to-haves). (2) Strengthen tests/auditability.test.ts: scan dist/Code.js and html/ too, assert the manifest's oauthScopes against an allowlist, and flag MailApp/ContentService alongside UrlFetchApp. (3) Scrub or gate log payloads (messageId, exception text) OR amend PRIVACY.md's 'author has no access to your data' to disclose Cloud Logging visibility for the central deployment ? self-deployers escape this, which is one more argument for that path.
160 [long-term] Make self-deployment the primary public distribution channel: The 100-user Testing cap, the weekly re-consent decay, the CASA wall at user 101, and the auditability invariant all point to the same strategy: the product's public form is 'clone, build, deploy your own' (each user owns their OAuth client ? no verification, no token expiry, no developer-visible logs, perfect invariant compliance), with the centrally hosted deployment reserved for friends/family until CASA is consciously bought. Decide this as posture, not just a runbook section; it changes what 'launch' means and what Phase 4 ops work matters.
161 [long-term] Do the Phase 2 feasibility math before committing OCR to the in-account tier: Before Phase 2 scope is fixed: prototype Drive OCR-via-Doc-conversion on a real receipt corpus and measure per-doc latency and quality; compute backfill wall-time under 90 min/day; design intermediate-Doc cleanup; and explicitly classify password-locked PDFs as downstream-only (GAS cannot open them ? no decryption primitive exists). If the numbers say in-account OCR only works for incremental new mail, write that into ARCHITECTURE.md so Phase 2 promises match the platform.

162   [long-term] Adopt lightweight ADR hygiene before the pivot generates real decisions: Add date, status, and a one-line 'alternatives considered' to each decision-log row (start by recording what ADR-008's Options B/C were). Cross-link decisions that bind both repos (Gmail ownership, handoff contract) so muneem's docs and this repo's docs can't silently diverge again. This is also the paper trail a future CASA assessment will ask for.
163

23. assistant (2026-06-10T16:33:58.791Z)

Now the market research / PMF section:

24. user (2026-06-10T16:34:05.223Z)

SUMMARY:
The direction is sound as a personal system and as a niche open-source play, but not (yet) as a mainstream consumer product. The Indian PFM market has consolidated around RBI's Account Aggregator rails (Fold, Jupiter, INDmoney), which are cloud-first, lending-subsidized, and still have real coverage gaps (credit-card detail, bonds, joint accounts, historical depth) ? exactly the gaps muneem's reconciliation-gated local PDF parsing fills. The Gmail-attachment-to-Drive layer is fully commoditized (Digital Inspiration's add-on alone has 2M+ installs and already decrypts password-protected bank PDFs), so Repo A's value must come from the classification/extraction front-door vision, not archiving. Email-parsing consumer products historically survived only by (a) selling aggregated purchase data (Slice/Rakuten?NielsenIQ), (b) pivoting to bank-linking (Truebill?Rocket Money), or (c) owning a narrow high-pain vertical (TripIt for travel) ? a cautionary base rate for the 'extraction front door' as a business. Publishing a Gmail-reading app in 2026 costs roughly $540?$4,500/year in CASA assessments plus ~2 months of process, annually recurring; the self-deploy 'run the code you read' model legitimately sidesteps this via Google's personal-use exemption and is itself the cheapest PMF probe: count completed self-deployments. Recommended sequencing: finance remains the first vertical for personal utility (already proven on real HDFC/Axis/AU data); grocery is a good second/demo vertical but a weak public wedge; the India tax-season document pack is the most under-considered alternative wedge.

=== MARKET MAP ===
* [India AA-based PFM] Fold Money: Bengaluru startup (founded 2019, ~11 employees, $2.5M seed, ?4.99Cr revenue FY25 per Tracxn) that pulls bank/CC/investment/EPF/NPS data via the RBI Account Aggregator framework and focuses on cash-flow awareness; categorization engine built with Setu (AA TSP). | RELEVANCE: The closest spiritual competitor to muneem's consolidated-ledger vision, but cloud-based and AA-dependent. Its tiny revenue after 6 years signals how hard it is to get Indians to pay for pure PFM. Its AA dependence means it inherits AA's gaps ? which is muneem's opening.
* [India PFM superapp] INDmoney: Investment-led superapp tracking stocks/MF/US equities plus free credit-card bill and savings-account tracking with auto-categorized expenses. | RELEVANCE: Shows the dominant Indian PFM monetization pattern: tracking is a free funnel into brokerage/distribution revenue, not a paid product. Competing head-on as a paid tracker is structurally hard.
* [India neobank with AA tracking] Jupiter (Spend Insights): Neobank whose Spend Insights links external bank accounts and credit cards via AA and auto-categorizes spends, dues, and statements. | RELEVANCE: AA-based aggregation is now table-stakes in Indian consumer fintech; differentiation cannot be 'see all accounts in one place' ? it must be depth (item-level, historical, verified) or privacy posture.
* [SMS-parsing expense tracker ? lender] Axio (ex-Walnut/Capital Float): Walnut pioneered SMS-parsing expense tracking in India; merged into Capital Float, rebranded Axio (NBFC), and was acquired by Amazon for ~$200M in September 2025 with RBI approval, primarily for its lending license/BNPL stack. | RELEVANCE: The canonical Indian story: expense tracking was never the business ? it was underwriting-data acquisition for credit. Also the direct precedent for muneem's SMS-freshness idea: technically proven, but the surrounding Play Store policy regime has since hardened.
* [India email-statement parser (incumbent)] CRED Protect: CRED's feature that, with consent, reads users' Gmail for credit-card statement emails, parses password-protected PDF statements (using stored user data to unlock them), and surfaces dues/spend analysis. Free; monetized via cards ecosystem. | RELEVANCE: Proof that email-based statement parsing works at scale in India AND that a deep-pocketed incumbent gives it away free. muneem cannot win on convenience; it can win on locality (CRED processes in its cloud), full-ledger scope beyond cards, and user-owned data.

* [Infrastructure (RBI rails)] Sahamati / AA ecosystem: 180+ FIPs and 950+ FIUs live; AA-facilitated lending at ?1.47 lakh crore in H1 FY26. PFM is a recognized consent use case. No individual/self-custody access path: data flows only to regulated FIUs via NBFC-AAs; joint accounts excluded; FIP-AA connectivity is bilateral and fragmented; investment data tilts toward MF/depository holdings; credit-card FIP coverage remains the weakest data class. DPDP Rules 2025 create 'Consent Managers' modeled on AA but do not open raw data access to individuals. | RELEVANCE: Confirms (as of mid-2026) that muneem's AA-deferral decision still holds: nothing changed in 2025-26 to give individuals self-custody pulls. The coverage gaps (CC detail, bonds, history) define exactly where PDF parsing remains the only complete source.
* [Gmail?Drive add-on (Repo A's layer)] Digital Inspiration 'Save Emails and Attachments': Workspace Marketplace add-on with 2M+ installs; saves emails/attachments to Drive in native formats, free tier + premium, and notably already decrypts password-protected bank/CC PDFs. | RELEVANCE: The archiving layer is commoditized by a single dominant indie (Amit Agarwal) plus cloudHQ. Repo A cannot compete on 'save attachments'; its only defensible angle is the auditable self-deploy trust model and the downstream extraction/index vision.
* [Gmail?Drive/Sheets add-ons] cloudHQ (Save Emails / Export Emails to Sheets): Suite of Gmail export add-ons with freemium tiers (Free/Premium Backup/Starter/Plus), converting threads to PDF and attachments to Drive, plus email-to-spreadsheet extraction. | RELEVANCE: Second proof the layer is commoditized ? and that 'email ? structured rows in a sheet' is already a productized category, though not finance-specialized for India.
* [Email-parsing survivor (travel)] TripIt (SAP Concur): Parses forwarded/inbox-synced booking confirmation emails into master itineraries since 2008-2010; still alive and the category default. | RELEVANCE: The one durable consumer email-parsing success. Lesson: it won a narrow, high-pain, time-critical vertical where consolidation has obvious immediate utility ? a template for choosing wedges.
* [Email-receipt parsing (data-resale model)] Slice Intelligence / Rakuten Intelligence ? NielsenIQ: Parsed purchase receipts from users' inboxes (fed by Unroll.me, acquired 2014); the consumer app was a loss-leader for an e-receipt panel-data business sold to NielsenIQ in 2022. | RELEVANCE: Email receipt extraction at scale monetized by selling aggregated purchase data ? the exact model muneem's privacy thesis rejects. Warns that grocery/receipt data has B2B resale value but weak direct consumer willingness-to-pay.
* [Subscription-finder PFM (US)] Rocket Money (ex-Truebill): Started around inbox/statement scanning for forgotten subscriptions; pivoted fully to Plaid bank-linking; acquired by Rocket Companies; freemium with premium bill negotiation. | RELEVANCE: Shows (a) subscriptions/recurring-charge detection is the email-adjacent wedge consumers demonstrably pay for, and (b) successful players abandoned email parsing for bank-feed parsing once available ? AA is the Indian analogue of that gravity.
* [Platform-native purchase tracking] Gmail Purchases view (Google native): Google has long auto-extracted receipts into a Purchases page (myaccount.google.com/purchases, data back to 2012) and in 2025 shipped a refreshed 'Purchases' view in Gmail consolidating orders/shipping/delivery. | RELEVANCE: The platform owner already does generic receipt extraction for free, natively. Any public front-door product must do something Gmail won't: India-specific finance depth, user-owned structured export, local processing, cross-source ledger.
* [SMB email-receipt extraction] SparkReceipt / WellyBox / Zoho Expense Gmail add-on: Commercial tools that connect Gmail/IMAP, auto-detect receipt emails, and extract vendor/amount/tax line items via AI, targeting freelancers/SMB bookkeeping. | RELEVANCE: Receipt extraction from email is a solved, crowded SMB category globally. The unowned space is consumer-side, India-format-specific, privacy-first extraction ? narrower than it looks.
* [OSS local-first PFM] Actual Budget / Firefly III: Self-hosted budgeting/finance apps; ingestion is generic CSV/bank-sync (EU/US oriented); Firefly relies on a data importer plus user-contributed CSV import configs; no Indian PDF statement ingestion exists in either. | RELEVANCE: The OSS PFM world has UI/ledger/budgeting solved but has NO Indian ingestion story. muneem could be the missing ingestion layer (export to Actual/Firefly/beancount) rather than rebuilding serving ? a cheap distribution wedge into existing communities.
* [OSS India importers] beancount-importers-india (plain-text accounting): Community importers for ICICI/SBI/Zerodha CSV/XLS tradebooks into Beancount; HDFC absent; CSV-based, single-maintainer, no PDF parsing, no validation guarantees. | RELEVANCE: Direct evidence the Indian plain-text-accounting niche exists but is underserved: importers are CSV-only, stale, and trust-free. Reconciliation-gated PDF parsing is a genuine step up for this exact audience.
* [OSS hobby parsers] Indie Indian statement parsers (xaneem/hdfc-cc-statement-parser, joeirimpan/hdfc-cc-parser-rs, codito/bout, StmtForge, vishwaraja/hdfc-pdf-converter): Scattered single-bank or multi-bank PDF?CSV/QIF parsers for HDFC/ICICI/SBI/Axis statements; mostly unmaintained one-offs; StmtForge claims 9 banks, offline, Streamlit dashboard. | RELEVANCE: Validates demand among Indian devs (people keep rebuilding this) and validates the gap: none

offer arithmetic reconciliation gating, encrypted ledgers, or maintained multi-issuer coverage. Also: muneem has competitors for mindshare in 'parse my HDFC PDF' GitHub searches.
* [India receipt-scanning rewards] magicpin / Club Corra: magicpin (since 2015) pays points for uploaded bills including grocery; Club Corra runs OCR-verified receipt uploads for 40+ partner brands redeemable as UPI cashback. | RELEVANCE: The only Indian consumer receipt-collection businesses pay users for receipts (data/marketing value to brands) ? strong signal that Indian consumers won't proactively consolidate grocery receipts without being paid, undermining grocery as a public-product wedge.
* [B2B statement parsing] Precisa / bank-statement OCR vendors (B2B India): Commercial bank-statement analysis tools (Precisa, et al.) parsing Indian bank PDFs for lenders/CAs with fraud detection, volatility scores, AA integration. | RELEVANCE: Statement parsing IS a paid market in India ? but B2B (lending ops, CAs), not consumer. If muneem ever monetizes parsing itself, this is the adjacent proven buyer, and also the entrenched competition.
* [Grocery data sources] Blinkit/Zepto/Instamart invoicing: Blinkit issues downloadable GST tax invoices per order (in-app, My Orders); launched GSTIN invoicing for businesses in Oct 2024; government uses q-commerce price data for GST-cut monitoring. Itemized invoices exist but live primarily in-app rather than reliably as email attachments. | RELEVANCE: Grocery item-level data exists and is genuinely absent from AA/PFM apps, but the front door for it is only partially email ? an Apps Script-only ingestion strategy under-covers the vertical (inference from invoice-retrieval flows).

=== GROCERY WEDGE ===
PROS ? Personal utility: item-level basket data is absent from every AA-based PFM (AA shows the ?487 Blinkit debit, never the basket ? inference, but structurally certain since FIP data schemas don't carry merchant line items); for a Bangalore household concentrated on Blinkit/Zepto/BigBasket/Instamart it would unlock price-over-time, shrinkflation, and category analytics nobody else offers. Development-process fit is excellent: grocery receipts are low-sensitivity (no account numbers), so they can be a cloud-LLM-permitted corpus under muneem's own security rules, making parser iteration fast and safely testable ? same property that made the promo-email corpus work. PROS ? Public product: genuinely unowned space in India; no consumer product consolidates itemized grocery receipts across q-commerce platforms; demo appeal ('see your real grocery inflation') is high and shareable. CONS ? Personal: source coverage is the killer ? Blinkit's itemized GST invoice is primarily an in-app download from My Orders, not a guaranteed email attachment, and Zepto/Instamart behave similarly (in-app first, inconsistent email), so an email-only front door under-covers the vertical (inference from documented invoice-retrieval flows); q-commerce template churn is rapid; the payoff is interesting analytics, not money or correctness ? low recurring pull. CONS ? Public: the demand signal is actively negative: the only receipt-collection businesses anywhere (magicpin, Club Corra in India; Fetch, Slice in the US) PAY consumers for receipts because the value accrues to brands/panels, not the consumer ? and the data-resale monetization that makes receipts a business is precisely what the privacy thesis forbids. Google's native Gmail Purchases view also free-rides the generic version. VERDICT: Good second vertical and an excellent low-risk demo/test corpus for the classification front door; as the FIRST vertical it is wrong on both axes ? on personal utility it loses to finance (already built, higher pain), and as a public wedge it has no willingness-to-pay and incomplete email coverage. Build it after the finance spine exists, scoped to the user's own 2-3 platforms.

=== FINANCE WEDGE ===
PROS ? Personal utility: this is the strongest possible first vertical and it is already de-risked ? muneem parses HDFC/Axis/AU on real data with reconciliation gating, which no AA app or OSS tool matches. It directly fills documented AA gaps as of 2025-26: credit-card data remains the weakest FIP class (AA Phase 1 was CASA; investment data tilts to MF/depository), joint accounts are excluded from AA pulls, historical depth is limited, and there is still no individual/self-custody AA path ? so PDF statements remain the only complete, user-controlled source of one's own financial history. The reconciliation gate ('rows persist only if the balance walk matches the bank summary') is a real correctness differentiator over the dozen hobby parsers on GitHub that silently drop or duplicate rows. CONS ? Public product: the trust asymmetry is brutal ? an unknown indie tool asking for bank statements competes with CRED Protect, which already parses password-protected CC statement PDFs from Gmail, free, backed by a recognized brand; Fold/Jupiter/INDmoney offer one-tap AA linking with zero parsing friction; and the published-app path runs straight into gmail.readonly restricted-scope verification + annual CASA ($540?$4,500/yr + ~2 months process). Password-protected Indian statement PDFs force handling of PII-derived password rules, raising both engineering and liability surface. Parser maintenance across dozens of issuers is a permanent treadmill, and every quarter AA coverage improves, the wedge narrows. VERDICT: As a personal tool, unambiguously the right first vertical

? already proven, highest pain, fills real gaps. As a public product, viable ONLY in the self-deploy/auditable-OSS form ('run the code you read', personal-use exemption, no CASA) targeting the privacy-conscious Indian dev/finance-nerd niche ? realistically hundreds to low thousands of users (inference) ? not as a head-to-head consumer app against free AA-based incumbents.

=== OTHER WEDGES ===
* India FY tax-season document pack (STRONGEST alternative): auto-collect Form 16, interest certificates, capital-gains statements (Zerodha/CAMS/KFintech emails), AIS/26AS alerts, insurance 80C/80D receipts into one CA-ready bundle each April-July. Effort: moderate ? sources are a finite, slow-churning set of large issuers, mostly genuine email attachments (unlike grocery), and it aligns exactly with muneem's broker/MF-statement roadmap. Payoff: high ? acute seasonal pain every Indian filer feels, a natural annual-renewal pricing story, and the clearest answer to 'what would someone pay for'. Sensitive scope applies, but the self-deploy model covers it. This is arguably a better PUBLIC first wedge than either grocery or general finance.
* Warranties / durable-goods invoices + AMC dates: extract invoices for electronics/appliances from email (Amazon.in, Flipkart, Croma, brand stores), index purchase date, warranty period, serial numbers; remind before expiry. Effort: low-moderate ? invoice emails are common, formats moderately stable, low sensitivity (could even avoid full-content scopes in a self-deployed script). Payoff: moderate ? real but episodic utility; the Blinkit-GST-invoice-for-warranty pattern shows Indians already care about keeping invoices for claims. Underserved niche, weak monetization, good goodwill/demo vertical.
* Subscriptions, renewals & e-mandate alerts: detect recurring charges from renewal emails, SI/e-mandate notifications, and insurance premium reminders ? the Truebill origin story, India-localized (RBI's e-mandate regime generates a clean, semi-standardized email/SMS trail ? inference). Effort: low-moderate. Payoff: this is the one email-adjacent category with PROVEN consumer willingness-to-pay globally (Rocket Money's premium business). Risk: card-side and AA-side detection by incumbents commoditizes it; still, as a front-door feature it is the highest-WTP signal extractor per unit effort.
* Travel/bookings (TripIt-for-India): parse MakeMyTrip/Cleartrip/IRCTC/airline confirmation emails into itineraries. Effort: moderate (formats well-structured). Payoff: low as a standalone ? TripIt, Gmail's native parsing, and Google Wallet already cover most of it; only worth doing as a corpus-diversity exercise for the classifier, not as a wedge.
* Coupons/deals carryover (the Email-Mining predecessor): lowest sensitivity, fun, already partially built. Payoff: lowest ? Gmail's Promotions tab, CashKaro, and GrabOn commoditize it, and expiring-coupon value is small. Keep as a free delight feature inside the front door, never the wedge.

=== PMF ASSESSMENT ===
OVERALL: the direction is reasonable ? with the crucial clarification that today this is a personal system with product-grade engineering, and the honest near-term ambition should be 'category-best personal tool + niche OSS' rather than consumer product. PERSONAL-UTILITY AXIS: near-maximal. The two repos compose correctly (Apps Script = acquisition inside the user's own trust domain; muneem = local sensitive processing), and the architecture's constraints (no cloud LLMs on statements, reconciliation gating, encrypted local ledger) cost little personally while delivering something no Indian app offers: a complete, verified, user-owned financial history including credit cards, bonds, and pre-AA history. PUBLIC-PRODUCT AXIS: the moat question. 'Reconciliation-gated parsing of Indian PDFs, fully local' IS differentiated ? vs AA apps (cloud-resident data, lending-subsidized business models per the Walnut?Axio?Amazon arc, and persistent coverage gaps in CC detail/bonds/joint accounts/history), vs OSS PFMs (Actual/Firefly/beancount have zero Indian PDF ingestion; community importers are CSV-only and stale), and vs hobby parsers (single-bank, unmaintained, no correctness guarantees). But it is a maintenance moat, not a defensibility moat: the asset is the discipline (validation gates, provenance, regression corpora) plus the grind of tracking issuer template churn ? replicable by any funded team, and increasingly automatable with LLMs (which cuts both ways; muneem's local-VLM fallback is the right hedge). The deepest structural insight from the market history: every consumer email-parsing business either sold the data (Slice), pivoted to bank feeds (Truebill), or owned one narrow vertical (TripIt). A privacy-first product refuses the first path and India's AA is the second ? so a public muneem must win the third way: own a vertical where PDFs/email beat AA permanently (credit-card statement detail, tax-season documents, historical archives). WHAT MAKES IT PAYABLE: (1) the tax-season pack ? annual, acute, CA-shareable output; (2) 'verified import' as the brand promise ? every statement reconciled or flagged, which no competitor states; (3) a parser-definitions maintenance subscription on top of MIT code (the WordPress/Sidekiq pattern) for self-hosters; (4) optionally B2B (CAs/family offices) ? but Precisa-class vendors already serve lenders, so that is a different, crowded motion. THE 100-USER APPS SCRIPT PROBE: sensible and nearly free, with two concrete caveats: (a) a

centrally-published Testing-status OAuth client with restricted scopes issues refresh tokens that expire every 7 days (per Google's OAuth docs), forcing weekly re-auth that will contaminate retention signals; (b) the better-designed probe is the one already implied by 'run the code you read' ? per-user self-deployment keeps every install inside Google's personal-use/internal exemptions indefinitely, costs zero CASA, and the conversion rate from README-reader to completed-self-deploy IS the PMF metric. If self-deploys stall in the dozens, the verdict is 'excellent personal tool, no product' ? a perfectly good outcome. If they reach hundreds organically (the beancount-India and r/IndiaInvestments self-hosting crowds are the seedbed ? inference), that justifies spending the ~$1-5K/yr CASA toll to publish. Do not pay the toll before the signal.

=== PUBLISHING CONSTRAINTS ===
As of mid-2026, publishing an app that reads Gmail content (gmail.readonly is a RESTRICTED scope; there is no non-restricted substitute for reading message bodies/attachments) requires: (1) OAuth brand verification (typically 2-3 business days); (2) restricted-scope verification ? scope-by-scope justification, a demo video of the OAuth flow and scope usage, accurate consent screen, declared application type ? which Google says 'can potentially take several weeks'; and (3) a CASA (Cloud Application Security Assessment, App Defense Alliance framework) security assessment, repeated AT LEAST EVERY 12 MONTHS, for any app that can move restricted data through its own/third-party servers. CASA tiers and 2026 costs: Tier 2 lab assessments run roughly $540?$1,800 (the Tier 2 self-scan option has been removed ? an authorized lab is now mandatory, a 2025 tightening); Tier 3 (required for higher-risk apps, large user counts, and Marketplace badge scenarios) runs ~$4,500 (TAC Security's published example). Authorized labs: TAC Security, Leviathan, DEKRA, Bishop Fox. Realistic first-time timeline: 1-4 weeks of lab turnaround inside an overall ~2-month project including remediation; October 2025 Google policy updates made CASA unambiguous for restricted Gmail/Drive scopes. EXEMPTIONS that matter for this project: personal-use-only apps, apps in development/testing/staging, and internal/domain-wide-install apps are exempt from verification ? so (a) the current <100-user Testing mode is free and legal, but external Testing-status clients with restricted scopes get 7-DAY refresh-token expiry (weekly re-consent) per Google's OAuth documentation, and users see an unverified-app warning; (b) the 'each user deploys their own copy of the MIT code in their own account' model keeps every deployment inside the personal-use exemption permanently ? no verification, no CASA, no token expiry pathology (Apps Script running as the account owner), at the cost of onboarding friction. One design note: an Apps Script that only files attachments into the user's own Drive could in principle narrow Drive-side scope to drive.file (non-restricted), but the Gmail-reading side stays restricted regardless of vertical ? the toll is triggered by reading email, not by what is extracted. Practical 2026 budget if publishing: $1,000?$5,000/year recurring (CASA + incidentals) plus 1-2 months of calendar time for the first pass ? material for a solo developer, trivial for a funded competitor, which is itself strategic information.

25. assistant (2026-06-10T16:36:13.114Z)

```
---
name: deep-review-2026-06
description: Full findings of the 2026-06-10 deep review of muneem + gmail-attachments-to-gdrive
(code, security, architecture, roadmap, PMF) ? 35-agent workflow, all critical/high findings
adversarially verified 3/3.
metadata:
  type: project
---
```

Deep review ? muneem + gmail-attachments-to-gdrive (2026-06-10)

Method: 8 parallel reviewers (muneem core / parsers / security / tests-CI / docs-roadmap; gmail code / gmail docs; web market research), every critical/high code finding verified by 3 adversarial agents with distinct lenses. 9/9 confirmed, 0 refuted. Offline suite run live: 390 passed, 6.35s, 86.17% coverage.

Confirmed critical/high code findings

muneem

1. **[critical] rotate-key destroys the only durable key copy before rekeying** ? `cli.py:438-446` writes the fresh key into the keychain *first*, then rekeys; rollback only

catches `EncryptedDatabaseError` while `PRAGMA rekey` raises raw `sqlcipher3.OperationalError` (e.g. database-is-locked ? likely on Windows with AV). Crash/Ctrl-C/locked-DB in the window ? keychain=fresh, file=old, old key gone; ledger + all backups unrecoverable unless recovery file exists (which wraps the OLD key and goes stale after successful rotation). Inline comment asserts the opposite of the behavior; contradicts `rekey_database`/`set_key` docstrings. Fix: pending-key dance (write fresh under `db-key-pending`, rekey, promote, delete) or at minimum rekey-file-first + broad except + keep old key under `db-key-prev`. Zero test coverage on the failure path.
2. **[high] `db backup <ledger-path>` destroys the ledger** ? `db.py:292-295` unlinks the destination before validating it isn't the live DB ? silently replaces ledger with empty DB.
3. **[high?medium] transient keychain read failure mints a new key over the real one** ? `secrets.get_secret` swallows `KeyringError`?None; `get_or_create_key` (db_key.py:24-31) treats None as "no key ever" and overwrites. `connect_encrypted` calls it even when an encrypted ledger already exists on disk ? exactly when minting is never right.
4. **[high] savings oracle ignores opening/closing bookends when the balance walk passes** ? `savings.py:78-84,194-205` short-circuits; a statement missing leading/trailing transactions persists as `reconciled` (empirically confirmed).
5. **[high] Cr/Dr suffix sign-stripped in `parse_amount`** ? `validation.py:32,45`; an overdraft (Dr-balance) statement reconciles with every debit/credit INVERTED and persists as verified (empirically confirmed).
6. **[high] HDFC persists rows even when UNRECONCILED; `txns list` never filters by document status** ? `hdfc.py:435-458`. Violates the store-nothing-unverified philosophy the other parsers follow.

gmail-attachments-to-gdrive

7. **[high] backfill advances `lastSynced` past windows with attachment-level errors** ? `sync.ts:213-218`; failed messages are never retried ("mark done only on clean completion" invariant broken at window level).
8. **[high/med] incremental sync advances historyId past messages truncated by the daily cap or transient failures** ? `sync.ts:175-189`; silent data loss (opt-in flag, default off).
9. **[med] `GmailApp.getMessageById` throws on deleted messages** ? `gmailHistory.ts:70-77`; wedges incremental sync until cursor expiry.

Notable mediums (selected)

- muneem: reconcile tolerance == currency quantum w/ strict-greater ? ~42% of one-paisa corruptions pass (validation.py:70-118); substring+alphabetical parser routing ? '@aubank' in a narration routes an HDFC statement to AUParser (au.py:33-37); sha256 dedup permanently blocks re-ingesting a quarantined doc after a parser fix (cli.py:121-124); Axis CC parser persists with ZERO verification; dates never parsed/normalized (raw locale strings persisted); migration runner partial-failure window (DDL commits before recording; untested); forward-migration checksums recorded but never re-verified; RedactingFilter misses PAN/spaced-card/Aadhaar; key spliced into PRAGMA without 64-hex validation.
- muneem security (posture strong, history verified clean ? nothing sensitive ever committed): real parsed CC rows sit in plaintext `testdata/snapshots/*.json`, written by a now-dead `snapshot` fixture in conftest (no test uses it); `client_secret_*.json` (Google's default download name) escapes all three guard layers once outside testdata/; no lockfile/pinning/hash-checking incl. the sqlcipher3 native wheel; recovery passphrase floor 8 chars; pre-commit hook opt-in.
- gmail: `getOrCreateFolderInFolder(DriveApp,?)` iterates ALL Drive folders not root children; state persisted only at run end (6-min kill loses bookkeeping); setup re-run races in-flight sync (no lock in processInput); same-filename attachments silently dropped; Cloud Logging carries message ids + error text (tension with PRIVACY.md "author has no access"); committed timeZone America/Los_Angeles; `SUMMARY_EMAIL_RECIPIENT` global; CI creates a new deployment per push.

Architecture / roadmap issues

1. **Cross-repo contradiction (the #1 issue overall):** gmail repo's *binding* auditability invariant (ADR-008, 2026-06-03) says it is the ONLY Gmail-touching code and "Muneem never holds the Gmail grant / sees only minimized non-PII signal" ? while muneem ships full gmail.readonly OAuth ingestion and plans `muneem fetch` (Phase F). Bank statements are PII; they cannot ride a "non-PII" handoff; GAS cannot even open password-locked PDFs. Sister also mischaracterizes muneem as a "hosted service" (it's local-first by design). Neither repo references the other's constraint. Options: (a) muneem keeps ingest + invariant rescoped to the public product; (b) muneem consumes the (undefined) handoff ? blocked, PII-incoherent; (c) **middle path

(recommended): sister archives attachments to Drive in-account (already its hardened core), muneem ingests from the locally-synced Drive folder** ? single Gmail grant, unlock stays local, no handoff contract needed.
2. Two competing sequencing sources of truth: ROADMAP.md Milestones/Tracks vs feedbackJune2026 Phases A?H, colliding letter namespaces; ROADMAP checkbox rot; CURRENT_STATE.md says "Schema v4"/per-row provenance planned (stale: v5 shipped, separate-table decided); parser-architecture.md says Axis 2/4, HDFC 4/4 (stale: 6/7, 6/6) and §9 still points at the superseded unified-table direction.
3. Phase C (instrument family) is the riskiest, least-specified phase: no broker/MF/CAS corpus in hand, no oracle design (what replaces the balance walk for holdings?), artifact choice (CAS vs broker stmt vs tradebook) unparked ? and sequenced BEFORE regression CI Layer 2 + D.3 eval, contradicting the project's own "characterize failure modes first" logic. Gate behind an entry checklist; consider swapping C and D; verify Axis CC on real data first (cheapest unverified thing already written).
4. **No daily user value anywhere in the roadmap**: only `txns list <bank>`/`offers list` exist after ~46 PRs; the Phase-B serving view was decided+promised and silently dropped (provenance table shipped without it). Ship view + `txns all --since/--month` + monthly summary early ? it's what keeps the user feeding the tool through the C?G middle.
5. Missing roadmap categories: cash/manual entry (source_type='manual' already reserved), multi-year Gmail backfill + raw-PDF staging retention strategy (PDFs outside SQLCipher protection), quarantine UX, docs/recovery.md runbook, refreshed definition of done; SMS-alert parsing (P11) unowned. Phase A quietly incomplete (no lockfile; structured logging stalled on log-format decision).
6. gmail repo: vision far ahead of roadmap ? zero tracked items for Phase 0/1, the handoff contract, or the grocery vertical VISION names first; handoff contract exists in no artifact; Phase 2 in-account OCR not feasibility-checked (and impossible for password-locked PDFs); auditability weaker than stated for centrally-deployed users (Cloud Logging, no reproducible build/deploy-your-own guide yet); 7-day refresh-token expiry in Testing mode kills the always-on value prop and contaminates the <100-user probe; user 101 = CASA wall ($540?$4,500/yr + ~2 months, annually recurring; both gmail.readonly and full drive are restricted). Self-deploy ("run the code you read") is the distribution channel that scales with zero verification and maximal invariant compliance ? make it the primary public posture.

PMF (researched, mid-2026)

- India PFM consolidated on AA rails (Fold, INDmoney, Jupiter) ? cloud-first, lending-subsidized, free; coverage gaps persist exactly where muneem plays: CC detail, bonds, joint accounts, pre-AA history; still no individual/self-custody AA path (AA-deferral decision still holds).
- CRED Protect already parses password-protected CC statement PDFs from Gmail, free ? can't win on convenience; win on locality + whole-ledger + verified-import.
- The attachment-archiving layer is commoditized (Digital Inspiration 2M+ installs, already decrypts bank PDFs; cloudHQ) ? Repo A's value must be the classification/extraction front door + auditable self-deploy trust model, not archiving.
- Email-parsing consumer history: sell the data (Slice?NielsenIQ), pivot to bank feeds (Truebill?Rocket Money), or own one narrow vertical (TripIt). Privacy thesis forbids #1, AA is #2 ? must win #3: own verticals where PDFs/email beat AA permanently.
- Moat: "reconciliation-gated parsing of Indian PDFs, fully local" is genuinely differentiated vs AA apps, OSS PFMs (zero Indian PDF ingestion in Actual/Firefly/beancount), and hobby parsers ? but it's a maintenance moat, not a defensibility moat.
- Wedge verdicts: **finance = right first vertical personally (already proven)**; grocery = wrong first wedge (Blinkit/Zepto invoices live in-app not reliably in email; receipt-collection businesses must PAY consumers; Google's Gmail Purchases free-rides) but excellent SECOND vertical / front-door test corpus (low-sensitivity ? cloud-LLM-permitted ? fast iteration); **India FY tax-season document pack = strongest public wedge** (Form 16, interest certs, CG statements from CAMS/KFintech/Zerodha, 80C/80D ? acute seasonal pain, finite slow-churning issuer set, email-native, annual-renewal pricing, aligns with instrument-family roadmap); subscriptions/e-mandate renewal detection = highest proven WTP per effort; warranties/invoices = goodwill niche; travel/coupons = no (commoditized).
- The 100-user probe: sensible but use self-deploy conversions as the PMF metric (Testing-mode weekly re-consent contaminates retention signal). Don't pay CASA before organic self-deploys reach hundreds.

Run/verification facts

- Offline suite: 390 passed / 21 deselected, 6.35s, coverage 86.17% (gate 80). CI matrix real

(ubuntu 3.11?3.13 + windows 3.13).
- git history verified clean across both repos for sensitive paths
(statements/credentials/snapshots/env/DB ? never committed).

26. user (2026-06-10T16:36:13.135Z)

File created successfully at: C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-muneem\memory\deep-review-2026-06.md (file state is current in your context ? no need to Read it back)

27. assistant (2026-06-10T16:36:35.754Z)

---
name: session-handoff
description: ? Start here ? living handoff for muneem (and its sister repo gmail-attachments-to-gdrive); current state, next steps, ramp-up kit, key decisions.
metadata:
  type: project

Session handoff ? muneem (+ gmail-attachments-to-gdrive)

> Last refreshed: 2026-06-10, after the cross-repo deep review.

1. You are here

- muneem `main` @ 98481ee (provenance table, PR #46). Parser precision on real data: HDFC 6/6, AU 4/4, Axis 6/7. Encryption + recovery + migrations shipped. Sister repo `gmail-attachments-to-gdrive` master @ a764030 (auditability invariant, PR #14).
- A 35-agent deep review of BOTH repos (code, security, architecture, roadmap, PMF) completed 2026-06-10. **All findings in [[deep-review-2026-06]]** ? 9 confirmed critical/high code bugs, the cross-repo Gmail-ownership contradiction, roadmap gaps, and a researched grocery-vs-finance-vs-tax-pack wedge assessment delivered to Avi. **Awaiting Avi's direction** on (a) which fixes to start, (b) the cross-repo seam decision, (c) the first-vertical/wedge choice.

2. Next steps / open threads

1. Avi to react to the review: pick wedge (recommendation: finance stays first personally; grocery as 2nd/front-door corpus; tax-season pack as the public wedge candidate).
2. If fixes approved, P0 order: rotate-key atomicity ? `db backup` destination guard ? savings-oracle bookend check ? Cr/Dr sign semantics ? HDFC unreconciled persistence ? gmail backfill retry. One PR per item (see [[pr-workflow-cadence]]).
3. Cross-repo ADR needed: who reads Gmail (recommended middle path: sister archives to Drive in-account; muneem ingests the locally-synced Drive folder; invariant scoped to the public product). Record in both repos.
4. Doc debt: consolidate sequencing into one owner (ROADMAP vs feedbackJune2026 Phases A?H); refresh CURRENT_STATE (schema v5) + parser-architecture (stale precision numbers, superseded §9); ship the dropped Phase-B serving view + `txns all`.
5. Hygiene quick wins: delete dead `snapshot` fixture + the two real-CC-row JSONs in testdata/snapshots/; block `client_secret*.json` everywhere; add lockfile.

3. Ramp-up kit

- [[deep-review-2026-06]] ? the full review findings (memory dir).
- muneem: `CLAUDE.md` ? `DESIGN.md` ? `docs/feedbackJune2026.md` (the governing phase plan) ? `docs/CURRENT_STATE.md` (stale re: schema v5 ? see review).
- Sister: `docs/VISION.md` ? `docs/ARCHITECTURE.md` (ADR-008 invariant) ? `ROADMAP.md`.
- Memory index `MEMORY.md` for standing decisions (licenses, local-first, storage shape, L3, AA-deferred).

4. Key decisions (don't relitigate)

- Python-only; PDF-first/AA-deferred; flat per-issuer tables + separate provenance table + (promised, unbuilt) serving view; hand-rolled migrations (no Alembic); SQLCipher + keychain + passphrase recovery; D.3 eval before L3; licenses MIT/BSD/Apache only (PyMuPDF banned); no cloud LLMs on sensitive docs; sister repo: <100-user Testing mode, CASA deferred.

- Still open (genuinely, verified in code): log format; self-hosted runner; macOS stance; the cross-repo Gmail seam (NEW ? surfaced by the review); instrument-family artifact choice (CAS vs broker stmt vs tradebook).

28. user (2026-06-10T16:36:35.764Z)

File created successfully at: C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-muneem\memory\session-handoff.md (file state is current in your context ? no need to Read it back)

29. user (2026-06-10T16:36:44.124Z)

<tool_use_error>File has not been read yet. Read it first before writing to it.</tool_use_error>

30. user (2026-06-10T16:36:50.925Z)

<system-reminder>This memory is 6 days old. Memories are point-in-time observations, not live state ? claims about code behavior or file:line citations may be outdated. Verify against current code before asserting as fact.</system-reminder>
1 # muneem ? Memory Index
2
3 - [PR workflow cadence](pr-workflow-cadence.md) ? one PR per work item; verify locally, then squash-merge as you go.
4 - [Prefer local over paid APIs](prefer-local-over-paid-apis.md) ? self-hosted/local/own-trained first; paid APIs only as a fallback.
5 - [Commercial license safety](commercial-license-safety.md) ? only MIT/BSD/Apache deps; avoid (A)GPL. PyMuPDF (AGPL) is banned.
6 - [HDFC parser tech decision](hdfc-parser-tech-decision.md) ? hand-roll coordinate clustering on pdfplumber; camelot-py is the license-safe fallback.
7 - [Parser validation pattern](parser-validation-pattern.md) ? every parser: split parse/persist, reconcile via shared validation, bookend vs the bank's SUMMARY, record documents.status.
8 - [Scope: all personal finance](scope-all-personal-finance.md) ? muneem covers the WHOLE financial life; equity/MF/ETF/bond parsing + instrument-event classification are core near-term goals, not backlog. Don't over-fit the concept core to bank rows.
9 - [L3 vision + eval strategy](l3-vision-and-eval-strategy.md) ? L3 fallback = DeepSeek-OCR-3B (MIT), reconcile-gated; dev RTX 2070S (8GB/Turing) ? 4-bit + SDPA. Golden evals: local + synthetic first; cloud eval-only & approval-gated.
10 - [DB encryption at rest](db-encryption-at-rest.md) ? D.2 DECIDED & IMPLEMENTED: SQLCipher via `sqlcipher3` (transparent AES-256), 256-bit key in OS keychain; `db.connect_encrypted()`; `:memory:`/tests stay plaintext.
11 - [Open Banking + storage decisions](open-banking-and-storage-decisions.md) ? AA/open-banking DEFERRED (PDF-first; regulatory + cloud-decrypt barrier); storage = flat per-issuer source-of-truth tables + derived serving layer; provenance/confidence in a SEPARATE `provenance` table (not per-row); SMS-alert parsing is a local freshness candidate.
12

31. user (2026-06-10T16:36:58.908Z)

The file C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-muneem\memory\MEMORY.md has been updated successfully. (file state is current in your context ? no need to Read it back)

32. assistant (2026-06-10T16:38:36.395Z)

Everything is verified, persisted to memory, and ready. Here is the full synthesis.

Deep review: muneem + gmail-attachments-to-gdrive

TL;DR: Engineering quality in both repos is genuinely high ? the review confirmed the hygiene claims are real, not aspirational (git history verifiably clean, 390 offline tests green in 6.4s at 86% coverage, invariant tripwires work). But it found **9 confirmed critical/high

bugs** (each independently verified by 3 adversarial agents; zero refuted), concentrated in exactly the two places you'd least want them: muneem's **encryption-key lifecycle** and the **arithmetic verification oracle that is the product's core promise**. The single biggest issue isn't code at all: the two repos' "binding" architecture documents **contradict each other head-on** about who reads Gmail. On direction: **finance is the right first step and it's already de-risked ? keep it. Grocery is the wrong *first* wedge but an excellent *second* vertical**, and the research surfaced a stronger public wedge than either: the **India tax-season document pack**. Details below; the full findings are saved to project memory ([deep-review-2026-06.md](C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-muneem\memory\deep-review-2026-06.md)).

---

1. Code issues (all confirmed by 3/3 adversarial verifiers)

muneem ? key lifecycle (the scariest cluster)

1. **`db rotate-key` can permanently lock you out of the ledger** ([cli.py:438](src/muneem/cli.py:438)). It overwrites the keychain with the *new* key **before** rekeying the file, and the rollback only catches `EncryptedDatabaseError` ? but `PRAGMA rekey` raises raw `sqlcipher3.OperationalError` (e.g. "database is locked", very plausible on Windows with antivirus holding the file), which escapes uncaught. Result: keychain holds the new key, file is still under the old key, old key is gone. Backups share the key, so they're lost too. The recovery file helps only if set up ? and it wraps the *old* key, so it silently goes stale after every successful rotation. The inline comment claims the opposite of what the code does, the failure path has **zero test coverage**, and docs/encryption-at-rest.md repeats the inverted claim.
2. **`muneem db backup <ledger-path>` destroys the ledger** ([db.py:292](src/muneem/db.py:292)) ? the destination is unlinked before any check that it isn't the live DB.
3. **A transient keychain read failure mints a fresh key over the real one** ([db_key.py:24](src/muneem/db_key.py:24)) ? `get_secret` swallows `KeyringError` as "no key", and `connect_encrypted` calls `get_or_create_key` even when an encrypted ledger already exists on disk, which is precisely the situation where minting is never the right answer.

These three share one theme: **the encryption is sound, but the key's lifecycle isn't atomic.** For a tool whose pitch is "hold my whole financial life for years," this is the same class of risk your own feedbackJune2026 §2 correctly named P0 ? the recovery file solved half of it; rotation/backup/mint re-opened the other half.

muneem ? the verification oracle has holes exactly where arithmetic can't see

4. **The savings oracle never checks the printed opening/closing bookends when the per-row balance walk passes** ([savings.py:78](src/muneem/parsers/savings.py:78)) ? a statement missing its leading or trailing transactions persists as `reconciled`. Empirically confirmed by the reviewer.
5. **`parse_amount` strips Cr/Dr suffixes without sign semantics** ([validation.py:32](src/muneem/parsers/validation.py:32)) ? an overdraft statement (Dr balances) reconciles perfectly with **every debit and credit inverted** and persists as verified. Also empirically confirmed. This is the worst-case failure mode your architecture exists to prevent: wrong rows that *pass* the oracle.
6. **The HDFC parser persists rows even when unreconciled**, and `txns list` never filters by document status ([hdfc.py:435](src/muneem/parsers/hdfc.py:435)) ? the reference implementation violates the store-nothing-unverified philosophy that Axis/AU follow.

Worth knowing too: the reconcile tolerance equals the currency quantum with a strict-greater compare, so ~42% of exact one-paisa corruptions pass the walk (measured by the reviewer); parser routing is bare-substring + alphabetical, so an HDFC statement containing an `@aubank` UPI handle in a narration routes to the AU parser; sha256 dedup permanently blocks re-ingesting a quarantined statement after you fix the parser; the Axis credit-card parser persists with **no verification at all**; and dates are never parsed/normalized ? raw locale strings go into the ledger.

gmail-attachments-to-gdrive

The backfill core is well-designed (the ±1-day window overlap, the trigger fan-out, the FIFO dedup all check out), but:

7. **Backfill advances `lastSynced` past windows that had attachment-level errors** (`src/sync.ts:213`) ? failed messages are never retried. The "mark done only on clean completion" invariant holds per-message but not per-window.
8. **The opt-in incremental sync advances the historyId cursor past messages truncated by the daily cap or transient failures** (`src/sync.ts:175`) ? silent data loss in steady-state mode.
9. **`GmailApp.getMessageById` throws for deleted messages** (`src/gmailHistory.ts:70`) ? one deleted email wedges incremental sync until the cursor expires.

Plus mediums that matter for the product story: `getOrCreateFolderInFolder(DriveApp, ?)` iterates **every folder in the user's Drive** (not root children); state persists only at run end, so the 6-minute hard kill loses bookkeeping; execution logs carry Gmail message ids into developer-visible Cloud Logging ? in tension with PRIVACY.md's "the author has no access to your data"; and the committed `timeZone` is America/Los_Angeles, which governs day boundaries for a Bangalore owner.

Security posture (muneem)

Genuinely strong ? and the review verified the ground truth: **nothing sensitive is tracked now or was ever committed** in either repo's full history. The residual risks are local: two JSON files in `testdata/snapshots/` contain **real parsed credit-card transactions in plaintext** on the same disk the ledger encrypts (written by a `snapshot` fixture in conftest that no test uses anymore ? a loaded footgun); the real `client_secret_*.json` is protected only by being inside `testdata/` ? Google's default download filename is not blocked by `.gitignore`, the pre-commit hook, or the hygiene test if it ever lands anywhere else; and there is **no lockfile or version pinning at all**, including for the `sqlcipher3` native wheel that provides the at-rest encryption itself.

---

2. The architecture issue that outranks all code findings

The gmail repo's **binding auditability invariant** (ADR-008, codified June 3) declares: *this repo is the only code that ever touches Gmail; muneem never holds the Gmail grant and receives only minimized, non-PII signal.* Meanwhile muneem **ships** full `gmail.readonly` OAuth ingestion and plans `muneem fetch` (Phase F). Neither repo references the other's constraint. And it's worse than a stale doc ? the contract itself is incoherent for the finance vertical:

- Bank statements **are PII through and through**; merchant names, amounts, and account identifiers *are* the signal. A "non-PII minimized" payload is useless to muneem.
- Apps Script **cannot open password-protected PDFs** (no decryption primitive), so the REDACT/MINIMIZE stage can't even read the document it's supposed to scrub.
- The sister repo frames muneem as "our own isolated, **hosted** service" ? but muneem's DESIGN declares hosted/cloud-sync a non-goal, period.
- The "versioned handoff contract" the entire decoupling rests on **exists in no artifact in either repo**.

There's a clean reconciliation neither doc names: **the sister repo archives raw attachments to the user's own Drive in-account (already its hardened core ? that's not egress), and muneem ingests from the locally-synced Drive folder.** Single Gmail grant, unlock stays local, no handoff contract needed, invariant intact. The pragmatic interim is to scope the invariant explicitly ("binds the public product; the owner's personal local tools are exempt"). Either way, this needs a one-page cross-repo ADR recorded in both repos ? right now two documents written the same week, both calling themselves binding, contradict each other silently, and it **blocks Phase F**.

---

3. Roadmap issues

muneem:
- **Two competing sources of truth** ? ROADMAP.md's Milestones/Tracks vs feedbackJune2026's

Phases A?H, with colliding letter namespaces ("Track A" ? "Phase A", "Milestone C" ? "Phase C"). A restart can't tell which plan governs. CURRENT_STATE.md ? the designated live-status doc ? still says "Schema v4" and describes per-row provenance as the planned direction, both superseded; parser-architecture.md still says Axis 2/4 and HDFC 4/4.
- **Phase C (instrument family) is the riskiest, least-specified phase**: no broker/MF/CAS corpus in hand (the bank parsers' credibility came precisely from real-statement validation), no oracle design (what arithmetic identity replaces the balance walk for holdings? dividends/fees have none), the source artifact (CDSL/NSDL CAS vs broker statement vs tradebook) not even parked as a decision ? and it's sequenced *before* the regression-CI safety net, contradicting your own "characterize failure modes before building" logic from the D.3-before-L3 push-back. Cheapest unverified thing already written: the Axis CC parser. Validate that first.
- **Nothing in the roadmap delivers daily user value.** After ~46 PRs the only read surface is `txns list <bank>` (50 raw rows, one bank). The Phase-B serving view was decided, promised alongside the provenance table ? and silently dropped when #46 shipped. Filters/analytics sit in Phase H, dead last. For a single-user tool, a weekly-usable "show me last month across all accounts" is what keeps you feeding it data through the long C?G middle.
- **Missing categories entirely:** cash/manual entry (the schema already reserves `source_type='manual'`), multi-year Gmail backfill + retention policy for raw PDFs in the staging dir (which sit *outside* SQLCipher's protection), quarantine UX for the 1-in-7 statement that stores nothing, the prescribed `docs/recovery.md` runbook, and a refreshed definition of done. Phase A is also quietly incomplete: no lockfile, structured logging stalled on the open log-format decision.

gmail repo: the vision and roadmap describe two different projects. ROADMAP.md is a rear-view mirror of completed copier hardening; **zero tracked items** exist for Phase 0 (EmailMessage ingest model), Phase 1 (classifier/extractors/Sheets index), the handoff contract, or the grocery vertical the VISION names *first*. Phase 2's in-account OCR has no feasibility math (and is impossible for locked PDFs). And two product-killing operational facts are documented as one-line caveats: **Testing-mode refresh tokens expire every 7 days** ? your <100 test users experience a background archiver that dies weekly, so retention signal from the probe will be noise ? and **user 101 is a hard CASA wall** (~$540?$4,500/yr recurring + ~2 months process; both `gmail.readonly` and full `drive` are restricted scopes). The distribution channel that scales past 100 with zero verification and *maximal* invariant compliance ? self-deployment ? exists only as unchecked roadmap stubs.

---

4. Product-market fit: is the direction reasonable?

Yes ? as a personal system with product-grade engineering, and as a niche open-source play. Not (yet) as a mainstream consumer product. The market research (mid-2026, sourced) says:

- **India PFM has consolidated on Account Aggregator rails** (Fold, INDmoney, Jupiter) ? cloud-first, free, lending-subsidized (the Walnut?Axio?Amazon arc shows tracking was always underwriting-data acquisition). Fold's tiny revenue after 6 years shows Indians don't pay for pure PFM. But AA's gaps are persistent and structural: weak credit-card detail, no bonds, no joint accounts, ~2y history, and **still no individual/self-custody path** ? your AA-deferral decision holds, and those gaps are exactly where reconciliation-gated PDF parsing is the only complete source.
- **CRED Protect already parses password-protected CC statement PDFs from Gmail, free.** You cannot win on convenience; you can win on locality, whole-ledger scope, and "every row verified or quarantined" ? a promise no competitor makes.
- **The attachment-archiving layer is fully commoditized** ? Digital Inspiration's add-on alone has 2M+ installs *and already decrypts password-protected bank PDFs*. Repo A's value is the classification front door and the auditable self-deploy trust model, never archiving itself.
- **The history of consumer email-parsing is a warning:** survivors either sold the data (Slice?NielsenIQ ? your privacy thesis forbids it), pivoted to bank feeds (Truebill?Rocket Money ? AA is that gravity in India), or owned one narrow high-pain vertical (TripIt). A public muneem must win the third way.
- **The moat is real but it's a maintenance moat** (validation discipline + issuer-template grind + regression corpora), not a defensibility moat. The OSS angle is underexploited: Actual/Firefly/beancount have **zero Indian PDF ingestion**; the beancount-India importers are CSV-only and stale. muneem as *the* verified ingestion layer for that ecosystem is a cheap distribution wedge into an existing community.

- **The 100-user probe is nearly free and sensible ? but measure self-deploy conversions, not Testing-mode retention** (which the 7-day token expiry contaminates). If organic self-deploys stall in the dozens: excellent personal tool, no product ? a fine outcome. If they reach hundreds, *then* the CASA toll is justified.

---

5. Grocery vs finance vs something else ? my recommendation

The question is partly a false binary, because the two repos answer it at different layers:

- **Finance stays first ? and it's not really a choice anymore: it's already built and proven** (HDFC 6/6, AU 4/4, Axis 6/7 on real data, with an architecture nothing in the Indian market matches). The marginal cost of continuing is low, the personal pain it solves is the highest, and it fills documented AA gaps. Abandoning a de-risked vertical to start a new one would be strictly worse.
- **Grocery is the wrong *first* consolidation target.** The killer is source coverage: Blinkit/Zepto/Instamart itemized GST invoices live primarily **in-app**, not reliably as email attachments ? an email-only front door under-covers the vertical from day one. The public demand signal is actively negative (the only receipt-collection businesses anywhere ? magicpin, Club Corra, Fetch ? must *pay* consumers for receipts; the value accrues to brands, monetized by exactly the data-resale your thesis forbids; Google's Gmail Purchases free-rides the generic version). **But grocery is an excellent *second* vertical and the right first *test corpus* for the gmail repo's classifier** ? receipts are low-sensitivity, so they're cloud-LLM-permitted under your own rules, which makes extractor iteration fast, exactly like the promo corpus did for muneem.
- **The strongest wedge you haven't considered: the India FY tax-season document pack.** Auto-collect Form 16, bank interest certificates, capital-gains statements (Zerodha/CAMS/KFintech), AIS/26AS alerts, and 80C/80D receipts into one CA-ready bundle every April?July. The sources are a finite, slow-churning set of large issuers that genuinely arrive by email; the pain is acute, seasonal, and universal among Indian filers; it has a natural annual-renewal pricing story; and it aligns exactly with the instrument-family work already on muneem's roadmap.
Subscriptions/e-mandate renewal detection is the runner-up ? the one email-adjacent category with globally proven willingness-to-pay (Rocket Money), and RBI's e-mandate regime generates a clean notification trail.

So: **muneem keeps grinding finance; the gmail repo's Phase-1 extractor targets grocery receipts as its proof-of-pipeline (which its own VISION already says, for the right reason ? forgiving fidelity bar); and if/when you want a public wedge, lead with the tax pack, distributed self-deploy-first.**

---

6. Improvement plan

Short-term (concrete, fixable now ? roughly in priority order):
1. **rotate-key atomicity** ? pending-key dance (or rekey-file-first + broad except + keep old key under `db-key-prev`), plus failure-path tests. Also: have rotation refresh/invalidate-with-warning the recovery file, which currently goes silently stale.
2. **`db backup` destination guard** (refuse the live ledger path) and **never-mint-when-ledger-exists** in `get_or_create_key`.
3. **Oracle integrity**: bookend check even when the walk passes; Cr/Dr sign semantics in `parse_amount`; stop persisting unreconciled HDFC rows (or make `txns list` status-aware); widen the one-paisa tolerance comparison.
4. **Hygiene**: delete the dead `snapshot` fixture + securely delete the two real-CC-row JSONs; block `client_secret*.json` in all three guard layers; add a lockfile (overdue for a compiled-dep project).
5. **gmail repo**: retry windows with attachment errors; don't advance historyId past unprocessed messages; try/catch `getMessageById`; fix the folder-iteration and timezone defaults.
6. **Docs (hours, high leverage)**: collapse sequencing into one owner; refresh CURRENT_STATE + parser-architecture; back-record the L3 decision in DESIGN §8.3.
7. **Ship the dropped Phase-B serving view + `muneem txns all --month`** ? days of work on a settled decision, and it's the first thing that makes the tool worth opening weekly.

Longer horizon:
- **Write the cross-repo ADR** (Gmail seam ? recommend the Drive-folder middle path; scope the invariant) and a handoff-contract v0 artifact.
- **Gate Phase C** behind an entry checklist (real broker/CAS corpus + a one-page holdings-oracle design + the artifact decision), and consider swapping it with Phase D (regression CI + D.3 eval) per your own sequencing logic. Verify the Axis CC parser on real data first.
- **Add the missing roadmap categories**: manual/cash entry, backfill + staging-dir retention policy, quarantine UX, recovery runbook, refreshed definition of done.
- **Make self-deployment the primary public distribution posture** for the gmail repo (it resolves the token-expiry, CASA, and auditability problems simultaneously), and measure README?completed-deploy conversion as the PMF metric.
- **Product sequencing**: finance (continue) ? grocery as front-door corpus ? tax-pack as the public wedge probe ? subscriptions as the WTP feature.

Everything above, with file:line evidence and verifier notes, is persisted in the project memory; the session handoff now points at it. If you want, I can start on the fix list ? the natural first PR is the rotate-key atomicity fix with its missing failure-path tests, one PR per item as usual.

33. user (2026-06-10T16:40:00.482Z)

Check on workflow wctcr4nab (deep review of muneem + gmail-attachments-to-gdrive). If complete, synthesize the final report answering: issues in code/architecture/roadmap, PMF assessment, short-term vs long-term improvements, and the grocery-vs-finance first-step recommendation.

34. assistant (2026-06-10T16:40:10.737Z)

Already done ? the wakeup was just the fallback heartbeat. Workflow `wctcr4nab` completed (35 agents, 9/9 critical/high findings confirmed by adversarial verification), and the full synthesis was delivered in my previous message: code/architecture/roadmap issues, the PMF assessment, short-term vs long-term improvement plans, and the recommendation (finance stays first; grocery as the gmail repo's second/test vertical; the India tax-season pack as the strongest public wedge).

The findings are also persisted durably: [deep-review-2026-06.md](C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-muneem\memory\deep-review-2026-06.md) holds the full detail, and the [session handoff](C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-muneem\memory\session-handoff.md) points at it with the proposed next steps.

Nothing is pending on my side ? the next move is yours: pick which fixes to start (my suggested first PR is the `rotate-key` atomicity fix plus its missing failure-path tests), and weigh in on the cross-repo Gmail-seam decision and the wedge choice.

35. user (2026-06-10T16:42:05.958Z)

Can you make a plan of first fixing what is checked in and making it bulletproof while I deeply scan all your amazing findings and refresh my brain cache as to how such inconsistencies arose? Please show me a plan of maximum fixes or improvements and then you can go and help make this repo more civilized

36. assistant (2026-06-10T16:44:21.239Z)

I'll first confirm the local git state (two gated branches are already queued from the prior session, and push is blocked on your PAT), then lay out the maximum-fix plan and start executing it.

37. user (2026-06-10T16:44:23.589Z)

```
  antigravity-response-docs 4641d2e docs: record antigravity-feedback response + deprecation
trigger + parked monetization
* main                     98481ee [origin/main] feat(db): provenance table (first forward
migration) + record parse provenance (#46)
  serving-view             3d9ee97 docs: refresh CURRENT_STATE (schema v6, serving view,
```

superseded step-5 wording)
3d9ee97 docs: refresh CURRENT_STATE (schema v6, serving view, superseded step-5 wording)
f51cd21 feat(db): cross-bank serving view (migration v6) + `txns list all`
98481ee feat(db): provenance table (first forward migration) + record parse provenance (#46)
4641d2e docs: record antigravity-feedback response + deprecation trigger + parked monetization
98481ee feat(db): provenance table (first forward migration) + record parse provenance (#46)
e285b98 docs: Open Banking (AA) research + decisions (AA deferred, flat-schema, SMS) (#45)

38. user (2026-06-10T16:45:42.880Z)

Task #1 created successfully: P1: rotate-key atomicity (pending-key protocol + failure-path tests)

39. user (2026-06-10T16:45:44.285Z)

Task #2 created successfully: P2: db backup destination guard

40. user (2026-06-10T16:45:46.170Z)

Task #3 created successfully: P3: keychain never-mint-over-existing-ledger

41. user (2026-06-10T16:45:47.584Z)

Task #4 created successfully: P4: Cr/Dr sign semantics in parse_amount/verifier

42. user (2026-06-10T16:45:49.029Z)

Task #5 created successfully: P5: bookend check even when balance walk passes

43. user (2026-06-10T16:46:11.249Z)

Task #19 created successfully: P19: docs/recovery.md runbook

44. user (2026-06-10T16:45:50.881Z)

Task #6 created successfully: P6: reconcile tolerance ? one-paisa corruption must fail

45. user (2026-06-10T16:45:52.326Z)

Task #7 created successfully: P7: HDFC oracle gating (store nothing when unreconciled)

46. user (2026-06-10T16:45:53.704Z)

Task #8 created successfully: P8: parser routing precedence (header/sender-anchored)

47. user (2026-06-10T16:45:55.117Z)

Task #9 created successfully: P9: status-aware dedup (re-ingest after parser fix)

48. user (2026-06-10T16:45:56.997Z)

Task #10 created successfully: P10: snapshot cleanup (dead fixture + real-CC JSONs)

49. user (2026-06-10T16:45:58.879Z)

Task #11 created successfully: P11: client_secret filename guards everywhere

50. user (2026-06-10T16:46:00.294Z)

Task #12 created successfully: P12: redaction pattern gaps (PAN, spaced cards, Aadhaar)

51. user (2026-06-10T16:46:01.707Z)

```
Task #13 created successfully: P13: dependency lockfile (constraints.txt + CI)
```

52. user (2026-06-10T16:46:03.592Z)

```
Task #14 created successfully: P14: migrations hardening (atomic apply+record, checksum re-
verify)
```

53. user (2026-06-10T16:46:04.997Z)

```
Task #15 created successfully: P15: small-fixes batch (verified lows)
```

54. user (2026-06-10T16:46:06.932Z)

```
Task #16 created successfully: P16: synthetic corpus expansion (all 4 families + adversarial
fixtures)
```

55. user (2026-06-10T16:46:07.834Z)

```
Task #17 created successfully: P17: OCR CI job on weekly cron
```

56. user (2026-06-10T16:46:09.709Z)

```
Task #18 created successfully: P18: docs refresh (parser-architecture, L3 back-record, README)
```

57. user (2026-06-10T16:46:16.596Z)

```
Updated task #1 status
```

58. user (2026-06-10T16:46:17.548Z)

```
1       """The ledger's encryption key ? minted once, kept in the OS keychain.
2
3       muneem's local ledger is **encrypted at rest** with SQLCipher (DESIGN.md §5.4 rule 4).
4       The 256-bit key is generated on first use and stored in the OS keychain via
5       :mod:`muneem.secrets` (Windows Credential Manager / macOS Keychain / Secret Service) ?
6       never on disk in plaintext, never in the repo, never logged.
7
8       It is handed to SQLCipher in **raw-key form** (``PRAGMA key = "x'<64 hex>'"``), which
uses
9       the 256-bit key directly and *skips* the passphrase KDF. That is correct here because
the
10      key is already full-entropy random; KDF stretching only buys security for low-entropy
human
11      passphrases (an opt-in we may add later ? see ROADMAP D.2 phased plan).
12      """
13
14      from __future__ import annotations
15
16      import secrets as _stdlib_secrets
17
18      from muneem import secrets as _keychain
19
20      # Keychain entry name (under the shared "muneem" service) holding the 64-hex-char DB
key.
21      _DB_KEY_NAME = "db-key"
22
23
24      def get_or_create_key() -> str:
25          """Return the ledger's 256-bit key as 64 hex chars, minting+storing one on first
use."""
26          existing = _keychain.get_secret(_DB_KEY_NAME)
27          if existing:
28              return existing
29          key = _stdlib_secrets.token_bytes(32).hex()
```

```
30          _keychain.set_secret(_DB_KEY_NAME, key)
31          return key
32
33
34      def peek_key() -> str | None:
35          """Return the stored key, or ``None`` if none has been minted yet. Never creates
one."""
36          return _keychain.get_secret(_DB_KEY_NAME)
37
38
39      def new_key() -> str:
40          """Generate a fresh random 256-bit key as 64 hex chars (not stored)."""
41          return _stdlib_secrets.token_bytes(32).hex()
42
43
44      def set_key(key: str) -> None:
45          """Overwrite the stored DB key ? used by key rotation, **after** the file has been
rekeyed."""
46          _keychain.set_secret(_DB_KEY_NAME, key)
47
```

59. user (2026-06-10T16:46:17.671Z)

```
1       """Secret storage via the OS keychain.
2
3       OAuth tokens, cached statement passwords, and the DB encryption key are stored here
(Windows
4       Credential Manager / macOS Keychain / Secret Service), never in the repo, on disk in
plaintext, or
5       in logs. See DESIGN.md section 5.
6
7       External-call discipline (every keyring call is wrapped with attribution):
8       - **read** (:func:`get_secret`) is *graceful* ? a missing or unavailable backend returns
``None``
9         (absence is a valid state; this also keeps headless CI happy).
10      - **write** (:func:`set_secret`) is *loud* ? a failure raises
:class:`~muneem.errors.KeychainError`,
11        because silently losing a write (e.g. the DB key) would be dangerous. The secret
*name* is
12        included for diagnosis; the value never is.
13      """
14
15      from __future__ import annotations
16
17      import keyring
18      from keyring.errors import KeyringError, PasswordDeleteError
19
20      from muneem.errors import KeychainError
21
22      _SERVICE = "muneem"
23
24
25      def get_secret(name: str) -> str | None:
26          """Return the stored secret, or ``None`` if absent or the keychain is
unavailable."""
27          try:
28              return keyring.get_password(_SERVICE, name)
29          except KeyringError:
30              return None
31
32
33      def set_secret(name: str, value: str) -> None:
34          """Store (or replace) a secret in the OS keychain. Raises KeychainError on
failure."""
35          try:
```

```
36              keyring.set_password(_SERVICE, name, value)
37          except KeyringError as exc:
38              # Name only ? never the value ? in the message (it may be a password or the DB
key).
39              raise KeychainError(f"could not store secret {name!r} in the OS keychain") from
exc
40
41
42      def delete_secret(name: str) -> None:
43          """Delete a secret if present; a no-op if it is not (or the keychain is
unavailable)."""
44          try:
45              keyring.delete_password(_SERVICE, name)
46          except (PasswordDeleteError, KeyringError):
47              pass
48
```

60. user (2026-06-10T16:46:25.360Z)

```
408:        f"encrypted {Path(db_path).name}: {n} table(s) migrated. A plaintext backup remains
at "
428:@db_app.command("rotate-key")
429:def db_rotate_key(
432:    """Rotate the ledger's encryption key (SQLCipher rekey) and update the keychain."""
439:    # Keychain-first with rollback: if the file rekey fails, the keychain still matches the
file.
442:        dbmod.rekey_database(db_path, current, fresh)
447:    typer.echo(f"rotated the encryption key for {Path(db_path).name}.")
450:@db_app.command("backup")
451:def db_backup(
452:    dest: Path = typer.Argument(..., help="Destination path for the encrypted backup
copy."),
455:    """Write an encrypted online-backup copy of the ledger (consistent even with WAL
active)."""
461:        dbmod.backup_database(db_path, dest, key)
468:@db_app.command("setup-recovery")
469:def db_setup_recovery(
471:        None, help="Recovery file path (default: <data dir>/muneem-recovery.json)."
474:    """Create a passphrase-wrapped recovery file for the ledger's encryption key.
476:    Lets you restore the key (and thus the ledger + backups) if the OS keychain is ever
lost. The
481:    from muneem import recovery
488:    out_path = out or (load_config().data_dir / "muneem-recovery.json")
489:    passphrase = getpass.getpass("New recovery passphrase: ")
494:        recovery.write_recovery_file(out_path, key, passphrase)
495:    except recovery.RecoveryError as exc:
499:        f"wrote recovery file to {out_path}. Store it OFF this machine (password manager / "
500:        "separate drive) and remember the passphrase ? you need BOTH to recover. Without
them, a "
501:        "lost keychain means the ledger and its backups are unrecoverable."
505:@db_app.command("recover")
506:def db_recover(
510:    """Restore the ledger's encryption key from a recovery file + passphrase (into the
keychain)."""
514:    from muneem import db_key, recovery
518:        key = recovery.read_recovery_file(file, passphrase)
519:    except recovery.RecoveryError as exc:
523:    # If the ledger is present, verify the recovered key actually opens it before committing
it.
529:            typer.echo("recovered a key, but it does not open the current ledger.",
err=True)
532:    typer.echo("recovered the encryption key into the keychain.")
```

61. user (2026-06-10T16:46:26.258Z)

```
        credit REAL,
        balance REAL
    );

    CREATE TABLE IF NOT EXISTS hdfc_transactions (
        id INTEGER PRIMARY KEY,
        document_id INTEGER NOT NULL REFERENCES documents(id) ON DELETE CASCADE,
        account_number TEXT,
        txn_date TEXT,
        value_date TEXT,
        particulars TEXT,
        ref TEXT,
        withdrawal REAL,
        deposit REAL,
        closing_balance REAL
    );
    """


    def _raw_key_pragma(key: str) -> str:
        """The SQLCipher raw-key PRAGMA: use the 256-bit ``key`` (64 hex chars) directly, no
KDF."""
        return f"PRAGMA key = \"x'{key}'\""


    def is_plaintext_sqlite(db_path: Path | str) -> bool:
        """True iff the file exists and begins with the *unencrypted* SQLite magic header.

        Lets us refuse to silently treat a plaintext ledger as encrypted (and vice-versa)
        without needing the key.
        """
        try:
            with open(db_path, "rb") as fh:
                return fh.read(16) == _SQLITE_MAGIC
        except OSError:
            return False


    def _harden_permissions(path: str) -> None:
        """Best-effort: restrict the DB file to its owner (0600). No-op on failure or
``:memory:``."""
        if path == ":memory:":
            return
        try:
            os.chmod(path, 0o600)
        except OSError:
            pass


    def _open_encrypted(path: str, key: str) -> sqlite3.Connection:
        """Open an encrypted connection via SQLCipher, verifying the key before returning.

        A keyed open NEVER silently downgrades to plaintext: if the binding is missing, or
the
        key is wrong (or the file is not an encrypted muneem ledger), this raises.
        """
        try:
            import sqlcipher3
        except ImportError as exc:  # pragma: no cover - exercised only without the binding
            raise EncryptedDatabaseError(
                "database encryption requested but the 'sqlcipher3' package is not
installed"
            ) from exc

        conn = sqlcipher3.connect(path)
```

```
        conn.execute(_raw_key_pragma(key))  # MUST be the first statement on the connection
        try:
            conn.execute("SELECT count(*) FROM sqlite_master")  # forces key verification
        except sqlcipher3.DatabaseError as exc:
            conn.close()
            raise EncryptedDatabaseError(
                f"could not open encrypted database {Path(path).name}: wrong key, or not an
"
                "encrypted muneem ledger"
            ) from exc
        conn.row_factory = sqlcipher3.Row
        return conn


    @contextmanager
    def connect(db_path: Path | str, *, key: str | None = None) ->
Iterator[sqlite3.Connection]:
        """Open a connection (commit on success, always close). Parent dirs created.

        With ``key`` (a 64-hex-char SQLCipher key) the database is **encrypted at rest** via
        SQLCipher. Without it, a plaintext stdlib :mod:`sqlite3` connection is used ? the
path for
        ``:memory:`` and tests. A keyed open never silently falls back to plaintext (see
        :func:`_open_encrypted`); the production ledger is opened via
:func:`connect_encrypted`.
        """
        path = str(db_path)
        if path != ":memory:":
            Path(path).parent.mkdir(parents=True, exist_ok=True)
        if key is not None:
            conn = _open_encrypted(path, key)
            _harden_permissions(path)
        else:
            conn = sqlite3.connect(path)
            conn.row_factory = sqlite3.Row
        conn.execute("PRAGMA foreign_keys = ON")
        try:
            yield conn
            conn.commit()
        finally:
            conn.close()


    @contextmanager
    def connect_encrypted(db_path: Path | str) -> Iterator[sqlite3.Connection]:
        """Open the real on-disk ledger, encrypted with the keychain-managed key.

        The production path: resolves (or mints, on first use) the 256-bit key from the OS
        keychain and opens with SQLCipher. If ``db_path`` is a pre-existing *plaintext*
SQLite
        file, refuses with a clear pointer to ``muneem db encrypt`` rather than a cryptic
        "file is not a database" error.
        """
        from muneem.db_key import get_or_create_key

        path = str(db_path)
        if is_plaintext_sqlite(path):
            raise EncryptedDatabaseError(
                f"{Path(path).name} is an unencrypted SQLite database; run 'muneem db
encrypt' "
                "to migrate it to an encrypted ledger"
            )
        with connect(path, key=get_or_create_key()) as conn:
            yield conn

```

```
210
211     def encrypt_database(db_path: Path | str, key: str) -> int:
212         """Migrate a plaintext SQLite ledger to an encrypted one in place, via
``sqlcipher_export``.
213
214         The original is preserved as ``<name>.plaintext.bak`` for the caller to verify and
then
215         securely delete (it still holds the cleartext data). Returns the number of user
tables
216         copied. Raises :class:`EncryptedDatabaseError` if ``db_path`` is not a plaintext
SQLite file.
217         """
218         import sqlcipher3
219
220         path = Path(db_path)
221         if not is_plaintext_sqlite(path):
222             raise EncryptedDatabaseError(
223                 f"{path.name} is not a plaintext SQLite database (already encrypted or
missing)"
224             )
225
226         enc_tmp = path.with_name(path.name + ".enc.tmp")
227         enc_tmp.unlink(missing_ok=True)
228         try:
229             src = sqlcipher3.connect(str(path))  # open the plaintext source (no key)
230             try:
231                 src_tables = src.execute(
232                     "SELECT count(*) FROM sqlite_master WHERE type = 'table'"
233                 ).fetchone()[0]
234                 target = str(enc_tmp).replace("\\", "/")  # SQL string literal: forward
slashes
235                 src.execute(f"ATTACH DATABASE '{target}' AS encrypted KEY \"x'{key}'\"")
236                 src.execute("SELECT sqlcipher_export('encrypted')")
237                 src.execute("DETACH DATABASE encrypted")
238             finally:
239                 src.close()
240
241             # Verify the encrypted copy BEFORE the destructive swap: it must open with the
key, pass
242             # an integrity check, and carry every table across. (The plaintext .bak is kept
either way,
243             # so a failed check is recoverable ? but we refuse to swap in a bad file.)
244             with connect(str(enc_tmp), key=key) as check:
245                 integrity = check.execute("PRAGMA integrity_check").fetchone()[0]
246                 ntables = check.execute(
247                     "SELECT count(*) FROM sqlite_master WHERE type = 'table'"
248                 ).fetchone()[0]
249             if integrity != "ok" or ntables != src_tables:
250                 raise EncryptedDatabaseError(
251                     f"encryption migration verification failed for {path.name} "
252                     f"(integrity={integrity!r}, tables {ntables}/{src_tables}); original
left untouched"
253                 )
254
255             backup = path.with_name(path.name + ".plaintext.bak")
256             backup.unlink(missing_ok=True)
257             path.rename(backup)
258             enc_tmp.rename(path)  # consumed on success
259             _harden_permissions(str(path))
260         finally:
261             # Never leave a partial temp behind: no-op on success (already renamed), cleans
up on error.
262             enc_tmp.unlink(missing_ok=True)
263         return int(ntables)
264
```

```
265
266     def rekey_database(db_path: Path | str, current_key: str, new_key: str) -> None:
267         """Re-encrypt the ledger under ``new_key`` in place (SQLCipher ``PRAGMA rekey``).
268
269         Opens with ``current_key`` first (which verifies it), then rekeys. Raises
270         :class:`EncryptedDatabaseError` if ``current_key`` is wrong or the file is not an
encrypted
271         muneem ledger. The caller is responsible for updating the keychain entry to
``new_key`` only
272         after this returns successfully.
273         """
274         conn = _open_encrypted(str(db_path), current_key)  # validates current_key
275         try:
276             conn.execute(f"PRAGMA rekey = \"x'{new_key}'\"")
277             conn.commit()
278         finally:
279             conn.close()
280         _harden_permissions(str(db_path))
281
282
283     def backup_database(db_path: Path | str, dest: Path | str, key: str) -> None:
284         """Write an **encrypted** online-backup copy of the ledger to ``dest`` (same key).
285
286         Uses the SQLite Online Backup API for a consistent snapshot ? safe even with WAL
active, so no
287         naive file copy and no manual checkpoint needed. The destination is keyed with the
same key, so
288         it stays encrypted at rest. Raises :class:`EncryptedDatabaseError` if ``key`` is
wrong.
289         """
290         import sqlcipher3
291
292         dest_path = Path(dest)
293         dest_path.parent.mkdir(parents=True, exist_ok=True)
294         dest_path.unlink(missing_ok=True)
295         source = _open_encrypted(str(db_path), key)  # validates key
296         try:
297             target = sqlcipher3.connect(str(dest_path))
298             try:
299                 target.execute(_raw_key_pragma(key))  # key the destination so the backup is
encrypted
300                 source.backup(target)  # consistent snapshot of the live DB
301             finally:
302                 target.close()
303         finally:
304             source.close()
305         _harden_permissions(str(dest_path))
306
307
308     def init_db(conn: sqlite3.Connection) -> None:
309         """Bring the schema up to date via the migration runner (creates tables on a fresh
DB).
310
311         The current ``_SCHEMA`` is the frozen *baseline* (version ``SCHEMA_VERSION``);
future changes
312         are forward migrations in :mod:`muneem.migrations`, recorded in
``schema_migrations``.
313         """
314         from muneem import migrations
315
316         migrations.run_migrations(conn, baseline_sql=_SCHEMA,
baseline_version=SCHEMA_VERSION)
317
318
319     def get_document_by_sha(conn: sqlite3.Connection, sha256: str) -> sqlite3.Row | None:
```

62. user (2026-06-10T16:46:33.267Z)

```
380         else:
381             state = "encrypted (SQLCipher)"
382         typer.echo(f"ledger : {db_path}")
383         typer.echo(f"state  : {state}")
384         typer.echo(f"key    : {key_state}")
385
386
387     @db_app.command("encrypt")
388     def db_encrypt(
389         db: Path | None = typer.Option(None, help="SQLite path (defaults to the local data
dir)."),
390     ) -> None:
391         """Migrate an existing *plaintext* ledger to an encrypted one (SQLCipher), in
place."""
392         from muneem import db as dbmod
393         from muneem.db_key import get_or_create_key
394
395         db_path = db or load_config().db_path
396         if not Path(db_path).exists():
397             typer.echo(
398                 f"nothing to encrypt: {db_path} does not exist (a new ledger is born
encrypted)."
399             )
400             return
401         if not dbmod.is_plaintext_sqlite(db_path):
402             typer.echo(
403                 f"{Path(db_path).name} is already encrypted (or not a SQLite file); nothing
to do."
404             )
405             return
406         n = dbmod.encrypt_database(db_path, get_or_create_key())
407         typer.echo(
408             f"encrypted {Path(db_path).name}: {n} table(s) migrated. A plaintext backup
remains at "
409             f"{Path(db_path).name}.plaintext.bak ? verify, then securely delete it."
410         )
411
412
413     def _require_encrypted_ledger(db_path: Path) -> str:
414         """Return the ledger's keychain key, or exit with a clear message if it can't be
used."""
415         from muneem import db as dbmod
416         from muneem.db_key import peek_key
417
418         if not Path(db_path).exists() or dbmod.is_plaintext_sqlite(db_path):
419             typer.echo("no encrypted ledger here (run 'muneem db encrypt' first).",
err=True)
420             raise typer.Exit(1)
421         key = peek_key()
422         if not key:
423             typer.echo("no encryption key in the keychain for this ledger.", err=True)
424             raise typer.Exit(1)
425         return key
426
427
428     @db_app.command("rotate-key")
429     def db_rotate_key(
430         db: Path | None = typer.Option(None, help="SQLite path (defaults to the local data
dir)."),
431     ) -> None:
432         """Rotate the ledger's encryption key (SQLCipher rekey) and update the keychain."""
433         from muneem import db as dbmod
434         from muneem import db_key
```

```
435
436         db_path = db or load_config().db_path
437         current = _require_encrypted_ledger(db_path)
438         fresh = db_key.new_key()
439         # Keychain-first with rollback: if the file rekey fails, the keychain still matches
the file.
440         db_key.set_key(fresh)
441         try:
442             dbmod.rekey_database(db_path, current, fresh)
443         except dbmod.EncryptedDatabaseError as exc:
444             db_key.set_key(current)  # roll back; the file is still under the old key
445             typer.echo(str(exc), err=True)
446             raise typer.Exit(1) from exc
447         typer.echo(f"rotated the encryption key for {Path(db_path).name}.")
448
449
450     @db_app.command("backup")
451     def db_backup(
452         dest: Path = typer.Argument(..., help="Destination path for the encrypted backup
copy."),
453         db: Path | None = typer.Option(None, help="SQLite path (defaults to the local data
dir)."),
454     ) -> None:
455         """Write an encrypted online-backup copy of the ledger (consistent even with WAL
active)."""
456         from muneem import db as dbmod
457
458         db_path = db or load_config().db_path
459         key = _require_encrypted_ledger(db_path)
460         try:
461             dbmod.backup_database(db_path, dest, key)
462         except dbmod.EncryptedDatabaseError as exc:
463             typer.echo(str(exc), err=True)
464             raise typer.Exit(1) from exc
465         typer.echo(f"backed up {Path(db_path).name} to {dest} (encrypted).")
466
467
468     @db_app.command("setup-recovery")
469     def db_setup_recovery(
470         out: Path | None = typer.Option(
471             None, help="Recovery file path (default: <data dir>/muneem-recovery.json)."
472         ),
473     ) -> None:
474         """Create a passphrase-wrapped recovery file for the ledger's encryption key.
475
476         Lets you restore the key (and thus the ledger + backups) if the OS keychain is ever
lost. The
477         file holds only the *wrapped* key ? useless without the passphrase.
478         """
479         import getpass
480
481         from muneem import recovery
482         from muneem.db_key import peek_key
483
484         key = peek_key()
485         if not key:
486             typer.echo("no encryption key in the keychain yet (create the ledger first).",
err=True)
487             raise typer.Exit(1)
488         out_path = out or (load_config().data_dir / "muneem-recovery.json")
489         passphrase = getpass.getpass("New recovery passphrase: ")
490         if passphrase != getpass.getpass("Confirm passphrase: "):
491             typer.echo("passphrases did not match.", err=True)
492             raise typer.Exit(1)
493         try:
```

```
494             recovery.write_recovery_file(out_path, key, passphrase)
495         except recovery.RecoveryError as exc:
496             typer.echo(str(exc), err=True)
497             raise typer.Exit(1) from exc
498         typer.echo(
499             f"wrote recovery file to {out_path}. Store it OFF this machine (password manager
/ "
500             "separate drive) and remember the passphrase ? you need BOTH to recover. Without
them, a "
501             "lost keychain means the ledger and its backups are unrecoverable."
502         )
503
504
505     @db_app.command("recover")
506     def db_recover(
507         file: Path = typer.Argument(..., exists=True, dir_okay=False, help="Recovery file
path."),
508         db: Path | None = typer.Option(None, help="SQLite path (defaults to the local data
dir)."),
509     ) -> None:
510         """Restore the ledger's encryption key from a recovery file + passphrase (into the
keychain)."""
511         import getpass
512
513         from muneem import db as dbmod
514         from muneem import db_key, recovery
515
516         passphrase = getpass.getpass("Recovery passphrase: ")
517         try:
518             key = recovery.read_recovery_file(file, passphrase)
519         except recovery.RecoveryError as exc:
520             typer.echo(str(exc), err=True)
521             raise typer.Exit(1) from exc
522         db_path = db or load_config().db_path
523         # If the ledger is present, verify the recovered key actually opens it before
committing it.
524         if Path(db_path).exists() and not dbmod.is_plaintext_sqlite(db_path):
525             try:
526                 with dbmod.connect(db_path, key=key) as conn:
527                     conn.execute("SELECT 1")
528             except dbmod.EncryptedDatabaseError as exc:
529                 typer.echo("recovered a key, but it does not open the current ledger.",
err=True)
530                 raise typer.Exit(1) from exc
531         db_key.set_key(key)
532         typer.echo("recovered the encryption key into the keychain.")
533
534
535     if __name__ == "__main__":
536         app()
537
```

63. user (2026-06-10T16:48:11.980Z)

[Request interrupted by user]